Introduction to Transformers: an NLP Perspective

Tong Xiao xiaotong@mail.neu.edu.cn
NLP Lab., Northeastern University, Shenyang, China
NiuTrans Research, Shenyang, China

Jingbo Zhu zhujingbo@mail.neu.edu.cn
NLP Lab., Northeastern University, Shenyang, China
NiuTrans Research, Shenyang, China

arXiv:2311.17633v2 [cs.CL] 2 Jul 2026

Abstract

Transformers have dominated empirical machine learning models of natural language processing. In this paper, we introduce basic concepts of Transformers and present key techniques that form the recent advances of these models. This includes a description of the standard Transformer architecture, a series of model refinements, and common applications. Given that Transformers and related deep learning techniques might be evolving in ways we have never seen, we cannot dive into all the model details or cover all the technical areas. Instead, we focus on just those concepts that are helpful for gaining a good understanding of Transformers and their variants. We also summarize the key ideas that impact this field, thereby yielding some insights into the strengths and limitations of these models.

arxiv: https://arxiv.org/abs/2311.17633

github: https://github.com/NiuTrans/Introduction-to-Transformers

Contents

1 Background **4**

2 The Basic Model **4**
2.1 The Transformer Architecture 5
2.2 Positional Encoding 8
2.3 Multi-head Self-attention 9
2.4 Layer Normalization 11
2.5 Feed-forward Neural Networks 12
2.6 Decoder-side Attention 13
2.7 Training and Inference 15

3 Syntax-aware Models **17**
3.1 Syntax-aware Input and Output 18
3.2 Syntax-aware Attention Models 19
3.3 Multi-branch Models 22
3.4 Multi-scale Models 25
3.5 Transformers as Syntax Learners 26

4 Improved Architectures **29**
4.1 Locally Attentive Models 30
4.2 Deep Models 34
4.3 Numerical Method-Inspired Models 40
4.4 Wide Models 43

5 Efficient Models **46**
5.1 Sparse Attention 47
5.2 Recurrent and Memory Models 50
5.3 Low-dimensional Models 56
5.4 Parameter and Activation Sharing 61
5.5 Alternatives to Self-Attention 62
5.6 Conditional Computation 69
5.7 Model Transfer and Pruning 74
5.8 Sequence Compression 76
5.9 High Performance Computing Methods 77

6 Applications **80**
6.1 Language Modeling 81
6.2 Text Encoding 82
6.3 Speech Translation 84
6.4 Vision Models 85
6.5 Multimodal Models 88

7 Summary **90**

Appendices **95**

A Sinusoidal Positional Encoding **95**

1. Background

Transformers are a type of neural network (Vaswani et al., 2017). They were originally known for their strong performance in machine translation, and are now a de facto standard for building large-scale self-supervised learning systems (Brown et al., 2020; Devlin et al., 2019). The past few years have seen the rise of Transformers not only in natural language processing (NLP) but also in several other fields, such as computer vision and multi-modal processing. As Transformers continue to mature, these models are playing an increasingly important role in the research and application of artificial intelligence (AI).

Looking back at the history of neural networks, Transformers have not been around for a long time. While Transformers are "newcomers" in NLP, they were developed on top of several ideas, the origins of which can be traced back to earlier work, such as word embedding (Bengio et al., 2003; Mikolov et al., 2013) and attention mechanisms (Bahdanau et al., 2014; Luong et al., 2015). As a result, Transformers can benefit from the advancements of different sub-fields of deep learning, and provide an elegant way to combine these neural models. On the other hand, Transformers are unique, and differ from previous models in several ways. First, they do not depend on recurrent or convolutional neural networks for modeling sequences of words, but use only attention mechanisms and feed-forward neural networks. Second, the use of self-attention in Transformers makes it easier to deal with global contexts and dependencies among words. Third, Transformers are very flexible architectures and can be easily modified to accommodate different tasks.

The widespread use of Transformers motivates the development of cutting-edge techniques in deep learning. For example, there are significant refinements in self-attention mechanisms, which have been incorporated into many state-of-the-art NLP systems. The resulting techniques, together with the progress in self-supervised learning, have led us to a new era of AI: we are beginning to obtain models of universal language understanding, generation and reasoning. This has been evidenced by recent Transformer-based large language models (LLMs) which demonstrate amazing performance across a broad variety of tasks (Bubeck et al., 2023).

This paper provides an introduction to Transformers while reflecting the recent developments in applying these models to different problems. However, Transformers are so successful that there have been numerous related studies and we cannot give a full description of them. Therefore, we focus this work on the core ideas of Transformers, and present a basic description of the common techniques. We also discuss some recent advances in Transformers, such as model improvements for efficiency and accuracy considerations. Because the field is very active and new techniques are coming out every day, it is impossible to survey all the latest literature and we are not attempting to do so. Instead, we focus on just those concepts and algorithms most relevant to Transformers, aimed at the people who wish to get a general understanding of these models.

2. The Basic Model

Here we consider the model presented in Vaswani et al. (2017)'s work. We start by considering the Transformer architecture and then discuss its components in detail.

2.1 The Transformer Architecture

Figure 1 shows the standard Transformer model which follows the standard encoder-decoder framework. A Transformer encoder comprises a number of stacked **encoding layers** (or **encoding blocks**). Each encoding layer has two different sub-layers (or sub-blocks), called the self-attention sub-layer and the feed-forward neural network (FFN) sub-layer. Suppose we have a source-side sequence $\mathbf{x} = x_1 \cdots x_m$ and a target-side sequence $\mathbf{y} = y_1 \cdots y_n$. The input of an encoding layer is a sequence of m vectors $\mathbf{h}_1 \cdots \mathbf{h}_m$, each having d_{model} dimensions (or d dimensions for simplicity). Here we use $\mathbf{H} \in \mathbb{R}^{m \times d}$ to denote these input vectors[1]. The self-attention sub-layer first performs a self-attention operation $\text{Att}_{\text{self}}(\cdot)$ on $\mathbf{H}$ to generate an output $\mathbf{C}$:

$$\mathbf{C} = \text{Att}_{\text{self}}(\mathbf{H}). \tag{1}$$

Here $\mathbf{C}$ is of the same size as $\mathbf{H}$, and can thus be viewed as a new representation of the inputs. Then, a residual connection and a layer normalization unit are added to the output so that the resulting model is easier to optimize.

The original Transformer model employs the **post-norm** structure where a residual connection is created before layer normalization is performed, as follows:

$$\mathbf{H}_{\text{self}} = \text{LNorm}(\mathbf{C} + \mathbf{H}), \tag{2}$$

where the addition of $\mathbf{H}$ denotes the residual connection, and $\text{LNorm}(\cdot)$ denotes the layer normalization function. Substituting Eq. (1) into Eq. (2), we obtain the form of the self-attention sub-layer

$$\begin{aligned} \text{Layer}_{\text{self}}(\mathbf{H}) &= \mathbf{H}_{\text{self}} \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{H}) + \mathbf{H}). \end{aligned} \tag{3}$$

The definitions of $\text{LNorm}(\cdot)$ and $\text{Att}_{\text{self}}(\cdot)$ have been given previously, and we will also discuss them later in the section.

The FFN sub-layer takes $\mathbf{H}_{\text{self}}$ and outputs a new representation $\mathbf{H}_{\text{ffn}} \in \mathbb{R}^{m \times d}$. It has the same form as the self-attention sub-layer, with the attention function replaced by the FFN function, given by

$$\begin{aligned} \text{Layer}_{\text{ffn}}(\mathbf{H}_{\text{self}}) &= \mathbf{H}_{\text{ffn}} \\ &= \text{LNorm}(\text{FFN}(\mathbf{H}_{\text{self}}) + \mathbf{H}_{\text{self}}). \end{aligned} \tag{4}$$

Here $\text{FFN}(\cdot)$ can be any feed-forward neural network with non-linear activation functions. The most common structure of $\text{FFN}(\cdot)$ is a two-layer network involving two linear transformations and a ReLU activation function between them.

For deep models, we can stack the above neural networks. Let $\mathbf{H}^l$ be the output of layer l. Then, we can express $\mathbf{H}^l$ as a function of $\mathbf{H}^{l-1}$. We write this as a composition of two

1. Provided $\mathbf{h}_j \in \mathbb{R}^d$ is a row vector, we have $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}$.

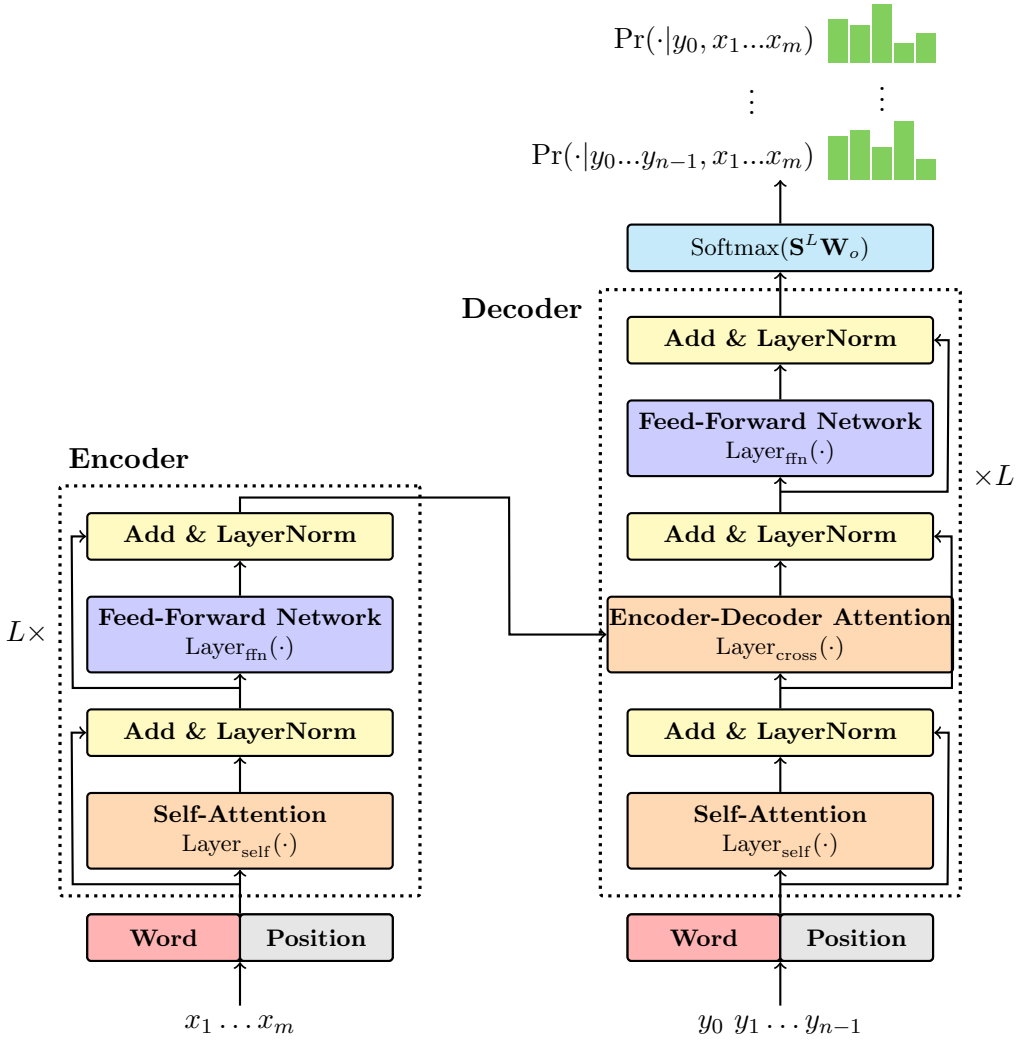


Figure 1: The Transformer architecture (Vaswani et al., 2017). There are L stacked layers on each of the encoder and decoder sides. An encoding layer comprises a self-attention sub-layer and an FFN sub-layer. Both of these sub-layers share the same structure which involves a core function (either $\text{Layer}_{\text{self}}(\cdot)$ or $\text{Layer}_{\text{ffn}}(\cdot)$), followed by a residual connection and a layer normalization unit. Each decoding layer has a similar architecture to the encoding layers, but with an additional encoder-decoder attention sub-layer sandwiched between the self-attention and FFN sub-layers. As with most sequence-to-sequence models, Transformer takes $x_1 \cdots x_m$ and $y_0 \cdots y_{i-1}$ for predicting y_i. The representation of an input word comprises a sum of a word embedding and a positional embedding. The distributions $\{\Pr(\cdot|y_0 \cdots y_{i-1}, x_1 \cdots x_m)\}$ are generated in sequence by a softmax layer, which operates on a linear transformation of the output from the last decoding layer.

sub-layers

$$\mathbf{H}^l = \text{Layer}_{\text{ffn}}(\mathbf{H}^l_{\text{self}}), \quad (5)$$
$$\mathbf{H}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{H}^{l-1}). \quad (6)$$

If there are L encoding layers, then $\mathbf{H}^L$ serves as the final output of the encoder. In this case, $\mathbf{H}^L$ can be viewed as a learned contextual representation of the input sequence. $\mathbf{H}^0$ denotes the initial input to the encoder. In recurrent and convolutional models, $\mathbf{H}^0$ typically consists simply of the word embeddings of the input sequence. The Transformer takes a different approach to representing input words and explicitly encodes positional information. In Section 2.2, we will discuss the embedding model used in the Transformer.

The Transformer decoder has a similar structure to the Transformer encoder, comprising L stacked **decoding layers** (or **decoding blocks**). Let $\mathbf{S}^l$ be the output of the l-th decoding layer. We can formulate a decoding layer using the following equations:

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{cross}}), \quad (7)$$
$$\mathbf{S}^l_{\text{cross}} = \text{Layer}_{\text{cross}}(\mathbf{H}^L, \mathbf{S}^{l-1}_{\text{self}}), \quad (8)$$
$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \quad (9)$$

Here there are three decoder sub-layers. The self-attention and FFN sub-layers are the same as those used in the encoder. $\text{Layer}_{\text{cross}}(\cdot)$ denotes a cross attention sub-layer (or encoder-decoder sub-layer) which models the transformation from the source-side to the target-side. In Section 2.6 we will see that $\text{Layer}_{\text{cross}}(\cdot)$ can be implemented using the same function as $\text{Layer}_{\text{self}}(\cdot)$.

The Transformer decoder outputs a distribution over a vocabulary V_{y} at each target-side position. This is achieved by using a softmax layer that normalizes a linear transformation of $\mathbf{S}^L$ to distributions of target-side words. To do this, we map $\mathbf{S}^L$ to an $n \times |V_{\text{y}}|$ matrix $\mathbf{O}$ by

$$\mathbf{O} = \mathbf{S}^L \cdot \mathbf{W}_{\text{o}}, \quad (10)$$

where $\mathbf{W}_{\text{o}} \in \mathbb{R}^{d \times |V_{\text{y}}|}$ is the parameter matrix of the linear transformation.

Then, the output of the Transformer decoder is given in the form

$$\begin{bmatrix} \Pr(\cdot|y_0, \mathbf{x}) \\ \vdots \\ \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x}) \end{bmatrix} = \text{Softmax}(\mathbf{O})$$
$$= \begin{bmatrix} \text{Softmax}(\mathbf{o}_1) \\ \vdots \\ \text{Softmax}(\mathbf{o}_n) \end{bmatrix}, \quad (11)$$

where $\mathbf{o}_i$ denotes the i-th row vector of $\mathbf{O}$, and y_0 denotes the start symbol $\langle\text{SOS}\rangle$. Under this model, the probability of $\mathbf{y}$ given $\mathbf{x}$ can be defined as usual,

$$\log \Pr(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^{n} \log \Pr(y_i|y_0 \cdots y_{i-1}, \mathbf{x}). \quad (12)$$

This equation resembles the general form of language modeling: we predict the word at position i given all the words up to position $i-1$. Therefore, the target output sequence is shifted one step forward in time relative to the input sequence, that is, the decoder takes $y_0 \cdots y_{n-1}$ as input to predict the output sequence $y_1 \cdots y_n$.

The Transformer architecture discussed above has several variants which have been successfully used in different fields of NLP. For example, we can use a Transformer encoder to represent text (referred to as the **encoder-only architecture**), a Transformer decoder to generate text (referred to as the **decoder-only architecture**), or a standard encoder-decoder Transformer model to transform an input sequence to an output sequence. In the rest of this chapter, much of the discussion is independent of the particular choice of application, and will be primarily focused on the encoder-decoder architecture. In Section 6, we will see applications of the encoder-only and decoder-only architectures.

2.2 Positional Encoding

In their original form, both FFNs and attention models used in Transformers ignore an important aspect of sequence modeling, namely that the order of the words plays a crucial role in expressing the meaning of a sequence. This means that the encoder and decoder are insensitive to the positional information of the input words. A simple approach to overcoming this problem is to add positional encodings to the representation of each word in the sequence. More formally, a word x_j can be represented as a d-dimensional vector

$$\mathbf{e}_j = \mathbf{x}_j + \mathrm{PE}(j). \tag{13}$$

Here $\mathbf{x}_j \in \mathbb{R}^d$ is the word embedding, which can be obtained using standard word embedding models. $\mathrm{PE}(j) \in \mathbb{R}^d$ is the representation of the position j. The vanilla Transformer employs a sinusoidal positional encoding model, which we write in the form

$$\mathrm{PE}(j, 2k) = \sin(j \cdot \frac{1}{10000^{2k/d}}), \tag{14}$$

$$\mathrm{PE}(j, 2k+1) = \cos(j \cdot \frac{1}{10000^{2k/d}}), \tag{15}$$

where $\mathrm{PE}(j, k)$ denotes the k-th entry of $\mathrm{PE}(j)$. The idea of positional encoding is to distinguish different positions using continuous functions. Here we use the sine and cosine functions with different frequencies. Such a method can be interpreted as a positional numeral system. Because the encoding is based on individual positions, it is also called **absolute positional encoding**. In Section 4.1, we will see an improvement to this method.

Once we obtain these embeddings, the sequence $\mathbf{e}_1 \cdots \mathbf{e}_m$ is used as input to the Transformer encoder, that is,

$$\mathbf{H}^0 = \begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_m \end{bmatrix}. \tag{16}$$

Similarly, we can define the input on the decoder side.

2.3 Multi-head Self-attention

The use of self-attention is perhaps one of the most significant advances in sequence-to-sequence models. It attempts to learn and make use of direct interactions between all pairs of inputs. From a representation learning perspective, self-attention models assume that the learned representation at position i (denoted by $\mathbf{c}_i$) is a weighted sum of the inputs over the sequence. The output $\mathbf{c}_i$ is thus given by

$$\mathbf{c}_i = \sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j, \tag{17}$$

where $\alpha_{i,j}$ represents the attention weight assigned to $\mathbf{h}_j$ when computing the representation at position i. We can thus view $\mathbf{c}_i$ as a representation of the global context at position i. $\alpha_{i,j}$ can be defined in different ways depending on the specific attention model considered. Here we use the scaled dot-product attention function to compute $\alpha_{i,j}$, as follows

$$\begin{aligned} \alpha_{i,j} &= \text{Softmax}\left(\mathbf{h}_i \mathbf{h}_j^\top / \beta\right) \\ &= \frac{\exp(\mathbf{h}_i \mathbf{h}_j^\top / \beta)}{\sum_{k=1}^{m} \exp(\mathbf{h}_i \mathbf{h}_k^\top / \beta)}, \end{aligned} \tag{18}$$

where the scaling factor β is typically set to $\sqrt{d}$.

Compared with conventional recurrent and convolutional models, an advantage of self-attention models is that they shorten the computational "distance" between two inputs. Figure 2 illustrates the information flow in these models. We can see that, given the input at position i, self-attention models can directly access any other input. By contrast, recurrent and convolutional models might need two or more jumps to see the entire sequence.

(a) RNN (b) CNN ($r = 3$) (c) Self-attention

Figure 2: Information flows in recurrent, convolutional, and self-attention models, shown as arrow lines between positions.

We can adopt a more general perspective on self-attention by using the QKV attention model. Suppose we have a sequence of κ queries $\mathbf{Q} = \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_\kappa \end{bmatrix}$, and a sequence of ψ key-value pairs ($\mathbf{K} = \begin{bmatrix} \mathbf{k}_1 \\ \vdots \\ \mathbf{k}_\psi \end{bmatrix}, \mathbf{V} = \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_\psi \end{bmatrix}$). The output of the model is a sequence of vectors, each

corresponding to a query. The form of the QKV attention is given by

$$\text{Att}_{\text{qkv}}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}})\mathbf{V}. \tag{19}$$

We can write the output of the QKV attention model as a sequence of row vectors

$$\begin{aligned}\mathbf{C} &= \begin{bmatrix}\mathbf{c}_1\\ \vdots \\ \mathbf{c}_\kappa\end{bmatrix}\\ &= \text{Att}_{\text{qkv}}(\mathbf{Q},\mathbf{K},\mathbf{V}).\end{aligned} \tag{20}$$

To apply this equation to self-attention, we simply have

$$\mathbf{H}^q = \mathbf{H}\mathbf{W}^q, \tag{21}$$
$$\mathbf{H}^k = \mathbf{H}\mathbf{W}^k, \tag{22}$$
$$\mathbf{H}^v = \mathbf{H}\mathbf{W}^v, \tag{23}$$

where $\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v \in \mathbb{R}^{d\times d}$ represent linear transformations of $\mathbf{H}$.

By considering Eq. (1), we then obtain

$$\begin{aligned}\mathbf{C} &= \text{Att}_{\text{self}}(\mathbf{H})\\ &= \text{Att}_{\text{qkv}}(\mathbf{H}^q,\mathbf{H}^k,\mathbf{H}^v)\\ &= \text{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}})\mathbf{H}^v.\end{aligned} \tag{24}$$

Here $\text{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}})$ is an $m\times m$ matrix in which each row represents a distribution over all input positions, that is

$$\text{row } i = \begin{bmatrix}\alpha_{i,1} & \cdots & \alpha_{i,m}\end{bmatrix}. \tag{25}$$

We can improve the above self-attention model by using a technique called **multi-head attention**. This method can be motivated from the perspective of learning from multiple lower-dimensional feature subspaces. It projects the input into multiple subspaces and learns separate representations in each. Specifically, we project the entire input space into τ subspaces (referred to as **heads**). For example, we transform $\mathbf{H}\in\mathbb{R}^{m\times d}$ into τ matrices of size $m\times\frac{d}{\tau}$, denoted by $\{\mathbf{H}_1^{\text{head}},\ldots,\mathbf{H}_\tau^{\text{head}}\}$. The attention model is then applied τ times, once for each head. Finally, the outputs of these model runs are concatenated and transformed by a linear projection. This procedure can be expressed by

$$\mathbf{C} = \text{Merge}(\mathbf{C}_1^{\text{head}},\cdots,\mathbf{C}_\tau^{\text{head}})\mathbf{W}_c. \tag{26}$$

For each head h,

$$\mathbf{C}_h^{\text{head}} = \text{Softmax}\left(\frac{\mathbf{H}_h^q[\mathbf{H}_h^k]^\top}{\sqrt{d}}\right)\mathbf{H}_h^v, \tag{27}$$
$$\mathbf{H}_h^q = \mathbf{H}\mathbf{W}_h^q, \tag{28}$$
$$\mathbf{H}_h^k = \mathbf{H}\mathbf{W}_h^k, \tag{29}$$
$$\mathbf{H}_h^v = \mathbf{H}\mathbf{W}_h^v. \tag{30}$$

Here Merge$(\cdot)$ is the concatenation function, and $\text{Att}_{\text{qkv}}(\cdot)$ is the attention function described in Eq. (20). $\mathbf{W}_h^q, \mathbf{W}_h^k, \mathbf{W}_h^v \in \mathbb{R}^{d \times \frac{d}{\tau}}$ are the parameters of the projections from a d-dimensional space to a $\frac{d}{\tau}$-dimensional space for the queries, keys, and values. Thus, $\mathbf{H}_h^q$, $\mathbf{H}_h^k$, $\mathbf{H}_h^v$, and $\mathbf{C}_h^{\text{head}}$ are all $m \times \frac{d}{\tau}$ matrices. Merge$(\mathbf{C}_1^{\text{head}}, \cdots, \mathbf{C}_\tau^{\text{head}})$ produces an $m \times d$ matrix. It is then transformed by a linear mapping $\mathbf{W}_c \in \mathbb{R}^{d \times d}$, leading to the final result $\mathbf{C} \in \mathbb{R}^{m \times d}$.

Although the notation may seem tedious, implementing multi-head models is straightforward using modern deep learning toolkits. A common method in Transformer-based systems is to store inputs from all the heads in data structures called tensors, enabling us to leverage parallel computing resources for greater efficiency. A more general discussion of the QKV attention and multi-head attention models can be found in the literature on encoder-decoder models.

2.4 Layer Normalization

Layer normalization provides a simple and effective means to make the training of neural networks more stable by standardizing the activations of the hidden layers in a layer-wise manner. As introduced by Ba et al. (2016), given a layer's output $\mathbf{h} \in \mathbb{R}^d$, the layer normalization method computes a standardized output $\text{LNorm}(\mathbf{h}) \in \mathbb{R}^d$ by

$$\text{LNorm}(\mathbf{h}) \quad = \quad \boldsymbol{\alpha} \odot \frac{\mathbf{h} - \mu}{\sigma + \epsilon} + \boldsymbol{\beta}. \tag{31}$$

Here $\mu \in \mathbb{R}$ and $\sigma \in \mathbb{R}$ are the scalar mean and standard deviation of the activations, respectively. Let h_k be the k-th dimension of $\mathbf{h}$. The scalars μ and σ are defined as

$$\mu \quad = \quad \frac{1}{d} \sum_{k=1}^{d} h_k, \tag{32}$$

$$\sigma \quad = \quad \sqrt{\frac{1}{d} \sum_{k=1}^{d} (h_k - \mu)^2}. \tag{33}$$

Here $\boldsymbol{\alpha} \in \mathbb{R}^d$ and $\boldsymbol{\beta} \in \mathbb{R}^d$ are the scale and shift parameters. They can be treated as parameters of layer normalization, whose values are learned jointly with other parameters of the Transformer model. The addition of ϵ to σ is used for numerical stability. In general, ϵ is chosen to be a very small number.

We illustrate the layer normalization method for the hidden states of an encoder in the following example (assume that $m = 4$, $d = 3$, $\boldsymbol{\alpha} = \mathbf{1}$, $\boldsymbol{\beta} = \mathbf{0}$, and $\epsilon = 0.1$).

$$\begin{array}{l} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \\ \mathbf{h}_4 \end{array} \begin{bmatrix} 1 & 1 & 2 \\ 0.9 & 0.9 & 0 \\ 0.7 & 0.8 & 0 \\ 3 & 1 & 7 \end{bmatrix} \quad \begin{array}{l} \mu = 1.3,\ \sigma = 0.5 \\ \mu = 0.6,\ \sigma = 0.4 \\ \mu = 0.5,\ \sigma = 0.4 \\ \mu = 3.7,\ \sigma = 2.5 \end{array} \quad \Longrightarrow \quad \begin{bmatrix} \frac{1-1.3}{0.5+0.1} & \frac{1-1.3}{0.5+0.1} & \frac{2-1.3}{0.5+0.1} \\ \frac{0.9-0.6}{0.4+0.1} & \frac{0.9-0.6}{0.4+0.1} & \frac{0-0.6}{0.4+0.1} \\ \frac{0.7-0.5}{0.4+0.1} & \frac{0.8-0.5}{0.4+0.1} & \frac{0-0.5}{0.4+0.1} \\ \frac{3-3.7}{2.5+0.1} & \frac{1-3.7}{2.5+0.1} & \frac{7-3.7}{2.5+0.1} \end{bmatrix}$$

As discussed in Section 2.1, the layer normalization unit in each sub-layer is used to standardize the output of a residual block. Here we describe a more general formulation

for this structure. Suppose that $F(\cdot)$ is a neural network sub-layer. Then, the post-norm structure of $F(\cdot)$ is given by:

$$\mathbf{H}_{\text{out}} \quad = \quad \text{LNorm}(F(\mathbf{H}_{\text{in}}) + \mathbf{H}_{\text{in}}), \tag{34}$$

where $\mathbf{H}_{\text{in}}$ and $\mathbf{H}_{\text{out}}$ are the input and output of this sub-layer. Clearly, Eq. (3) is an instance of this equation.

An alternative approach to incorporating layer normalization and residual connections into the modeling is to execute the $\text{LNorm}(\cdot)$ function right before the $F(\cdot)$ function, and to establish an identity mapping from the input to the output of the entire sub-layer. This structure, known as the **pre-norm** structure, can be expressed in the form

$$\mathbf{H}_{\text{out}} \quad = \quad F(\text{LNorm}(\mathbf{H}_{\text{in}})) + \mathbf{H}_{\text{in}}. \tag{35}$$

Both post-norm and pre-norm Transformer models are widely used in NLP systems. See Figure 3 for a comparison of these two structures. In general, residual connections are considered an effective means to make the training of multi-layer neural networks easier. In this sense, the pre-norm Transformer seems promising because it follows the convention that a residual connection is created to bypass the whole network and that the identity mapping from the input to the output leads to easier optimization of deep models. However, when considering the expressive power of a model, there may be modeling advantages to using the post-norm Transformer, because it does not rely as heavily on residual connections and enforces more sophisticated modeling for representation learning. In Section 4.2, we will present a further discussion on this issue.

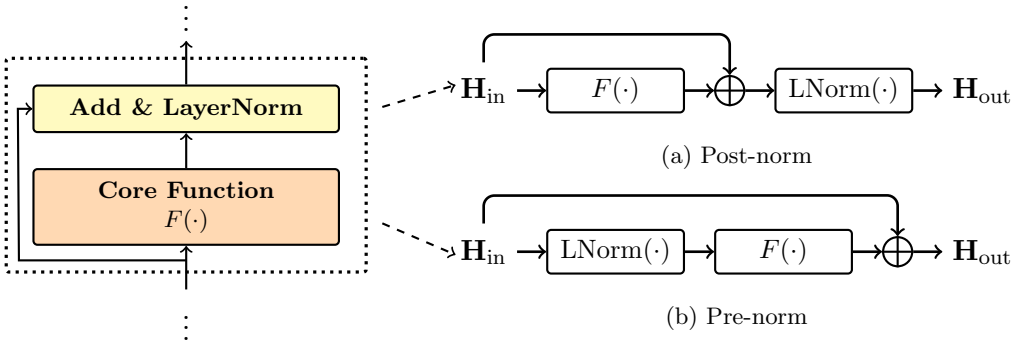


Figure 3: The post-norm and pre-norm structures. $F(\cdot)$ = core function, $\text{LNorm}(\cdot)$ = layer normalization, and $\oplus$ = residual connection.

2.5 Feed-forward Neural Networks

The use of FFNs in Transformers is inspired in part by the fact that complex outputs can be formed by transforming the inputs through non-linearities. While the self-attention model itself has some non-linearity (in $\text{Softmax}(\cdot)$), a more common way to introduce non-linearity is to consider additional layers with non-linear activation functions and linear

Sub-model		# of Parameters	Time Complexity	×
Encoder	Multi-head Self-attention	$4d^2$	$O(m^2 \cdot d)$	L
	Feed-forward Network	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(m \cdot d \cdot d_{\text{ffn}})$	L
	Layer Normalization	$2d$	$O(m \cdot d)$	$2L$
Decoder	Multi-head Self-attention	$4d^2$	$O(n^2 \cdot d)$	L
	Multi-head Cross-attention	$4d^2$	$O(m \cdot n \cdot d)$	L
	Feed-forward Network	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(n \cdot d \cdot d_{\text{ffn}})$	L
	Layer Normalization	$2d$	$O(n \cdot d)$	$3L$

Table 1: Number of parameters and time complexities of different Transformer modules under various setups. m = source-sequence length, n = target-sequence length, d = default number of dimensions of a hidden layer, d_{ffn} = number of dimensions of the FFN hidden layer, τ = number of heads in the attention models, and L = number of encoding or decoding layers. The column × indicates the number of times a sub-model is applied on the encoder or decoder side. The time complexities are estimated by counting the number of floating-point multiplications.

transformations. Given an input $\mathbf{H}_{\text{in}} \in \mathbb{R}^{m \times d}$ and an output $\mathbf{H}_{\text{out}} \in \mathbb{R}^{m \times d}$, the $\mathbf{H}_{\text{out}} = \text{FFN}(\mathbf{H}_{\text{in}})$ function in the Transformer has the following form

$$\mathbf{H}_{\text{out}} = \mathbf{H}_{\text{hidden}}\mathbf{W}_f + \mathbf{b}_f, \tag{36}$$

$$\mathbf{H}_{\text{hidden}} = \text{ReLU}(\mathbf{H}_{\text{in}}\mathbf{W}_h + \mathbf{b}_h), \tag{37}$$

where $\mathbf{H}_{\text{hidden}} \in \mathbb{R}^{m \times d_{\text{ffn}}}$ represents the hidden states, and $\mathbf{W}_h \in \mathbb{R}^{d \times d_{\text{ffn}}}$, $\mathbf{b}_h \in \mathbb{R}^{d_{\text{ffn}}}$, $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}} \times d}$ and $\mathbf{b}_f \in \mathbb{R}^d$ are the parameters. This is a two-layer FFN in which the first layer (or hidden layer) introduces a non-linearity through $\text{ReLU}(\cdot)$[2] and the second layer involves only a linear transformation. It is common practice in the Transformer to use a larger hidden layer size. For example, a common choice is $d_{\text{ffn}} = 4d$, that is, the size of each hidden representation is 4 times that of the input.

Note that using a wide FFN sub-layer has proven to be of great practical value in many state-of-the-art systems. However, a consequence of this is that the model's parameter count is dominated by the FFN. Table 1 shows the number of parameters and time complexities for different modules of a standard Transformer-based system. We see that FFNs dominate the model size when d_{ffn} is large, though they are not the most time-consuming components. In the case of very large Transformer models, we therefore wish to address this problem to build more efficient systems.

2.6 Decoder-side Attention

A decoder layer involves two attention sub-layers: the first is a self-attention sub-layer, and the second is a cross-attention sub-layer. These sub-layers are based on either the post-norm or the pre-norm structure, but differ in how the attention functions are defined. Consider, for example, the post-norm structure described in Eq. (34). We can define the

2. $\text{ReLU}(x) = \max\{0, x\}$.

cross-attention and self-attention sub-layers for a decoding layer to be

$$\begin{aligned}\mathbf{S}_{\text{cross}} &= \text{Layer}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) \\ &= \text{LNorm}(\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) + \mathbf{S}_{\text{self}}), \quad (38)\\ \mathbf{S}_{\text{self}} &= \text{Layer}_{\text{self}}(\mathbf{S}) \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{S}) + \mathbf{S}), \quad (39)\end{aligned}$$

where $\mathbf{S} \in \mathbb{R}^{n\times d}$ is the input to the self-attention sub-layer, $\mathbf{S}_{\text{cross}} \in \mathbb{R}^{n\times d}$ and $\mathbf{S}_{\text{self}} \in \mathbb{R}^{n\times d}$ are the outputs of the sub-layers, and $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{m\times d}$ is the output of the encoder [3].

As with conventional attention models, cross-attention is primarily used to model the correspondence between the source-side and target-side sequences. The $\text{Att}_{\text{cross}}(\cdot)$ function is based on the QKV attention model, which generates an output by querying a collection of key-value pairs. More specifically, we define the queries, keys, and values as linear mappings of $\mathbf{S}_{\text{self}}$ and $\mathbf{H}_{\text{enc}}$, as follows

$$\begin{aligned}\mathbf{S}^q_{\text{self}} &= \mathbf{S}_{\text{self}}\mathbf{W}^q_{\text{cross}}, \quad (40)\\ \mathbf{H}^k_{\text{enc}} &= \mathbf{H}_{\text{enc}}\mathbf{W}^k_{\text{enc}}, \quad (41)\\ \mathbf{H}^v_{\text{enc}} &= \mathbf{H}_{\text{enc}}\mathbf{W}^v_{\text{enc}}, \quad (42)\end{aligned}$$

where $\mathbf{W}^q_{\text{cross}}, \mathbf{W}^k_{\text{enc}}, \mathbf{W}^v_{\text{enc}} \in \mathbb{R}^{d\times d}$ are the parameters of the mappings. In other words, the queries are defined based on $\mathbf{S}_{\text{self}}$, while the keys and values are defined based on $\mathbf{H}_{\text{enc}}$.

$\text{Att}_{\text{cross}}(\cdot)$ is then defined as

$$\begin{aligned}\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) &= \text{Att}_{\text{qkv}}(\mathbf{S}^q_{\text{self}}, \mathbf{H}^k_{\text{enc}}, \mathbf{H}^v_{\text{enc}}) \\ &= \text{Softmax}\left(\frac{\mathbf{S}^q_{\text{self}}[\mathbf{H}^k_{\text{enc}}]^\top}{\sqrt{d}}\right)\mathbf{H}^v_{\text{enc}}. \quad (43)\end{aligned}$$

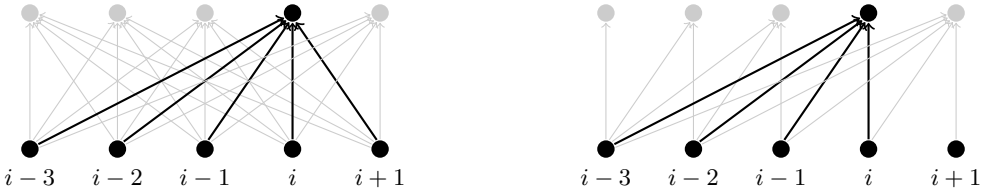
The $\text{Att}_{\text{self}}(\cdot)$ function has a similar form to $\text{Att}_{\text{cross}}(\cdot)$, with linear mappings of $\mathbf{S}$ taken as the queries, keys, and values, like this

$$\begin{aligned}\text{Att}_{\text{self}}(\mathbf{S}) &= \text{Att}_{\text{qkv}}(\mathbf{S}^q, \mathbf{S}^k, \mathbf{S}^v) \\ &= \text{Softmax}(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M})\mathbf{S}^v, \quad (44)\end{aligned}$$

where $\mathbf{S}^q = \mathbf{S}\mathbf{W}^q_{\text{dec}}$, $\mathbf{S}^k = \mathbf{S}\mathbf{W}^k_{\text{dec}}$, and $\mathbf{S}^v = \mathbf{S}\mathbf{W}^v_{\text{dec}}$ are linear mappings of $\mathbf{S}$ with parameters $\mathbf{W}^q_{\text{dec}}, \mathbf{W}^k_{\text{dec}}, \mathbf{W}^v_{\text{dec}} \in \mathbb{R}^{d\times d}$.

This form is similar to that of Eq. (20). However, a key difference compared to encoder self-attention is that the model here must follow the rule of left-to-right generation (see Figure 4). That is, given a target-side word at position i, the model can only see the target-side words in the left context $y_1, \ldots, y_{i-1}$. To enforce this, we add a masking variable $\mathbf{M}$ to the unnormalized weight matrix $\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}}$. Both $\mathbf{M}$ and the weight matrix are of size $n \times n$, so large negative values in $\mathbf{M}$ suppress the corresponding attention scores. To prevent attention to the right context (future words) at step i, $\mathbf{M}$ is defined as:

$$M(i,j) = \begin{cases}0 & i \geq j \\ -\infty & i < j\end{cases}, \quad (45)$$



(a) Encoder-side Self-attention (b) Decoder-side Self-attention

Figure 4: Self-attention on the encoder and decoder sides. Each line connects an input and an output of the self-attention model, indicating a dependency of an output state on an input state. For encoder self-attention, the output at any position is computed by having access to the entire sequence. By contrast, for decoder self-attention, the output at position i is computed by seeing only inputs at positions up to i.

where $M(i,j)$ indicates a bias term for the alignment score between positions i and j. Below we show an example of how the masking variable is applied (assume $n = 4$).

$$\begin{aligned}
& \text{Softmax}\left(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M}\right) \\
= \; & \text{Softmax}\left(\begin{bmatrix} 2 & 0.1 & 1 & 1 \\ 0 & 0.9 & 0.9 & 0.9 \\ 0.2 & 0.8 & 0.7 & 2 \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}\right) \\
= \; & \text{Softmax}\left(\begin{bmatrix} 2 & -\infty & -\infty & -\infty \\ 0 & 0.9 & -\infty & -\infty \\ 0.2 & 0.8 & 0.7 & -\infty \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix}\right) \\
= \; & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.3 & 0.7 & 0 & 0 \\ 0.2 & 0.4 & 0.4 & 0 \\ 0.05 & 0.1 & 0.05 & 0.8 \end{bmatrix}.
\end{aligned} \tag{46}$$

As noted in Section 2.3, it is straightforward to improve these models by using the multi-head attention mechanism. In addition, because decoders are typically the most time-consuming components in practical applications, the overall computational efficiency of these systems depends heavily on optimizing decoder-side attention.

2.7 Training and Inference

Transformers can be trained and used in a standard manner. For example, we can train a Transformer model by performing gradient descent to minimize a loss function on the

3. For an encoder having L encoder layers, $\mathbf{H}_{\text{enc}} = \mathbf{H}^L$.

training data, and test the trained model by performing beam search on unseen data. Below we present some of the techniques that are typically used in the training and inference of Transformer models.

- **Learning Rate Scheduling**. Like standard neural networks, Transformers can be directly trained using back-propagation. The training process is generally iterated many times to make the model fit the training data well. In each training step, we update the weights of the neural networks by moving them a small step in the direction of the negative gradient of the loss function. There are many ways to design the update rule for training. A popular choice is the Adam optimization method (Kingma and Ba, 2014). To adjust the learning rate during training, Vaswani et al. (2017) present a learning rate scheduling strategy that increases the learning rate linearly for a number of steps and then decays it gradually. They design a learning rate of the form

$$\eta \quad = \quad \eta_0 \cdot \min\left\{n_{\text{step}}^{-0.5},\ n_{\text{step}} \cdot (n_{\text{warmup}})^{-1.5}\right\}, \tag{47}$$

  where η_0 denotes the initial learning rate, n_{step} denotes the number of training steps we have executed, and n_{warmup} denotes the number of warmup steps. In the first n_{warmup} steps, the learning rate η grows larger as training proceeds. It reaches its highest value at the point where $n_{\text{step}} = n_{\text{warmup}}$, and then decreases as an inverse square root function (i.e., $\eta_0 \cdot n_{\text{step}}^{-0.5}$).
- **Batching and Padding**. To make a trade-off between global optimization and training convergence, it is common practice to update the weights using a relatively small collection of samples, called a **minibatch**. Therefore, we can consider a batch version of the forward and backward computation processes in which the entire minibatch is used together to obtain the gradient information. One advantage of batching is that it allows the system to leverage efficient tensor operations to process multiple sequences in a single run. This requires all input sequences in a minibatch to be stored in a single memory block so they can be read and processed simultaneously. To illustrate this idea, consider a minibatch containing four samples whose source-sides are:

A	B	C	D	E	F
M	N				
R	S	T			
W	X	Y	Z		

  We can store these sequences in a 4×6 continuous block where each "row" represents a sequence, like this

A	B	C	D	E	F
M	N	□	□	□	□
R	S	T	□	□	□
W	X	Y	Z	□	□

  Here, padding words □ are appended to the shorter sequences so that all sequences are properly aligned in memory. Typically, we do not want padding to affect the operation of the system, so we can simply define □ as a zero vector (referred to as

zero padding). On the other hand, in some cases we are interested in using padding to describe something that is not covered by the input sequences. For example, we can replace padding words with context words from the left (or right) of a sequence, though this may require modifications to the system to ensure that the newly added context words do not cause unintended content to appear in the output.

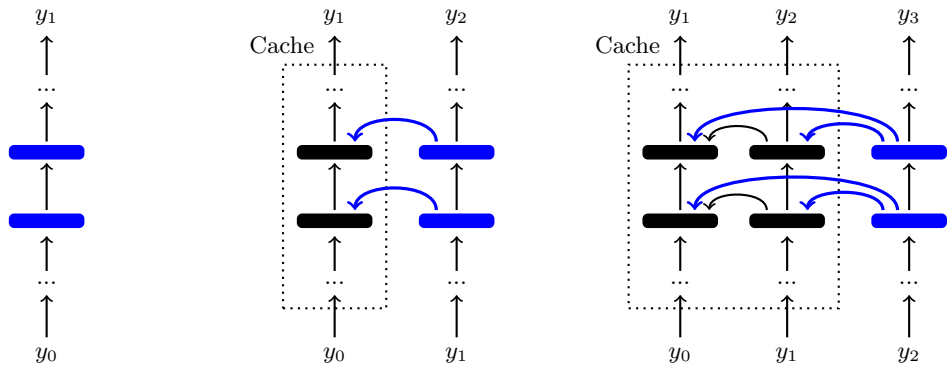
- **Search and Caching**. At test time, we need to search the space of candidate hypotheses (or candidate target-side sequences) to identify the hypothesis with the highest score

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \text{score}(\mathbf{x}, \mathbf{y}), \tag{48}$$

where $\text{score}(\mathbf{x}, \mathbf{y})$ is the model score of the target-side sequence $\mathbf{y}$ given the source-side sequence $\mathbf{x}$. While there are many search algorithms designed to achieve this, most share a similar structure: the search program operates by extending a pool of candidate target-side sequences one token at a time. In this way, the resulting algorithm can be viewed as a left-to-right generation procedure. For a more detailed discussion of search algorithms and model scores of general sequence-to-sequence models, we refer the reader to the literature. Note that all designs for $\text{score}(\mathbf{x}, \mathbf{y})$, no matter how complex, are based on computing $\Pr(\mathbf{y}|\mathbf{x})$. Because the attention models used in Transformers require computing the dot-product of each pair of input vectors in a layer, the time complexity of the search algorithm is a quadratic function of the length of $\mathbf{y}$. It is therefore inefficient to repeatedly compute the outputs of the attention models for previously processed positions. This problem can be addressed by caching the states of each layer for the words we have already seen. Figure 5 illustrates the use of the caching mechanism in a search step. All states for positions $< i$ are maintained and easily accessed in a cache. At position i, all we need is to compute the states for the newly added word and then update the cache.

3. Syntax-aware Models

Although Transformers are deep learning models that do not explicitly make use of linguistic structures or assumptions, it may be necessary to incorporate our prior knowledge into such systems. This is in part because NLP researchers have long believed that a higher level of abstraction in data representation is needed to develop ideal NLP systems, and there have been many systems that use structural priors. However, structure is a broad topic. For a discussion of the various types of structure one might consider, see the work of See (2018). For example, the inductive biases used in our model design can be thought of as some structural prior, and NLP models can also learn the underlying structure of problems by themselves. In this subsection we will discuss some of these issues. We will focus on the methods of introducing linguistic structure into Transformer models. As Transformers can be applied to many NLP tasks, which differ much in their input and output formats, we will primarily discuss modifications to Transformer encoders (call them **syntax-aware Transformer encoders**). Our discussion, however, is general, and these methods can easily be extended to Transformer decoders.



(a) decoding step 1 (b) decoding step 2 (c) decoding step 3

Figure 5: Illustration of the caching mechanism in Transformer decoders. Rectangles indicate the states of decoding layers or sub-layers. At step i, all the states from previous steps are stored in a cache (see dotted boxes), and we only need to compute the states for the current step (see blue rectangles and arrows). We then add the newly generated states to the cache and move on to step $i+1$.

3.1 Syntax-aware Input and Output

One of the simplest methods of incorporating structure into NLP systems is to modify the input sequence, leaving the system unchanged. As a simple example, consider a sentence where each word x_j is assigned a set of κ syntactic labels $\{\text{tag}_j^1, \cdots, \text{tag}_j^\kappa\}$ (e.g., POS labels and dependency labels). We can concatenate these symbols to define a new "word"

$$x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$$

Then, the embedding of this word is given by

$$\mathbf{e}_j \quad = \quad e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) + \text{PE}(j), \tag{49}$$

where $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \in \mathbb{R}^d$ is the embedding of $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$. Since $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$ is a complex symbol, we decompose the learning problem of $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ into easier problems. For example, we can develop κ embedding models, each producing an embedding for a given tag. Then, we write $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ as a sum of the word embedding and tag embeddings

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \quad = \quad \mathbf{x}_j + e(\text{tag}_j^1) + \cdots + e(\text{tag}_j^\kappa), \tag{50}$$

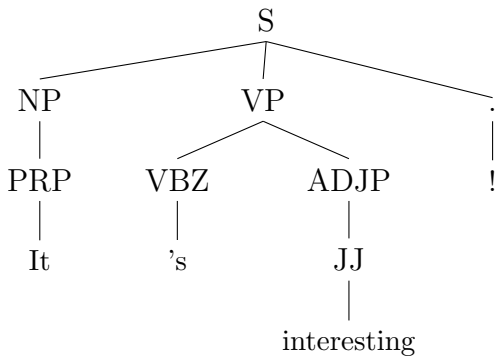
where $\{e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)\}$ are the embeddings of the tags. Alternatively, we can combine these embeddings via a neural network in the form

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \quad = \quad \text{FFN}_{\text{embed}}(\mathbf{x}_j, e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)), \tag{51}$$

where $\text{FFN}_{\text{embed}}(\cdot)$ is a feed-forward neural network that has one or two layers.

We can apply a similar technique on the decoder side, treating $y_i/\text{tag}_i^1/\cdots/\text{tag}_i^\kappa$ as a syntax-augmented target word. However, doing so may drastically increase the size of the target-side vocabulary, posing a computational challenge for both training and inference.

Another common method for representing a sentence is via a syntax tree. In linguistics, the syntax of a sentence can be interpreted in many different ways, resulting in various grammars and corresponding tree-based (or graph-based) representations. While these representations differ in their exact syntactic forms, a general approach for utilizing them in sequence modeling is **tree linearization**. Consider the following sentence annotated with a constituency-based parse tree:



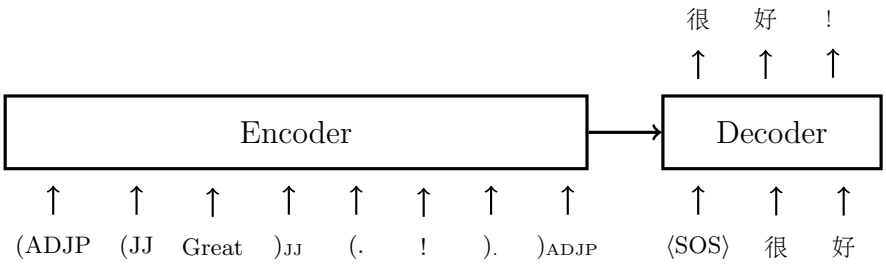
We can write this tree structure as a sequence of words, syntactic labels and brackets via a tree traversal algorithm, as follows

(S	(NP	(PRP	It	$)_{\text{PRP}}$	$)_{\text{NP}}$	(VP	(VBZ	's	$)_{\text{VBZ}}$	(ADJP	(JJ
interesting	$)_{\text{JJ}}$	$)_{\text{ADJP}}$	$)_{\text{VP}}$	(.	!	).	$)_{\text{S}}$				

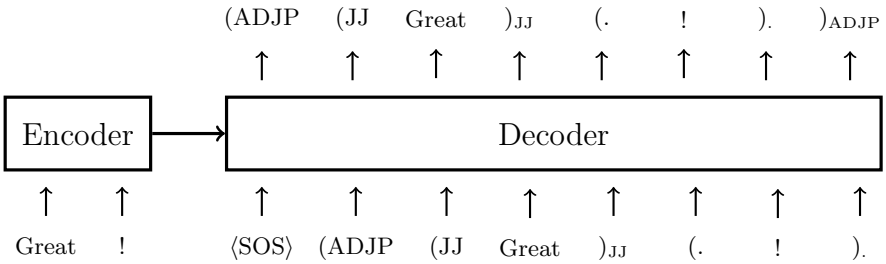
This sequence of syntactic tokens can be used directly as input to the encoder, that is, each token is represented by the sum of its word and positional embeddings, and this combined embedding is treated as a standard input to the encoder. One application of linearized trees is tree-to-string machine translation, in which a syntax tree in one language is translated into a string in another language (Currey and Heafield, 2018; Li et al., 2017). Linearized trees can also be utilized for tree generation. For example, parsing tasks can be framed as sequence-to-sequence problems, mapping an input text to a sequential representation of its corresponding syntax tree (Choe and Charniak, 2016; Vinyals et al., 2015). Figure 6 illustrates these models. It should be noted that the methods described here are not specific to the Transformer and can be applied to many sequence-based architectures, such as RNN-based models.

3.2 Syntax-aware Attention Models

For Transformer models, it is also natural to make use of syntax trees to guide the process of learning sequence representations. In the previous section we saw how representations of a sequence can be computed by relating different positions within that sequence. This allows us to impose structure on the attention distributions over positions. To do this, we



(a) Tree-to-String Machine Translation



(b) Constituency Parsing

Figure 6: Illustration of tree linearization on either the encoder or decoder side. For tree-to-string machine translation, the encoder takes a sequential representation of an input parse tree, and the decoder outputs the corresponding translation. For parsing, the encoder takes a sentence, and the decoder outputs the corresponding syntax tree.

use the encoder self-attention with an additive mask

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \quad = \quad \text{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} + \mathbf{M})\mathbf{H}^v \tag{52}$$

or alternatively with a multiplicative mask

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \quad = \quad \text{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{M})\mathbf{H}^v, \tag{53}$$

where $\mathbf{M} \in \mathbb{R}^{m \times m}$ is a matrix of masking variables in which a larger value of $M(i, j)$ indicates a stronger syntactic correlation between positions i and j. In the following description we choose Eq. (53) as the basic form.

One common way to design $\mathbf{M}$ is to project syntactic relations of the input tree structure into constraints over the sequence. Here we consider constituency parse trees and dependency parse trees for illustration. Generally, two types of masking methods are employed.

- **0-1 Masking**. This method assigns $M(i, j) = 1$ if the words at positions i and j are considered syntactically correlated and $M(i, j) = 0$ otherwise (Bai et al., 2021; Zhang et al., 2020). To model the relation between two words in a syntax tree, we can consider the distance between their corresponding nodes. One of the simplest forms is given by

$$M(i, j) \quad = \quad \begin{cases} 1 & \omega(i, j) \leq \omega_{\max} \\ 0 & \text{otherwise} \end{cases}, \tag{54}$$

  where $\omega(i, j)$ is the shortest path length between the nodes corresponding to the words at positions i and j. For example, given a dependency parse tree, $\omega(i, j)$ is the number of dependency edges in the path between the two words. For a constituency parse tree, all words are leaf nodes. Thus, $\omega(i, j)$ represents the tree distance between the two leaves within the same branch of the tree. $\omega_{\max}$ is a parameter used to control the maximum distance between two nodes that can still be considered syntactically correlated. For example, assuming a dependency parse tree with $\omega_{\max} = 1$, Eq. (54) enforces the constraint that the attention score between positions i and j is computed only if they have a parent-dependent relation[4].

- **Soft Masking**. Instead of treating $\mathbf{M}$ as a hard constraint, we can use it as a soft constraint that scales the attention weight between positions i and j based on the degree to which the corresponding words are correlated. One approach is to reduce the attention weight as $\omega(i, j)$ becomes larger. A simple method is to transform $\omega(i, j)$ such that $M(i, j)$ is negatively correlated with $\omega(i, j)$ and falls within the interval $[0, 1]$:

$$M(i, j) \quad = \quad \text{DNorm}(\omega(i, j)). \tag{55}$$

4. For multiplicative masks, $M(i, j) = 0$ does not mean that the final attention weight between j and i is zero, because an input of zero to the softmax function does not yield a zero probability. A method to strictly "mask out" an entry of Softmax $\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\right)$ is to use an additive mask and set $M(i, j) = -\infty$ if $\omega(i, j) > \omega_{\max}$.

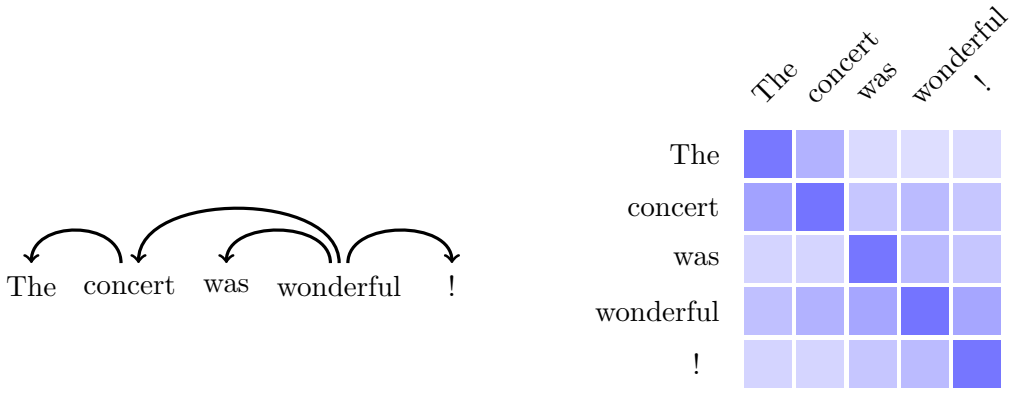
There are several alternative designs for DNorm$(\cdot)$. For example, one can compute a standardized score of $-\omega(i,j)$ by subtracting its mean and dividing by its standard deviation (Chen et al., 2018a), or normalize $1/\omega(i,j)$ over all possible j in the sequence (Xu et al., 2021b). In cases where parsers can output a score between positions i and j, this score can also be used to compute $M(i,j)$. For example, a dependency parser can produce the probability of the word at position i being the parent of the word at position j (Strubell et al., 2018). We can then write $M(i,j)$ as

$$M(i,j) \quad = \quad \Pr_{\text{parent}}(i|j) \tag{56}$$

or alternatively

$$M(i,j) \quad = \quad \max\{\Pr_{\text{parent}}(i|j), \Pr_{\text{parent}}(j|i)\}, \tag{57}$$

where $\Pr_{\text{parent}}(i|j)$ and $\Pr_{\text{parent}}(j|i)$ are the probabilities given by the parser. See Figure 7 for an example of inducing a soft masking variable from a dependency parse tree.



(a) Dependency Parse Tree

(b) Mask **M** (darker color means larger value)

Figure 7: Priors induced from a dependency parse tree. The row i of the matrix **M** represents a distribution that describes how much weight we can give to $M(i,j)$ in terms of the syntactic distance between i and j.

3.3 Multi-branch Models

Integrating syntax into NLP systems presents several challenges. This is partly because automatic parse trees may contain errors, and partly because the use of syntax may lead to strong assumptions about the underlying structure of a sentence. Rather than combining syntactic and word information into a single monolithic model, it may be more flexible and effective to build one model to encode syntactic information and another to encode word sequences. One way to achieve this is through the use of multiple neural networks (referred to as **branches** or **paths**), each dealing with one type of input. The outputs of these

branches are then combined to produce a final output (Fan et al., 2020; Lin et al., 2022b; Xie et al., 2017). Various methods have therefore been used to combine different types of input for neural models like Transformers.

One commonly used approach is to build two separate encoders, in which one model is trained to encode the syntactic input (denoted by $\mathbf{t}$), and the other is trained to encode the standard textual input (denoted by $\mathbf{x}$). Figure 8(a) illustrates this multi-encoder architecture. The syntactic encoder $\text{Encode}_{\text{syn}}(\mathbf{t})$ is based on models presented in Sections 3.1 and 3.2, and the text encoder $\text{Encode}_{\text{text}}(\mathbf{x})$ is a standard Transformer encoder. The representations generated by these encoders are then fed into the combination model as input and combined into a hybrid representation, given by

$$\begin{aligned}\mathbf{H}_{\text{hybrid}} &= \text{Combine}(\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}) \\ &= \text{Combine}(\text{Encode}_{\text{syn}}(\mathbf{t}), \text{Encode}_{\text{text}}(\mathbf{x})).\end{aligned} \tag{58}$$

There are several designs for $\text{Combine}(\cdot)$, depending on the tasks to which the encoders are applied. For example, if we want to develop a text classifier, $\text{Combine}(\cdot)$ can be a simple pooling network. For more complicated tasks, such as machine translation, $\text{Combine}(\cdot)$ can also be a Transformer encoder, and we can fuse information from different sources by performing self-attention on $[\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}]$.

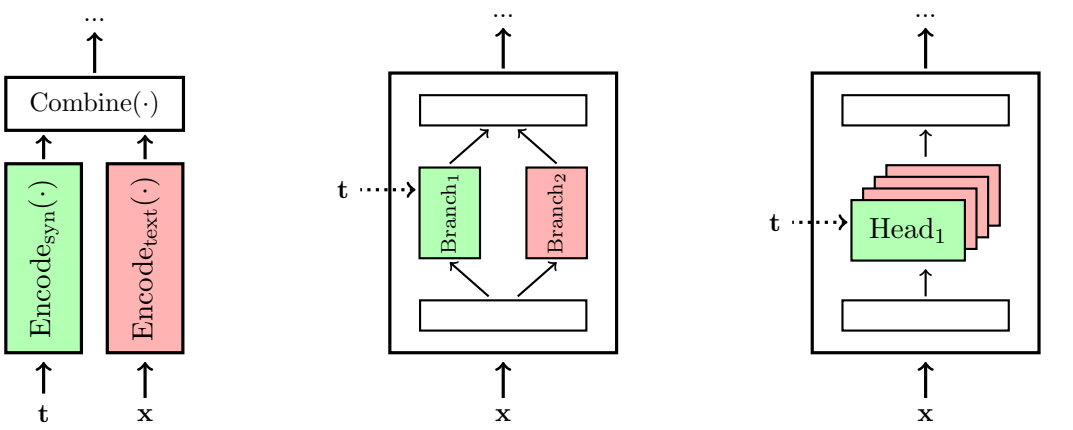
Although we focus primarily on syntactic models in this section, the general multi-encoder architecture can be used in many tasks where inputs from additional sources are required. For example, one can use one encoder to represent a sentence and another to represent the previous sentence in the same document. Combining the two encoders thus yields a context-aware model (Li et al., 2020a; Voita et al., 2018). Furthermore, the architectures of the encoders do not need to be restricted to Transformers. We can choose different models for different branches. For instance, in a widely used two-branch encoding architecture, we can employ a CNN-based encoder to model local context and a Transformer encoder to model global context (Wu et al., 2020).

Sub-models of Transformers can also be multi-branch neural networks. See Figure 8 (b) for an example involving two self-attention branches. One is the standard self-attention network $\text{Att}_{\text{self}}(\mathbf{H})$, and the other is the syntax-aware self-attention network $\text{AttSyn}_{\text{self}}(\mathbf{H})$. The output of the self-attention model is a linear combination of the outputs of these two branches (Xu et al., 2021b), given by

$$\mathbf{H}_{\text{self}} = \alpha \cdot \text{Att}_{\text{self}}(\mathbf{H}) + (1 - \alpha) \cdot \text{AttSyn}_{\text{self}}(\mathbf{H}), \tag{59}$$

where α is a combination coefficient. $\mathbf{H}_{\text{self}}$ can be used as usual by applying a layer normalization function and adding a residual connection, thus keeping the overall architecture identical to standard Transformer models.

Multi-head attention networks can also be viewed as a form of multi-branch models. Therefore, we can provide guidance from syntax to only some of the heads while keeping the rest unchanged (Strubell et al., 2018). This approach is illustrated in Figure 8 (c) where only one head of the self-attention sub-layer makes use of syntax trees for computing attention weights.



(a) Multi-encoder (b) Multi-branch as a Sub-model (c) Multi-head Attention

Figure 8: Multi-branch architectures. There are two inputs: a sentence (denoted by $\mathbf{x}$) and the syntax tree of the sentence (denoted by $\mathbf{t}$). In the multi-encoder architecture (see sub-figure (a)), two encoders are constructed to encode $\mathbf{x}$ and $\mathbf{t}$, respectively. A combination model then takes the outputs of the encoders and produces a combined representation of $\mathbf{x}$ and $\mathbf{t}$. The idea of multi-branch networks can be used for designing sub-models of the encoder. A simple example is that we create multiple paths in parallel for some layers of the encoder (see sub-figure (b)). Another example is multi-head attention (see sub-figure (c)) where we use different heads to learn different representations.

3.4 Multi-scale Models

In linguistics, syntax studies how sentences are built from smaller constituents. Different levels of these constituents are generally organized in a hierarchical structure, called a **syntactic hierarchy**. It is therefore possible to use multiple levels of syntactic constituents to explain the same sentence. For example, words capture how the sentence is constructed from small meaningful units, and phrases capture how the sentence is constructed from larger linguistic units.

Multi-scale Transformers leverage different levels of abstraction in data to represent a sentence using features at multiple scales. A common approach is to write a sentence in multiple different forms and then combine them using a multi-branch network (Hao et al., 2019). For example, consider the following sentence:

The oldest beer-making facility was discovered in China.

We can tokenize it into a sequence of words, denoted by

$$\mathbf{x}_{\text{words}} = \text{The oldest beer-making facility was discovered in China .}$$

Alternatively, we can write it as a sequence of phrases by using a parser, denoted by

$$\mathbf{x}_{\text{phrases}} = [\text{The oldest beer-making facility}]_{\text{NP}}\ [\text{was discovered in China}]_{\text{VP}}\ [.].$$

The simplest way to build a multi-scale model is to encode $\mathbf{x}_{\text{words}}$ and $\mathbf{x}_{\text{phrases}}$ using two separate Transformer encoders. The outputs of these encoders are then combined in some way. This leads to the same form as Eq. (58), and we can view this model as an instance of the general multi-encoder architecture.

Both $\mathbf{x}_{\text{words}}$ and $\mathbf{x}_{\text{phrases}}$ can be viewed as sequences of tokens. For example, $\mathbf{x}_{\text{words}}$ has nine word-based tokens, and $\mathbf{x}_{\text{phrases}}$ has three phrase-based tokens[5]. However, enumerating all possible phrases would result in a huge vocabulary. We therefore need a method to represent each phrase as an embedding in a computationally efficient manner. By treating phrase embedding as a sequence modeling problem, it becomes straightforward to learn sub-sequence representations by employing various sequence models. We now have a two-stage learning process. In the first stage, we learn the embeddings of input units at different scales using separate models. In the second stage, we learn to encode sequences at different scales using a multi-branch model.

More generally, we do not need to restrict ourselves to linguistically meaningful units in multi-scale representation learning. For example, we can learn sub-word segmentations from data and represent an input sentence as a sequence of sub-words. This results in a hierarchical representation of the sentence (e.g., sub-words $\rightarrow$ words $\rightarrow$ phrases). While the learned sub-words may not have linguistic meanings, they provide new insights into modeling words and phrases, as well as a new scale of features. Also, we do not strictly need to develop multiple encoders for multi-scale modeling. An alternative approach is

5. $\mathbf{x}_{\text{phrases}}$ comprises three tokens: *The oldest beer-making facility*, *was discovered in China*, and .

to incorporate representations at different scales directly into the multi-head self-attention modules, which makes it easier to model the interactions among different scales (Guo et al., 2020; Li et al., 2022b).

A problem with the approaches described above, however, is that the representations (or attention weight matrices) learned at different scales are of different sizes. For example, in the scenarios above, the representation learned from $\mathbf{x}_{\text{words}}$ is a $9 \times d$ matrix, while the representation learned from $\mathbf{x}_{\text{phrases}}$ is a $3 \times d$ matrix. A simple solution to this problem is to perform upsampling on the phrase-based representation to expand it to a $9 \times d$ matrix. Likewise, we can perform downsampling on the word-based representation to shrink it to a $3 \times d$ matrix. The combination model Combine$(\cdot)$ can then be implemented as described in Section 3.3.

It is worth noting that multi-scale modeling is widely discussed across several fields. In computer vision, for example, multi-scale modeling is often referred to as the process of learning a series of feature maps from the input image (Fan et al., 2021; Li et al., 2022e). Unlike the multi-branch models presented here, multi-scale vision Transformer models make use of the hierarchical nature of features when representing images. Systems of this kind are typically based on a stack of layers in which each layer learns features at a larger scale (e.g., a higher channel capacity) from the features produced by the previous layer.

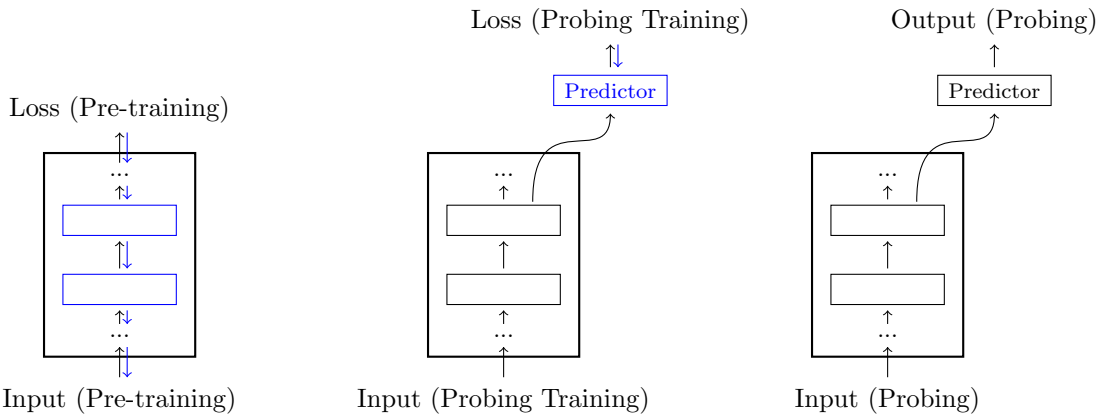
3.5 Transformers as Syntax Learners

So far we have discussed syntax trees as constraints or priors on the encoding process so that we can make use of linguistic representations in learning neural networks. It is natural to wonder whether these neural models can learn some knowledge of linguistic structure directly from data, without human-designed linguistic annotations. This reflects a fundamental goal in NLP: to learn linguistic knowledge from data and encode it implicitly within models.

To explore the linguistic properties learned by NLP systems, a straightforward method is to examine the syntactic behaviors of the systems' outputs. For example, we can examine whether the outputs of language generation systems contain grammatical errors. Another approach is to ask these systems to accomplish tasks that are linguistically meaningful, even though they were not explicitly trained to do so (Brown et al., 2020). However, merely examining how model predictions exhibit syntactic abilities is not sufficient to fully answer the question. It is also possible that neural networks learn linguistic knowledge but do not utilize it during prediction (Clark et al., 2019). Therefore, we need to investigate what is modeled and learned inside the neural networks themselves.

One approach to examining the latent linguistic structure in Transformer models is to develop **probes** to determine whether and to what extent these models capture linguistic notions such as dependency relations and parts-of-speech. A general probing approach involves extracting the internal representations of the models and probing them for linguistic phenomena. For Transformers, this is usually achieved by examining the attention maps and/or the outputs of attention layers. We then construct a **probing predictor** (or **probing classifier**) that takes these internal representations as input and outputs the corresponding linguistic notions (Belinkov, 2022). The probing predictor can be based on either simple heuristics or parameterized models optimized specifically for the probing task.

Recent work has shown that large-scale Transformer-based language models exhibit strong capabilities, referred to as **emergent abilities**, across various probing tasks. Figure 9 illustrates this probing process.



(a) Training the Transformer Model (b) Training the Probing Predictor (c) Probing on New Data

Figure 9: An overview of probing for Transformer-based models. Given a Transformer model (e.g., a Transformer-based language model), we first optimize the model parameters on some unlabeled data. Then, we develop a predictor which takes the states of a hidden layer of the Transformer model and generates outputs for a probing task (see sub-figure (a)). The predictor can be trained as usual in which only the parameters of the predictor are optimized and the parameters of the Transformer model are fixed (see sub-figure (b)). The Transformer model and the predictor are used together to make predictions on new data for probing (see sub-figure (c)).

Many probing methods have been used in recent work on analyzing and understanding what is learned in neural encoders. Here we describe some of the popular ones.

- **Trees**. Given a trained Transformer encoder, it is easy to determine how likely it is that two words in a sentence share a linguistic relationship by computing the attention weight between them. We can use this quantity to define a metric measuring the syntactic distance between two words at positions i and j

$$d_s(i,j) \quad = \quad 1 - \alpha(i,j). \tag{60}$$

  By using this metric, it is straightforward to construct the **minimum-spanning tree** for the sentence. That is, we connect all the words to form a tree structure with the minimum total distance. This tree structure can be viewed as a latent tree representation of the sentence induced from the neural network. While this dependency-tree-like structure can serve as a source of learned syntactic information for downstream tasks, it says little about the model's actual knowledge of syntax. An

approach to aligning the representations in the encoder with linguistic structure is to learn to produce syntax trees that are consistent with human annotations. To achieve this, we need to develop a probing predictor trained on tree-annotated data. Suppose we have a human-annotated dependency tree for a given sentence. For each pair of words, we can obtain a distance $\omega(i, j)$ by counting the number of edges between them. Then, we can learn a distance metric based on the internal representations of the encoder to approximate $\omega(i, j)$. A simple form of such a metric relies on the Euclidean distance (Manning et al., 2020). Let $\mathbf{A} \in \mathbb{R}^{d \times k_s}$ be a parameter matrix. The squared Euclidean distance is given by

$$d_s^2(i, j) \quad = \quad \|(\mathbf{h}_i - \mathbf{h}_j)\mathbf{A}\|_2^2, \tag{61}$$

where $\mathbf{h}_i$ and $\mathbf{h}_j$ are the representations produced by an encoding layer at positions i and j[6]. Given a set of tree-annotated sentences S, we can optimize the model by

$$\hat{\mathbf{A}} \quad = \quad \arg\min_{\mathbf{A}} \sum_{s \in S} \frac{1}{|s|^2} \sum_{i \in s, j \in s} |\omega(i, j) - d_s^2(i, j)|, \tag{62}$$

where $|s|$ is the length of the sentence s, and (i, j) indicates a pair of words in s. The optimized model is then used to parse test sentences via the minimum-spanning tree algorithm, and we can compare the parse trees against the human-annotated trees. To obtain directed trees, which are standard forms of dependency syntax, one can update the above model by considering the relative distance of a word to the root. More details can be found in Manning et al. (2020)'s work. Here the probing predictor functions similarly to a neural parser, trained to predict a syntax tree based on a representation of the input sentence. This idea can be extended to other forms of syntactic structure, such as phrase structure trees (Shi et al., 2016).

- **Syntactic and Semantic Labels**. Many syntactic and semantic parsing tasks can be framed as problems of predicting linguistic labels given a sentence or its segments. A simple example is part-of-speech tagging in which each word of a sentence is labeled with a word class. A probe for part-of-speech tagging can be a classifier that takes a representation $\mathbf{h}_j$ each time and outputs the corresponding word class. One general probing approach to these problems is **edge probing** (Tenney et al., 2019a,b). Given a sentence, a labeled edge is defined as a tuple

$$(\text{span}_1, \text{span}_2, \text{label}),$$

where span_1 is a span $[i_1, j_1]$, and span_2 is an optional second span $[i_2, j_2]$, and label is the corresponding label. Our goal is to learn a probe to predict label given span_1 and span_2. For example, for part-of-speech tagging, span_1 is a unit span $[j, j]$ for each position j, span_2 is an empty span, and label is the part-of-speech tag corresponding to the j-th word of the sentence. For dependency parsing and coreference resolution, span_1 and span_2 are two words or entities, and label is the relationship between them. For constituency parsing, span_1 is a span of words, span_2 is an empty span, and label

6. In general, $\mathbf{h}_i$ and $\mathbf{h}_j$ are the outputs of the last layer of the encoder. Alternatively, they can be weighted sums of the outputs of all the layers.

is the syntactic category of the tree node yielding span_1. In simple cases, the probing model can be a multi-layer feed-forward neural network with a softmax output layer. As usual, this model is trained on labeled data, and then tested on new data.

- **Surface Forms of Words and Sentences**. Probing tasks can also be designed to examine whether the representations encode the surface information of sentences or words (Adi et al., 2016; Conneau et al., 2018). A simple sentence-level probing task is **sentence length prediction**. To do this, we first represent the sentence as a single vector $\mathbf{h}$[7], and then build a classifier to categorize $\mathbf{h}$ into the corresponding length bin. Similarly, probes can be built to predict whether two words at positions i and j are reordered in the sentence given $\mathbf{h}_i$ and $\mathbf{h}_j$. Also, we can develop probes to address conventional problems in morphology. For example, we reconstruct the word at position j or predict its sense with the representation $\mathbf{h}_j$. In addition, probing tasks can be focused on particular linguistic problems, for example, numeracy (Wallace et al., 2019) and function words (Kim et al., 2019).
- **Cloze**. Of course, we can probe neural models for problems beyond syntax and morphology. One perspective on large-scale pre-trained Transformer models is to view them as knowledge bases containing facts about the world. It is therefore tempting to see if we can apply them to test factual knowledge. A simple method is to ask a probe to recover the missing item of a sentence (Petroni et al., 2019). For example, if we have a cloze test

  Shiji was written by ____.

  we wish the probe to give an answer *Sima Qian* because there is a subject-relation-object fact (Shiji, written-by, Sima Qian). This probe can simply be a **masked language model** that is widely used in self-supervised learning of Transformer encoders.

In NLP, probing is closely related to the pre-training of large language models. In general, we can view probing tasks as downstream applications of these pre-trained models, though probing is ordinarily used as an analytical tool to quickly test the capabilities of the models. Ideally, we want to develop a probe that makes the best use of the internal representations to solve a given task. However, when a probe is complex and sufficiently well-trained, it becomes difficult to determine whether the problem was solved by the rich information in the representations or by the modeling capacity of the probe itself. A common way to isolate the contribution of the representations is to compare the probes against reasonable baselines or to conduct comparisons using control tasks (Belinkov, 2022; Hewitt and Liang, 2019).

4. Improved Architectures

In this section we present several improvements to the vanilla Transformer model. Unlike the previous section, most of the improvements are from the perspective of machine learning, rather than linguistics.

7. $\mathbf{h}$ can be computed by performing a pooling operation on $\{\mathbf{h}_1, \cdots, \mathbf{h}_m\}$

4.1 Locally Attentive Models

The self-attention methods discussed in Section 2.3 can be viewed as learning representations over the entire input sequence. While this global attention mechanism provides a strong ability to capture long-distance dependencies, it does not explicitly model local information. Here, we consider a few techniques that attempt to explicitly capture locality in representations.

4.1.1 Priors for Local Modeling

One of the simplest ways to introduce local context modeling into Transformers is to add a penalty term to the attention function, thereby discouraging large attention weights between distant positions. On the encoder side, this leads to a form that we have already encountered several times in this chapter:

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \quad = \quad \text{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma \cdot \mathbf{G}\right)\mathbf{H}^v, \tag{63}$$

where γ is the weight (or temperature) of the penalty term, and $\mathbf{G} \in \mathbb{R}^{m \times m}$ is the penalty matrix. Each entry $G(i,j)$ indicates how heavily we penalize the model for attending to position j from position i. A simple choice for $G(i,j)$ is the absolute distance between i and j, for example

$$G(i,j) \quad = \quad |i-j|. \tag{64}$$

Alternatively, $G(i,j)$ can be defined using a Gaussian penalty function (Yang et al., 2018)

$$G(i,j) \quad = \quad \frac{(i-j)^2}{2\sigma_i^2}, \tag{65}$$

where σ_i is the standard deviation of the Gaussian distribution. For a given i, both penalty terms increase (either linearly or exponentially) as the distance $|i-j|$ grows.

This method can be extended to the cross-attention model, as follows

$$\text{AttLocal}_{\text{cross}}(\mathbf{H}, \mathbf{S}) \quad = \quad \text{Softmax}\left(\frac{\mathbf{S}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma \cdot \mathbf{G}\right)\mathbf{H}^v, \tag{66}$$

where $\mathbf{G}$ is an $n \times m$ matrix. Each entry of $\mathbf{G}$ can be defined as

$$G(i,j) \quad = \quad \frac{(\mu_i - j)^2}{2\sigma_i^2}, \tag{67}$$

where μ_i is the mean of the Gaussian distribution over the source-side positions. Both μ_i and σ_i can be determined using heuristics. Alternatively, we can develop additional neural networks to model them, learning their parameters jointly with the rest of the model. For example, we could use a feed-forward neural network to predict μ_i given $\mathbf{s}_i$.

An alternative to Eq. (63) (or Eq. (66)) is to treat the penalty term as a separate distribution and linearly combine it with the original attention model. For example, we can define the self-attention model as

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \quad = \quad \left((1-\beta) \cdot \text{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\right) + \beta \cdot \text{Softmax}\left(-\gamma \cdot \mathbf{G}\right)\right)\mathbf{H}^v, \tag{68}$$

where $\beta \in [0, 1]$ is the linear combination coefficient. Note that, to avoid manually tuning the hyperparameter β, we can use a gating network to dynamically predict β and train it end-to-end.

Another alternative is to use a multiplicative mask to incorporate the prior into the modeling, similar to Eq. (53). This is given by

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \quad = \quad \text{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{G}'\right)\mathbf{H}^v. \tag{69}$$

Here, $\mathbf{G}' \in [0, 1]^{m\times m}$ is a matrix with entries in $[0, 1]$. The scalar $G'(i, j)$ takes a maximum value of 1 when $i = j$ and decays as j moves further away from i. $G'(i, j)$ can be obtained by normalizing $-G(i, j)$ over all j, or by using other decaying functions.

4.1.2 Local Attention

The term *local attention* has been used broadly to describe a wide range of problems and to refer to many different models in the NLP literature. The methods discussed above are those that impose soft constraints on attention models. In fact, local attention has its origins in attempts to restrict the scope of attention models for modeling and computational considerations (Luong et al., 2015). Research in this area often focuses on introducing hard constraints, so that the resulting models can focus on parts of the input and ignore the rest. For example, we can predict a span of source-side positions to attend to, given a target-side position (Sperber et al., 2018; Sukhbaatar et al., 2019; Yang et al., 2018). Also, attention spans can be induced from syntax trees, for example, syntactic subtree structures can help narrow the attention scope. Thus, many syntax-constrained models can be viewed as instances of local attention (see Section 3.4). In addition, the concept of local attention can be generalized to a rich set of models, such as **sparse attention models**, although these models are often discussed in the context of efficient machine learning methods. We will see a few examples in Section 5.

In deep learning, one of the most widely used architectures for learning features from a restricted region of the input is CNNs. It is thus interesting to consider methods of combining CNNs and Transformer models to obtain the benefits of both approaches, for example, CNNs deal with short-term dependencies, and self-attention models deal with long-term dependencies. One approach is to build a two-branch sequence model where one branch is based on CNNs and the other is based on self-attention models (Wu et al., 2020). Another approach is to incorporate CNN layers into Transformer blocks in a way that enables the model to learn both local and global representations (Gulati et al., 2020; Wu et al., 2019).

4.1.3 Relative Positional Embedding

Relative positional embedding, also known as **relative positional representation** (**RPR**), is an improvement to the absolute positional embedding method used in standard Transformers(Huang et al., 2018; Shaw et al., 2018). The idea of RPR is that we model the distance between two positions in a sequence rather than giving each position a fixed representation. As a result, we have a pair-wise representation $\text{PE}(i, j)$ for two positions i and j. One simple way to define $\text{PE}(i, j)$ is to consider it as a lookup table for all pairs of i and

j. More specifically, let $\mathbf{u}_\pi$ be a d-dimensional representation for a given distance π. The form of $\mathrm{PE}(i,j)$ in the vanilla RPR method is given by

$$\mathrm{PE}(i,j) \quad = \quad \mathbf{u}_{\mathrm{clip}(j-i,k_{\mathrm{rpr}})}, \tag{70}$$

where $\mathrm{clip}(x,k_{\mathrm{rpr}})$ is a function that clips x in the interval $[-k_{\mathrm{rpr}},k_{\mathrm{rpr}}]$

$$\mathrm{clip}(x,k_{\mathrm{rpr}}) \quad = \quad \max\{-k_{\mathrm{rpr}},\min\{x,k_{\mathrm{rpr}}\}\}. \tag{71}$$

Thus, we have a model with parameters

$$\mathbf{U}_{\mathrm{rpr}} \quad = \quad \begin{bmatrix} \mathbf{u}_{-k_{\mathrm{rpr}}} \\ \vdots \\ \mathbf{u}_0 \\ \vdots \\ \mathbf{u}_{k_{\mathrm{rpr}}} \end{bmatrix}. \tag{72}$$

While this notation is somewhat informal, we can view $\mathbf{U}_{\mathrm{rpr}}$ as a matrix $\in \mathbb{R}^{(2k_{\mathrm{rpr}}+1)\times d}$, and select a row corresponding to $\mathrm{clip}(j-i,k_{\mathrm{rpr}})$ when computing RPR for given i and j.

Using the above method, we can define three RPR models $\mathrm{PE}^q(i,j)$, $\mathrm{PE}^k(i,j)$ and $\mathrm{PE}^v(i,j)$ for queries, keys, and values, respectively. Then, following the form of Eq. (17), the output of the self-attention model at position i can be written as

$$\begin{aligned} \mathbf{c}_i \quad &= \quad \sum_{j=1}^{m} \alpha_{i,j} \left[\mathbf{h}_j^v + \mathrm{PE}^v(i,j)\right] \\ &= \quad \sum_{j=1}^{m} \alpha_{i,j}\mathbf{h}_j^v + \sum_{j=1}^{m} \alpha_{i,j}\mathrm{PE}^v(i,j), \end{aligned} \tag{73}$$

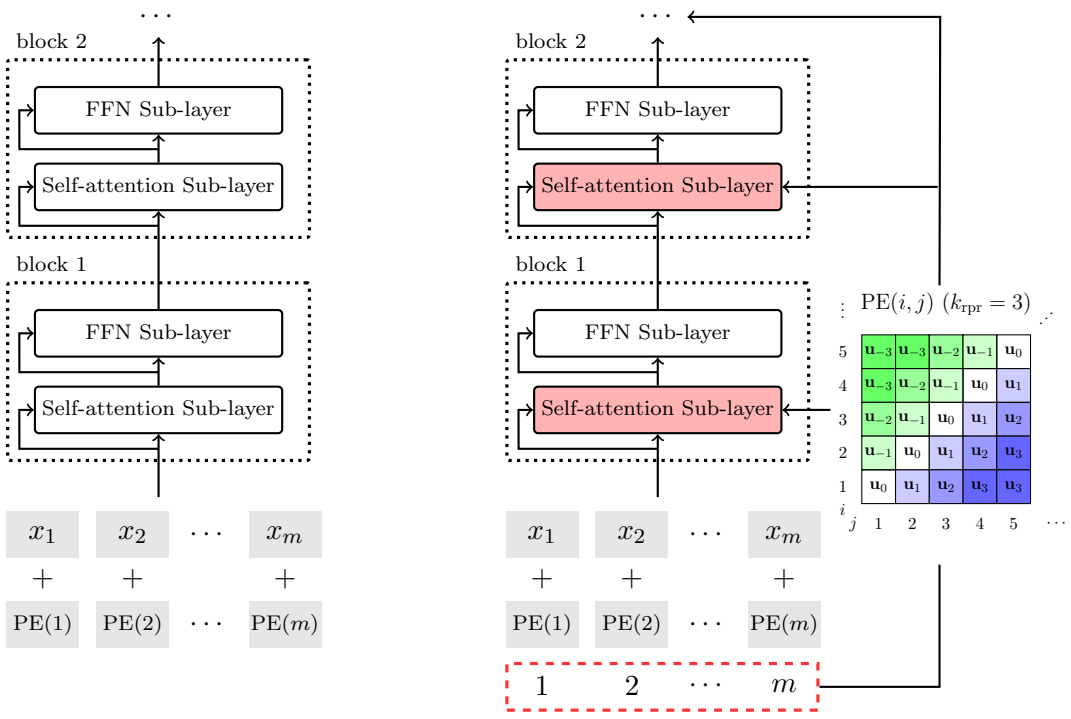
where $\mathbf{h}_j^v$ is the j-th row vector of $\mathbf{H}^v$. This representation comprises two components: $\sum_{j=1}^m \alpha_{i,j}\mathbf{h}_j^v$ is the basic representation, and $\sum_{j=1}^m \alpha_{i,j}\mathrm{PE}^v(i,j)$ is the positional representation.

The attention weight $\alpha_{i,j}$ is computed in a regular way, but with additional terms $\mathrm{PE}^q(i,j)$ and $\mathrm{PE}^k(i,j)$ added to each query and key:

$$\alpha_{i,j} \quad = \quad \mathrm{Softmax}\left(\frac{[\mathbf{h}_i^q + \mathrm{PE}^q(i,j)][\mathbf{h}_j^k + \mathrm{PE}^k(i,j)]^\top}{\sqrt{d}}\right). \tag{74}$$

Figure 10 shows the Transformer encoder architectures with and without RPR. When RPR is adopted, $\mathrm{PE}^q(i,j)$, $\mathrm{PE}^k(i,j)$, $\mathrm{PE}^v(i,j)$ are directly fed to each self-attention sublayer, and so we can make better use of positional information for sequence modeling. Note that the use of the clipping function (see Eq. (71)) makes the modeling simple because we do not need to distinguish the relative distances for cases $|j-i| \geq k_{\mathrm{rpr}}$. This clipped distance-based model can lead to better modeling in local context windows.

Eqs. (73) and (74) provide a general approach to position-sensitive sequence modeling. There are many variants of this model. In the early work on RPR by Shaw et al. (2018), the



(a) Transformer without RPR

(b) Transformer with RPR

Figure 10: Transformer encoders without and with relative positional representation (RPR). In RPR, each pair of positions is represented as a vector PE(i, j) using a model parameterized by $\mathbf{U}_{\text{rpr}}$. PE(i, j) is fed into each self-attention sub-layer so that we can make use of the positional information in intermediate steps of learning representations.

positional representations for queries are removed, and the model works only with $\text{PE}^k(i,j)$ and $\text{PE}^v(i,j)$, like this

$$\alpha_{i,j} = \text{Softmax}(\frac{\mathbf{h}_i^q[\mathbf{h}_j^k + \text{PE}^k(i,j)]^\top}{\sqrt{d}}). \quad (75)$$

By contrast, some variants attempt to improve the RPR model when computing attention weights but ignore $\text{PE}^v(i,j)$ when computing value representations (Dai et al., 2019; He et al., 2021). Instead of treating RPR as an additive term to each representation, researchers have also explored other ways of introducing RPR into the Transformer (Huang et al., 2020; Raffel et al., 2020). We refer interested readers to these papers for more details.

4.2 Deep Models

Many state-of-the-art NLP systems are based on deep Transformer models. For example, recent large language models generally comprise tens of Transformer layers (or more precisely, hundreds of layers), demonstrating strong performance on many tasks (Ouyang et al., 2022; Touvron et al., 2023a). By stacking Transformer layers, it is straightforward to construct a deep model. However, as is often the case, training very deep neural networks is challenging. One difficulty arises from the fact that the error surfaces of deep neural networks are highly non-convex and contain many local optima, making the training process likely to get stuck. While there are optimization algorithms that can help alleviate this problem, most practical approaches rely on gradient-based methods for optimizing deep neural networks. As a result, training a model with many Transformer layers becomes challenging due to vanishing and exploding gradients during back-propagation. Here, we consider several techniques for training deep Transformer models.

4.2.1 Re-thinking the Pre-Norm and Post-Norm Architectures

As introduced previously, a Transformer sub-layer is a residual network where a shortcut is created to add the input of the network directly to the output of the sub-layer. This allows gradients to flow more directly from the output back to the input, mitigating the vanishing gradient problem. In general, a residual connection in a Transformer is used together with a layer normalization unit to form a sub-layer. This leads to two types of architectures, called post-norm and pre-norm. Specifically, recall from Section 2.4 that the post-norm architecture can be expressed as:

$$\mathbf{z}^l = \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}), \quad (76)$$

where $\mathbf{z}^l$ and $\mathbf{z}^{l-1}$ are the output and input of the sub-layer l, respectively, and $F^l(\cdot)$ is the core function of this sub-layer. The pre-norm architecture takes the identity mapping $\mathbf{z}^{l-1}$ outside the layer normalization function, given in the form

$$\mathbf{z}^l = F^l(\text{LNorm}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \quad (77)$$

Consider the difference between the information flow in these two architectures:

- The post-norm architecture prevents the identity mapping of the input from being directly added to the final output of the sub-layer. This is not a true residual network because all the information is passed through a non-linear function (i.e., the layer normalization unit). Thus, the post-norm architecture is less effective for back-propagation. Wang et al. (2019) show that the gradient of the loss of an L-sub-layer Transformer network with respect to $\mathbf{z}^l$ is given by

$$\frac{\partial E}{\partial \mathbf{z}^l} = \frac{\partial E}{\partial \mathbf{z}^L} \cdot \prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k} \cdot \prod_{k=l}^{L-1} \left(1 + \frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}\right), \tag{78}$$

  where $\mathbf{z}^L$ is the output of the last layer, $\mathbf{v}^k$ is short for $F^k(\mathbf{z}^{k-1})$, and E is the error measured by some loss function. $\frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ and $\frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}$ are the gradients of the layer normalization function and the core function, respectively. Although the equation here appears complex, we see that $\prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ is simply a product of $L - l$ factors. This means that the error gradient will be rescaled repeatedly as L becomes larger, leading to a higher risk of vanishing or exploding gradients in deeper models.

- The pre-norm architecture describes a standard residual neural network where the input of the entire network is added directly to its output. We can write the gradient of the error at $\mathbf{z}^l$ as:

$$\begin{aligned} \frac{\partial E}{\partial \mathbf{z}^l} &= \frac{\partial E}{\partial \mathbf{z}^L} \cdot \left(1 + \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}\right) \\ &= \frac{\partial E}{\partial \mathbf{z}^L} + \frac{\partial E}{\partial \mathbf{z}^L} \cdot \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}. \end{aligned} \tag{79}$$

  It is easy to see that $\frac{\partial E}{\partial \mathbf{z}^l}$ receives direct feedback regarding the error, because the first term on the right-hand side (i.e., $\frac{\partial E}{\partial \mathbf{z}^L}$) is the gradient of the model output, which is independent of the network depth.

The use of the pre-norm architecture also helps optimization during early gradient descent steps. For example, it has been found that pre-norm Transformer models can be trained using a larger learning rate in the early stages of training, instead of requiring a gradual learning rate warmup from a small value (Xiong et al., 2020).

While the pre-norm architecture facilitates easier optimization of deep Transformer models, we cannot simply conclude that it is a universally better choice compared to the post-norm architecture. In fact, both post-norm and pre-norm Transformer models have been successfully used in many applications. For example, the post-norm architecture is widely used in BERT-like models, while the pre-norm architecture is the more popular choice for recent generative large language models. Broadly, these two architectures provide different paradigms for designing deep Transformer models, each with distinct advantages and disadvantages. The post-norm architecture forces the representation to be learned through more non-linear functions, resulting in a highly expressive but complicated model that is relatively hard to train. By contrast, the pre-norm architecture makes the training of Transformer models easier, but it may be less expressive than its post-norm counterpart if the learned model becomes overly dependent on the shortcut paths.

An improvement to these architectures is to control the extent to which we want to "skip" a sub-layer. A simple way to do this is to weight different paths rather than treating them equally. For example, a scalar factor of a residual connection can be introduced to determine how heavily we weight this residual connection relative to the path of the core function (He et al., 2016; Liu et al., 2020a,b). A more general form of this model is given by

$$\mathbf{z}^l \quad = \quad \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \beta \cdot \mathbf{z}^{l-1}) + \gamma \cdot \mathbf{z}^{l-1}, \tag{80}$$

where β is the weight of the identity mapping inside the layer normalization function, and γ is the weight of the identity mapping outside the layer normalization function. Clearly, both the post-norm and pre-norm architectures can be seen as special cases of this equation. That is, if $\beta = 1$ and $\gamma = 0$, then it will become Eq. (76); if $\beta = 0$ and $\gamma = 1$, it will become Eq. (77). This model provides a multi-branch view of building residual blocks. The input to this block can be computed through multiple paths with different modeling complexities. When β and γ are small, the representation is forced to be learned through a "deep" model with multiple layers of cascaded non-linear units. In contrast, when β and γ are large, the representation is more likely to be learned using a "shallow" model with fewer layers. To determine the optimal choices of β and γ, one can give them fixed values by considering some theoretical properties or system performance on validation sets, or compute these values by using additional functions that can be trained to do so (Srivastava et al., 2015). It should be emphasized that many other types of architecture can be considered in the design of a Transformer sub-layer. It is possible, for instance, to introduce more layer normalization units into a sub-layer (Ding et al., 2021; Wang et al., 2022b), or, conversely, to simply remove them from a sub-layer (Bachlechner et al., 2021).

4.2.2 Parameter Initialization

As with other deep neural networks, there is considerable interest in developing parameter initialization methods for deep Transformer models in order to facilitate optimization in a more favorable region of the parameter space. However, initialization is a broad topic in the optimization of machine learning models, and a general discussion of this topic lies beyond the scope of this section. Here, we will focus on specific parameter initialization methods used in Transformer-based systems rather than general optimization problems.

While the parameters of a neural network can be initialized in various ways, most practical systems adopt simple techniques to provide appropriate initial values. Consider, for example, the Xavier initialization for a parameter matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ (Glorot and Bengio, 2010). We define a variable η as follows

$$\eta \quad = \quad \text{gain} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \tag{81}$$

where gain is a hyper-parameter which equals 1 by default. Then, each entry of $\mathbf{W}$ can be initialized by using a uniform distribution

$$W \quad \sim \quad U\left(-\eta, \eta\right) \tag{82}$$

or, alternatively, using a Gaussian distribution

$$W \sim \mathcal{N}\left(0, \eta^2\right). \tag{83}$$

This method can be easily adapted to initialize Transformer models with a large number of layers. One common approach is to find a more suitable value for gain by taking into account the fact that the optimal initial conditions for optimization might differ across neural networks of varying depths. For example, one can increase the value of gain as the depth of the model grows. Thus, gain can be defined as a function of the network depth in the following form

$$\text{gain} = a \cdot L^b, \tag{84}$$

where a is a scalar, and L^b is the network depth raised to the power of b. Typically, both a and b are positive numbers, indicating a preference for larger initial parameter values in deeper models. For example, Wang et al. (2022a) demonstrate that by choosing appropriate values for a and b, a very deep Transformer model can be successfully trained.

Eq. (84) assigns gain the same value for all of the sub-layers. However, it is found that the norm of gradients becomes smaller as a sub-layer is located farther from the output layer. This consistent application of gain across the entire model could result in under-training of the lower layers due to the gradient vanishing problem. For this reason, one can develop methods that are sensitive to the position of a sub-layer in the neural network. The general form of such methods is given by

$$\text{gain} = \frac{a}{l^b}. \tag{85}$$

Here l denotes the depth of a sub-layer. If l is larger (i.e., the sub-layer is closer to the output), gain will be smaller and the corresponding parameters will be set to smaller values. An example of this method can be found in the work of Zhang et al. (2019).

It is also, of course, straightforward to apply general methods of initializing deep multi-layer neural networks to Transformer models. An example is to consider the **Lipschitz constant** in parameter initialization, which has been shown to help improve the stability of training deep models (Szegedy et al., 2014; Xu et al., 2020). Another approach is to use second-order methods to estimate the proper values of the parameters. For example, one can compute the Hessian of each parameter matrix to model its curvature (Skorski et al., 2021).

For models with a large number of layers, it is also possible to pre-train some of the layers via smaller models and use their trained parameters to initialize bigger models (Chen et al., 2015). That is, we first obtain a rough estimation of the parameters in a cheap way, and then continue the training process on the whole model as usual. These methods fall into a class of training methods, called **model growth** or **depth growth**.

As a simple example, consider a Transformer model (e.g., a Transformer encoder) of $2L$ sub-layers. We can train this model by using the **shallow-to-deep training** method (Li et al., 2020b). First, we train an L-sub-layer model (call it the shallow model) in a regular way. Then, we create a $2L$-sub-layer model (call it the deep model) by stacking the shallow model twice, and further train this deep model. To construct deeper models, this procedure

can be repeated multiple times, say, we start with a model of L sub-layers, and obtain a model of $L \cdot 2^I$ after I iterations. Note that many pre-trained models are used in a similar manner. For example, for BERT-like methods, a Transformer encoder is trained on large-scale data, and the optimized parameters are then used to initialize downstream systems.

4.2.3 Layer Fusion

Another problem with training a deep Transformer model is that the final prediction is conditioned on the last layer of the neural network. While the use of residual connections enables direct access to lower-level layers from a higher-level layer, there is still a "long" path for passing information from the bottom to the top. One simple way to address this is to create residual connections that skip more layers. For example, consider a group of L Transformer sub-layers. For the sub-layer at depth l, we can introduce $l-1$ residual connections, each connecting this sub-layer to a previous sub-layer. In this way, we develop a densely connected network where each sub-layer takes as input the outputs of all previous sub-layers (Huang et al., 2017). The output of the last sub-layer can be seen as a combination of representations at different levels.

Following the notation used in the previous subsections, we denote the output of the sub-layer at depth l by $\mathbf{z}^l$, and denote the function of the sub-layer by $\text{Layer}^l(\cdot)$. Then, $\mathbf{z}^l$ can be expressed as

$$\mathbf{z}^l = \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{86}$$

We can simply view $\text{Layer}^l(\cdot)$ as a function that fuses the information from $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$. There are many possible choices for $\text{Layer}^l(\cdot)$. For example, a simple form of $\text{Layer}^l(\cdot)$ is given by

$$\begin{aligned} \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) &= \text{LNorm}(F^l(\mathbf{Z}^l)), & (87)\\ \mathbf{Z}^l &= \phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). & (88) \end{aligned}$$

Here $\phi(\cdot)$ takes the layer outputs $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ and fuses them into a single representation $\mathbf{Z}^l$. A simple instance of $\phi(\cdot)$ is average pooling which computes the sum of $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ divided by $l-1$. See Table 2 for more examples of $\phi(\cdot)$.

Adopting a similar architecture for a Transformer sub-layer, we can also consider a post-norm form

$$\begin{aligned} \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) &= \text{LNorm}(\mathbf{Z}^l), & (89)\\ \mathbf{Z}^l &= \phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}), & (90) \end{aligned}$$

or a pre-norm form

$$\begin{aligned} \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) &= \mathbf{Z}^l, & (91)\\ \mathbf{Z}^l &= \phi(\text{LNorm}(F^l(\mathbf{z}^{l-1})), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). & (92) \end{aligned}$$

These models are quite general. For example, a standard post-norm encoder sub-layer can be recovered as a special case of Eqs. (89-90), if we remove the dependencies on sub-layers 1 to $l-2$, and define $\phi(\cdot)$ to be

$$\phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}. \tag{93}$$

Entry	Function
Average Pooling	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \frac{1}{l-1} \sum_{k=1}^{l-1} \mathbf{z}^k$
Weighted Sum	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k$
Feedforward Network	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}])$
Self-attention	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\text{Att}_{\text{self}}(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1})])$

Table 2: Fusion functions. $\text{FFN}(\cdot)$ = feedforward neural network, $[\cdot]$ = concatenating the input vectors, and $\text{Att}_{\text{self}}(\cdot)$ = self-attention function. All fusion functions can be followed by a layer normalization function, for example, we can write the weighted sum of $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ as $\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(\sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k)$.

Densely connected networks make it easier for information to flow through direct connections between sub-layers, but the resulting models are slightly more complex, especially when we use parameterized fusion functions. In practice, we typically add dense connections only to a subset of the sub-layers, so the overall network is not overly dense. For example, we might only add connections from lower sub-layers to the final few sub-layers. Thus, predictions can be made with direct access to different levels of representation (Wang et al., 2018a).

4.2.4 Regularization

In machine learning, regularization is used to mitigate overfitting when training deep neural networks. It is therefore straightforward to apply regularization techniques to Transformer models. Since the topic of regularization is well studied, here we consider several methods that are particularly useful for training deep Transformer models.

One approach to regularizing deep Transformer models is to randomly skip sub-layers or layers during training (Huang et al., 2016; Pham et al., 2019). In each run of the model, we retain each sub-layer with a probability ρ and stack the selected sub-layers to form a new model. Thus, we essentially train an ensemble of different neural networks with shared architectures and parameters on the same dataset. In this way, each sub-layer learns to operate somewhat independently, and overfitting is reduced by preventing the co-adaptation of sub-layers. In fact, dropping out sub-layers (or layers) and dropping out neurons are essentially variations on the same theme. Sometimes, the method described here is referred to as **sub-layer dropout** or **layer dropout**.

At test time, we need to combine all possible networks to make predictions. A simple method to achieve this is to rescale the outputs of the stochastic components of the model (Li et al., 2021). As an example, suppose each sub-layer has a pre-norm architecture. Then, the output of the sub-layer at depth l is given by

$$\mathbf{z}^l \quad = \quad \rho \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \tag{94}$$

Another idea is to share parameters across sub-layers. One of the simplest methods is to use the exact same parameters for all corresponding sub-layers (Dehghani et al., 2018). For example, all the FFN sub-layers could be based on the same feed-forward network.

This approach has a regularizing effect similar to methods that add the norms of parameter matrices to the loss function to penalize model complexity. For practical systems, there can be significant benefits to adopting a shared architecture, as it allows us to reuse the same sub-model to build a deep neural network, thereby substantially reducing the overall memory footprint. We will see further discussion regarding efficiency issues in Section 5.4.

4.3 Numerical Method-Inspired Models

A residual network computes its output as the sum of the identity mapping and a transformation of the input. Such a model can be interpreted as an Euler discretization of **ordinary differential equations** (**ODEs**) (Ee, 2017; Haber and Ruthotto, 2017). To illustrate this idea, we consider a general form of residual networks

$$\mathbf{z}^l \quad = \quad f^l\big(\mathbf{z}^{l-1}\big) + \mathbf{z}^{l-1}, \tag{95}$$

where $f^l(\mathbf{z}^{l-1})$ denotes a function that takes an input variable $\mathbf{z}^{l-1}$ and produces an output variable in the same space. Clearly, a Transformer sub-layer is a special case of this equation. For example, for a pre-norm Transformer, we have $f^l(\cdot) = \mathrm{LNorm}(F^l(\cdot))$.

For notational simplicity, we rewrite the above equation in an equivalent form

$$\mathbf{z}(l) \quad = \quad f\big(\mathbf{z}(l-1), l\big) + \mathbf{z}(l-1). \tag{96}$$

We use the notations $\mathbf{z}(l)$ and $f(\mathbf{z}(\cdot, l))$ to emphasize that $\mathbf{z}(\cdot)$ and $f(\cdot)$ are functions of l. Here we assume that l is a discrete variable. If we relax l to a continuous variable and $\mathbf{z}(l)$ to a continuous function of l, then we can express Eq. (96) as

$$\mathbf{z}(l) \quad = \quad \triangle l \cdot f\big(\mathbf{z}(l - \triangle l), l\big) + \mathbf{z}(l - \triangle l). \tag{97}$$

This can be further written as

$$\frac{\mathbf{z}(l) - \mathbf{z}(l - \triangle l)}{\triangle l} \quad = \quad f\big(\mathbf{z}(l - \triangle l), l\big). \tag{98}$$

Taking the limit $\triangle l \to 0$, we have an ODE

$$\frac{\mathrm{d}\mathbf{z}(l)}{\mathrm{d}l} \quad = \quad f\big(\mathbf{z}(l), l\big). \tag{99}$$

We can therefore say that a pre-norm Transformer sub-layer (i.e., Eqs. (96) and (95)) is an Euler discretization of the above ODE. This is an interesting result! A sub-layer essentially serves as an ODE solver.

Eqs. (96) and (95) represent standard forms of the **Euler method**. This method computes a new estimate of the solution by moving one step forward along l from the previous estimation. In general, two aspects can be considered in the design of numerical methods for ODEs.

- **Linear Multi-step Methods**. A linear multi-step method computes the current estimate of the solution by taking the estimations and derivative information from multiple previous steps. A general formulation of p-step methods can be expressed as

$$\mathbf{z}(l) \quad = \quad \sum_{i=1}^{p} a_i \cdot \mathbf{z}(l-i) + h \sum_{i=1}^{p+1} b_i \cdot f\big(\mathbf{z}(l-i),\ l-i+1\big), \tag{100}$$

where h is the size of the step we move each time[8], that is, $\triangle l$ in Eqs. (97) and (98). $\{a_i\}$ and $\{b_i\}$ are coefficients of the solution points and derivatives in the linear combination. Given this definition, we can think of the Euler method as a single-step, low-order method of solving ODEs[9].

- **(Higher-order) Runge-Kutta Methods**. Runge-Kutta (RK) methods and their variants provide ways to compute the next step solution by taking intermediate results in solving an ODE. As a result, we obtain higher-order methods but still follow the form of single-step methods, that is, the estimated solution is dependent only on $\mathbf{z}(l-1)$ rather than on the outputs at multiple previous steps.

In fact, linear multi-step methods, though not explicitly mentioned, have been used in the layer fusion methods discussed in Section 4.2. For example, taking Eqs. (91) and (92) and a linear fusion function, a pre-norm sub-layer with dense connections to all previous sub-layers can be expressed as

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) \quad = \quad a_1 \cdot \mathbf{z}^{l-1} + \cdots + a_{l-1} \cdot \mathbf{z}^1 + b_1 \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})). \tag{101}$$

This equation is an instance of Eq. (100) where we set $h = 1$ and remove some of the terms on the right-hand side.

It is also straightforward to apply Runge-Kutta methods to Transformer models (Li et al., 2022a). Given an ODE as described in Eq. (99), an explicit p-th order Runge-Kutta solution is given by

$$\mathbf{z}(l) \quad = \quad \mathbf{z}(l-1) + \sum_{i=1}^{p} \gamma_i \cdot \mathbf{g}_i, \tag{102}$$

$$\mathbf{g}_i \quad = \quad h \cdot f\big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j,\ l - 1 + \lambda_i \cdot h\big). \tag{103}$$

Here $\mathbf{g}_i$ represents an intermediate step which is present only during the above process. $\{\gamma_i\}$, $\{\beta_{i,j}\}$ and $\{\lambda_i\}$ are coefficients that are determined by using the Taylor series of $\mathbf{z}(l)$. To simplify the model, we assume that the same function f is used for all $\{\mathbf{g}_i\}$. Then, we remove the dependency on the term $l - 1 + \lambda_i \cdot h$ in f, and rewrite Eq. (103) as

$$\mathbf{g}_i \quad = \quad h \cdot f\big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j\big), \tag{104}$$

where $f(\cdot)$ is a function that is independent of i.

8. Let $\{t_0, \cdots, t_i\}$ denote the values of the variable l at steps $\{0, \cdots, i\}$. In linear multi-step methods, it is assumed that $t_i = t_0 + ih$.
9. In numerical analysis, the **local truncation error** of a method of solving ODEs at a step is defined to be the difference between the approximated solution computed by the method and the true solution. The method is called order p if it has a local truncation error $O(h^{p+1})$.

As an example, consider the 4th-order Runge-Kutta (RK4) solution

$$\mathbf{z}(l) = \mathbf{z}(l-1) + \frac{1}{6}(\mathbf{g}_1 + 2\mathbf{g}_2 + 2\mathbf{g}_3 + \mathbf{g}_4), \tag{105}$$

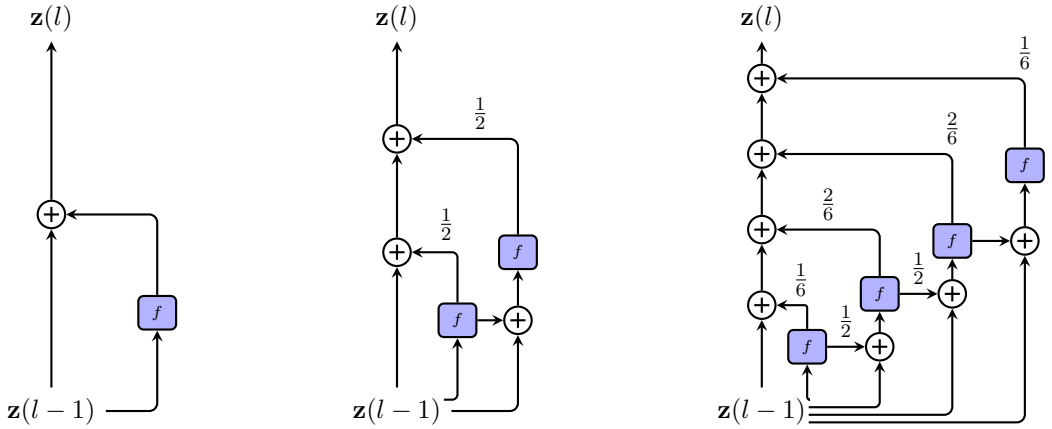
$$\mathbf{g}_1 = h \cdot f(\mathbf{z}(l-1)), \tag{106}$$

$$\mathbf{g}_2 = h \cdot f(\mathbf{z}(l-1) + \frac{1}{2}\mathbf{g}_1), \tag{107}$$

$$\mathbf{g}_3 = h \cdot f(\mathbf{z}(l-1) + \frac{1}{2}\mathbf{g}_2), \tag{108}$$

$$\mathbf{g}_4 = h \cdot f(\mathbf{z}(l-1) + \mathbf{g}_3). \tag{109}$$

These equations define a new architecture of sub-layer. For example, by setting $h = 1$ and $f(\cdot) = \text{LNorm}(F^l(\cdot))$, we obtain an RK4 Transformer sub-layer, as shown in Figure 11. This method leads to a deep model because each sub-layer involves four evaluations of $f(\cdot)$ in sequence. On the other hand, the resulting model is parameter efficient because we reuse the same function $f(\cdot)$ within the sub-layer, without introducing additional parameters.



(a) Pre-norm (b) RK2 (c) RK4

Figure 11: Pre-norm (a) and Runge-Kutta (b and c) sub-layer architectures. $\mathbf{z}(l-1)$ denotes the input of a sub-layer at depth l, $\mathbf{z}(l)$ denotes the output of the sub-layer, and f (in blue boxes) denotes the function $f(\cdot) = \text{LNorm}(F^l(\cdot))$.

So far in this subsection, our discussion has focused on designing Transformer architectures inspired by dynamical systems. While the basic ODE model is continuous with respect to the depth l, these methods still follow the general framework of neural networks in which l is treated as a discrete variable, and the representational power of the models is largely determined by this hyper-parameter. An alternative approach is to use neural ODEs to relax the "depth" into a continuous variable. In this way, we can obtain a continuous-depth model for computing the solution to ODEs. However, as a detailed discussion of neural ODEs lies beyond the scope of this chapter, we refer interested readers to related papers for more details (Chen et al., 2018c; Kidger, 2022; Xiao et al., 2026).

4.4 Wide Models

Most of the methods that we have studied so far in this section are examples of learning and using deep models. Another design choice we generally face is determining the width of a neural network. Typically, the **width** of a Transformer model is defined as the dimensionality of a representation at some position in the input sequence, that is, the parameter d. Increasing this width is a common method for obtaining a more complex and powerful model. For example, in the work of Vaswani et al. (2017), a wide model (referred to as Transformer-Big) leads to significant improvements in translation quality for machine translation systems. Such wide models have been widely used to boost system performance on large-scale tasks (Fedus et al., 2022b; Lepikhin et al., 2021).

However, developing very wide Transformer models is difficult. One major challenge is that training such systems is computationally expensive. While the number of model parameters (the model size) grows linearly with d, the time complexity of the model grows quadratically with d (see Table 1). In some NLP tasks, it has been found empirically that the training effort required to obtain satisfactory performance follows a scaling law (Kaplan et al., 2020). These results emphasize the need for ways to improve training efficiency when enlarging d.

One simple method is to incrementally grow the model along the dimension of d, rather than training the model from scratch. Suppose we have an initial model with a $d_1 \times d_1$ parameter matrix $\mathbf{W}_1$, for example, the linear transformation of each query or key in some layer. We can train this model to obtain optimized $\mathbf{W}_1$ in a regular way. Then, we want to extend this model to a wider model where $\mathbf{W}_1$ is replaced by a $d_2 \times d_2$ parameter matrix $\mathbf{W}_2$. Let us assume for simplicity that $d_2 = kd_1$. There are several ways to expand a $d_1 \times d_1$ matrix to a $kd_1 \times kd_1$ matrix. The simplest of these may be to use $\mathbf{W}_1$ to fill $\mathbf{W}_2$. We can write $\mathbf{W}_2$ in the form

$$\mathbf{W}_2 = \overbrace{\begin{bmatrix} \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \\ \vdots & & \vdots \\ \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \end{bmatrix}}^{k \text{ times}} \Big\} k \text{ times}, \tag{110}$$

where ρ is a hyper-parameter that is used to control the norm of $\mathbf{W}_2$. For example, if $\rho = k$, $\mathbf{W}_2$ will have the same l_1 norm as $\mathbf{W}_1$. The above equation provides a good starting point for training the wide model, and we can train $\mathbf{W}_2$ as usual after initialization. The procedure can be repeated a number of times for constructing a model with arbitrary width. Both this method and the depth growth method described in Section 4.2 are instances of the general method of model growth. In other words, we can obtain a larger model by extending a small model either vertically or horizontally, or both. Alternative methods for transforming $\mathbf{W}_1$ to $\mathbf{W}_2$ involve those considering other mathematical properties of the transformation (Chen et al., 2015). These models can fall under the reusable neural networks where we are concerned with models and algorithms for transferring parameters from small models to (significantly) larger models (Wang et al., 2023).

A second difficulty in building a wide Transformer model is the large memory requirement. Since the feedforward network generally has a larger hidden layer than other parts of the model, it demands relatively more memory as the model becomes wider. Consider

the feedforward network described in Section 2.5

$$\begin{aligned} \mathbf{H}_{\text{out}} &= \text{FFN}(\mathbf{H}_{\text{in}}) \\ &= \text{ReLU}(\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h + \mathbf{b}_h) \cdot \mathbf{W}_f + \mathbf{b}_f, \end{aligned} \tag{111}$$

where $\mathbf{W}_h \in \mathbb{R}^{d \times d_{\text{ffn}}}$ and $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}} \times d}$ are the parameters of the linear transformations. d_{ffn} is typically several times larger than d. Therefore, $\mathbf{W}_h$ and $\mathbf{W}_f$ will dominate the model size if d and d_{ffn} have very large values.

In some cases, the size of the feedforward network may exceed the memory capacity of a single device. This problem can be addressed by using the **mixture-of-experts** (**MoE**) models (Shazeer et al., 2017). An MoE model consists of M expert models $\{e_1(\cdot), \cdots, e_M(\cdot)\}$. Given an input $\mathbf{h}_{\text{in}} \in \mathbb{R}^d$, each expert model produces an output $e_k(\mathbf{h}_{\text{in}})$. The output of the MoE model is a linear combination of $\{e_1(\mathbf{h}_{\text{in}}), \cdots, e_M(\mathbf{h}_{\text{in}})\}$, given by

$$\mathbf{h}_{\text{out}} = \sum_{i=1}^{M} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}), \tag{112}$$

where $g(\cdot)$ is a gating model (also called **routing model**). Its output is a vector $g(\mathbf{h}_{\text{in}}) = \begin{bmatrix} g_1(\mathbf{h}_{\text{in}}) & \cdots & g_M(\mathbf{h}_{\text{in}}) \end{bmatrix}$ in which each entry $g_i(\mathbf{h}_{\text{in}})$ indicates the weight of the corresponding expert model. In many applications, it is assumed that $g(\mathbf{h}_{\text{in}})$ is a sparse vector. This means that only a small number of expert models are involved in computing the output. A widely-used form of $g(\mathbf{h}_{\text{in}})$ is given by using the softmax layer

$$g(\mathbf{h}_{\text{in}}) = \text{Softmax}(\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g), \tag{113}$$

where $\mathbf{W}_g \in \mathbb{R}^{d \times M}$ is the parameter matrix of the layer. To enforce sparsity on $g(\mathbf{h}_{\text{in}})$, we can select the top-$k$ entries of $g(\mathbf{h}_{\text{in}})$, that is, we set non-top-k entries to 0. An alternative method is to first perform top-k selection on $\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g$ and then normalize the top-k entries using the softmax function.

Let π be the set of the indices of the top-k expert models. The MoE model with top-k routing has the following form

$$\mathbf{h}_{\text{out}} = \sum_{i \in \pi} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}). \tag{114}$$

An advantage of this approach is that we can distribute different expert models to different processors, making it possible to execute these models on parallel computing machines. In each run of the MoE model, either during training or inference, we only need to activate and use k expert models rather than all of the expert models. In this way, the MoE approach is automatically learning a sparse model by limiting the number of active expert models each time in training and inference. The sparsity is determined by the hyperparameter k, e.g., a small value of k leads to a sparse model, and a large value of k leads to a dense model.

Let us return to the discussion of Eq. (111). It is straightforward to apply the MoE approach to feedforward neural networks. To simplify the discussion, consider the linear

transformation of the first layer as shown in Eq. (111), that is, $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$. We can approximate $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$ in an MoE form

$$\begin{aligned} \mathbf{H}_{\text{in}} \cdot \mathbf{W}_h &\approx \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot e_i(\mathbf{H}_{\text{in}}) \\ &= \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot [\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i]. \end{aligned} \tag{115}$$

Here $\mathbf{W}_h$ is divided into M slices (or sub-matrices) $\{\mathbf{W}_h^1, \cdots, \mathbf{W}_h^M\}$, written as

$$\mathbf{W}_h = \begin{bmatrix} \mathbf{W}_h^1 & \cdots & \mathbf{W}_h^M \end{bmatrix}. \tag{116}$$

Hence each expert model $e_i(\mathbf{H}_{\text{in}}) = \mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i$ solves a sub-problem of the original linear mapping, and Eq. (115) can be thought of as a divide-and-conquer solution to the matrix multiplication problem.

We can, of course, treat any feedforward neural network as an expert model, resulting in the following model

$$\mathbf{H}_{\text{out}} = \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot \text{FFN}_i(\mathbf{H}_{\text{in}}), \tag{117}$$

where $\text{FFN}_i(\cdot)$ is a "small" feedforward neural network that has the same form as Eq. (111). This model is illustrated with an example in Figure 12. In practical implementations, all these expert models can be run in parallel on different devices, and so the resulting system is efficient.

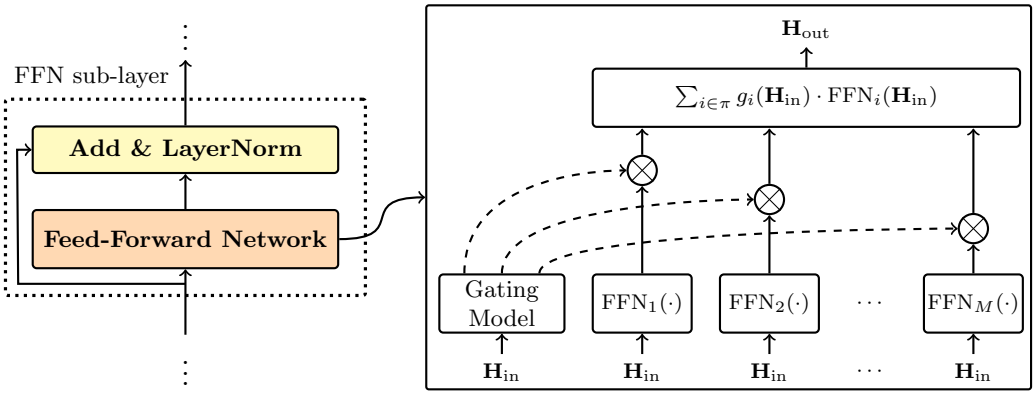


Figure 12: An illustration of the MoE model applied to an FFN sub-layer. There are M FFNs (call them expert models) and a gating model. Each FFN is weighted by the gating model. The output of the model is the sum of the weighted outputs of the top-k FFNs (denoted by π). Because these FFNs work independently and can be placed on different computing devices, the model can be easily scaled up as M is larger.

Note that, from a perspective of machine learning, MoE is a general approach to combining different neural networks, each of which is developed to address a different aspect

of the problem (Masoudnia and Ebrahimpour, 2014; Yuksel et al., 2012). The application here is just a special instance of the general framework of MoE. The approach is also often used to improve the overall performance of predictors, which can be discussed in the field of ensemble learning (Zhou, 2012).

Another difficulty in developing massive Transformer models is training instability. As with many other exceptionally large neural networks, the straightforward optimization of a Transformer model with billions of parameters may lead to the model getting trapped in local minima and, occasionally, to large spikes in the loss during training (Chowdhery et al., 2022; Fedus et al., 2022b; Lepikhin et al., 2021). Even with careful tuning of hyperparameters, training strategies, and initial parameter values, it is common to encounter situations where the training must be restarted from an earlier checkpoint to escape unstable optimization regions. One of the reasons for this training difficulty is that the standard implementations of linear algebra operations, such as matrix multiplication, can become numerically unstable when they operate on extremely large vectors and matrices. It is therefore crucial to improve training stability by employing numerically robust methods and mixed-precision techniques.

5. Efficient Models

Efficiency is an important consideration for many practical applications of Transformer models. For example, we may wish to run and/or train a Transformer model given memory and time constraints. Efficiency is not a single issue, but covers a wide range of problems. While these problems can be categorized in several different ways, there are two fundamental aspects one may consider in an efficiency problem.

- **Time and Space Efficiencies**. For a given problem, we wish the model to be small and fast, while remaining as accurate as possible in solving the problem. For example, in some machine translation applications, we may develop a model with a small number of parameters to fit into limited memory, and may develop a fast search algorithm to achieve low-latency translation. A practical difficulty here is that improving efficiency often leads to worse predictions. In many cases, we need to seek a trade-off between efficiency and accuracy.
- **Scalability**. When the problem is scaled up, we wish that the additional computational overhead required is minimized. For example, the training of a neural network is called efficient if it takes a reasonably short time to optimize it as the volume of training data increases. Another metric of efficiency is the amount of resources consumed in processing larger inputs. For instance, a machine translation system is inefficient in translating long sentences if the memory footprint and latency grow exponentially with the number of input words.

In this section, we will not discuss all the issues related to efficiency, which is a very broad topic. We instead consider the widely-used efficient approaches to Transformer-based sequence modeling and generation, some of which are refinements of model architectures, and some of which are architecture-agnostic approaches and could be used in other systems as well. Most of the discussions here are focused on developing lightweight and fast Transformer models that are relatively robust to long input and output sequences.

In general, the same optimization method can be applied to different modules of a Transformer-based system. To simplify the discussion, we will mostly consider self-attention sub-layers and FFN sub-layers in this section. Our discussion, however, is general and the methods presented here can be applied to other parts of a Transformer model, for example, cross-attention sub-layers.

5.1 Sparse Attention

In practice, the attention mechanisms used in Transformers are time-consuming, especially when the input sequences are long. To illustrate, consider a Transformer decoder that predicts a distribution over the vocabulary at each time step given the previous words. Suppose the sequence generated by the decoder has length n and the input to a self-attention sub-layer is an $n \times d$ matrix $\mathbf{S}$. First, $\mathbf{S}$ is linearly transformed to obtain the queries $\mathbf{S}^q \in \mathbb{R}^{n \times d}$, keys $\mathbf{S}^k \in \mathbb{R}^{n \times d}$, and values $\mathbf{S}^v \in \mathbb{R}^{n \times d}$. To simplify the notation in this subsection, we use $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ to represent $\mathbf{S}^q$, $\mathbf{S}^k$, and $\mathbf{S}^v$, respectively.

The output of the self-attention sub-layer can then be computed using

$$\text{Att}_{\text{self}}(\mathbf{S}) \quad = \quad \mathbf{AV}, \tag{118}$$

where $\mathbf{A}$ is an $n \times n$ attention matrix or attention map

$$\mathbf{A} \quad = \quad \text{Softmax}(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}). \tag{119}$$

Here, $\mathbf{M}$ is a masking matrix used to prevent the model from attending to future context at each position. Specifically, for a position i, $M(i,j) = 0$ if $j \leq i$, and $M(i,j) = -\infty$ otherwise. Both the time and space complexity of the self-attention sub-layer scale quadratically with n[10]. As a result, standard self-attention becomes computationally prohibitive for large values of n.

The standard implementation of the above model relies on dense matrix computations, such as the multiplications in Eqs. (118) and (119). One common approach to reducing both memory footprint and floating-point operations is sparsification. By assuming $\mathbf{A}$ is a sparse matrix, only a fraction $\varrho \cdot n^2$ of its entries remain non-zero, where ϱ is the **sparsity ratio**. Using sparse matrix representations drastically reduces memory requirements. Furthermore, computing $\frac{\mathbf{QK}^\top}{\sqrt{d}}$ and $\mathbf{AV}$ becomes more efficient, as the model only processes a small subset of relevant positions.

Given a position i, we define the **attention field** π_i to be the set of positions that are considered in computing the representation at this position. We therefore only need to compute the dot-product attention between the given position i and each position $j \in \pi_i$. This results in a sparse attention matrix $\mathbf{A}'$ where

$$A'(i,j) \quad = \quad \begin{cases} a_{i,j} & j \in \pi_i \text{ and } j \leq i \\ 0 & \text{otherwise} \end{cases}, \tag{120}$$

10. More precisely, the memory footprint is $n^2 + n \cdot d$, which is dominated by the n^2 term when $n \gg d$.

where $a_{i,j}$ is a non-zero weight. A simple implementation of this model involves a slight modification to $\mathbf{M}$, leading to a new masking variable $\mathbf{M}'$

$$M'(i,j) \quad = \quad \begin{cases} 0 & j \in \pi_i \text{ and } j \leq i \\ -\infty & \text{otherwise} \end{cases}. \tag{121}$$

In practical implementations, a more efficient approach is to employ sparse operations for $\mathbf{QK}^\top$ and $\mathbf{A}'\mathbf{V}$ by considering $\mathbf{M}'$ and $\mathbf{A}'$, respectively. That is, we compute only for pairs with non-zero attention weights and skip the rest.

Several approaches to sparse self-attention can be considered.

- **Span-based Attention/Local Attention**. As discussed in Section 4.1, the use of context in sequence modeling is local in many cases. The basic idea of local attention is to restrict attention to a local region of the input sequence. We can then write π_i as

$$\pi_i \quad = \quad [a_i^l, a_i^r], \tag{122}$$

  where a_i^l and a_i^r are the left and right ends of π_i. $a_i^r - a_i^l + 1$ determines the size of the region, and so we can use it to control the sparsity of the attention model, for example, if $a_i^r - a_i^l + 1 \ll n$, the model would be very sparse. a_i^l and a_i^r can be obtained by using either heuristics or machine learning methods. The reader may refer to related papers for more details (Luong et al., 2015; Sperber et al., 2018; Sukhbaatar et al., 2019; Yang et al., 2018). See Figure 13 (b) for an illustration of local attention.

- **Chunked Attention**. In this method, we segment a sequence into chunks and run the attention model on each of them (Parmar et al., 2018; Qiu et al., 2020). Given a sequence $\{1, \cdots, n\}$, we define $\{\text{chunk}_1, \cdots, \text{chunk}_q\}$ to be a segmentation of the sequence. A chunk can be expressed as a span

$$\text{chunk}_k \quad = \quad [c_k^l, c_k^r]. \tag{123}$$

  In the attention step, we treat each chunk as a sequence and perform self-attention on it as usual. In other words, the representation at position i is computed by using only the context in the chunk that i belongs to. In this sense, this model can be thought of as a variant of local attention. Figure 13 (c) shows an illustration of this model. A remaining issue is how to segment the sequence. There are several ways to do this. For example, as discussed in Section 3.4, we can do segmentation from a linguistic perspective, and segment the sequence into linguistically motivated units. In practical systems, it is sometimes more convenient to segment the sequence into chunks that are of equal length. Thus, the sparsity of the model is controlled by the size of these chunks, for example, the use of smaller chunks would lead to a more sparse attention model.

- **Strided Attention**. Since the chunked attention approach enforces a hard segmentation on the input sequence, it may lose the ability to learn representations from inputs in different chunks. An alternative way to achieve chunk-wise attention is to allow overlap between chunks (Ainslie et al., 2020; Beltagy et al., 2020; Child et al., 2019). This approach is analogous to the family of approaches that are commonly

used to apply a local model to 1D or 2D data to generate outputs of the same shape. Like CNNs, we use a context window to represent the field of input of the attention model. The context window slides along the sequence, each time moving forward a step of size stride. As a special case, if stride equals the size of the context window, this model reduces to the chunked attention model mentioned above. If stride is smaller than the size of the context window, the attention model will become denser. Figure 13 (d) shows the case of stride = 1 where the chunk overlapping is maximized. A way to achieve relatively sparser attention is to use a **dilated context window**. Figure 13 (e) shows an example of the dilated strided attention model, where the context window is discontinuous, with gaps of size 1.

- **Learning Attention Fields**. Because the attention field π_i can be any subset of $\{1, \cdots, n\}$, we can develop more general sparse attention models by considering attention maps beyond chunk-based patterns. The only question is how to determine which positions the model attends to for a given position. One simple approach is to use a computationally cheaper model to estimate the "importance" of each position. Then, attention weights are computed only for some of the positions which are thought to be most important (Zhou et al., 2021). A second approach is grouping: positions are grouped, and then the attention weights are computed only for positions in the same group. This can be achieved by clustering keys and queries. For example, we can cluster keys and queries via k-means clustering. The centroids of the clusters can be treated as additional parameters of the attention model, and so can be learned during optimization (Roy et al., 2021). One benefit of learning attention fields is that the model can spread its attention more broadly over the sequence. This is a useful property for many NLP problems because word dependencies are sometimes long-range, not restricted to a local context window. See Figure 13 (f) for an example of the attention map learned through this model. Alternative approaches to learning to attend are to use sorting or hashing functions to group similar key and query vectors (Kitaev et al., 2020; Tay et al., 2020a). These functions can be either heuristically designed functions or neural networks with learnable parameters. By using these functions, we can reorder the sequence so that the inputs in the same group are adjacent in the reordered sequence. In this way, the resulting attention map follows a chunk-wise pattern, and the model is computationally efficient through the use of the chunked attention approach.
- **Hybrid Methods**. Above, we have discussed a range of different sparse attention models. It is natural to explore methods that combine multiple models together to make use of their benefits in some way. A simple way to do this is to combine the attention fields of different models. For example, in Zaheer et al. (2020)'s system, the attention map is generated by considering three different sparse models, including local attention (chunked attention), global attention, and random attention[11]. The resulting model is still a sparse model, but is somewhat more robust as it involves multiple patterns from different perspectives of attention modeling. Another way of

11. Here the global attention model attends each word only to a special word which accounts for the entire sequence and is often placed at the beginning of the sequence. The random attention model attends each word to a random set of the words of the sequence.

combining multiple attention models is to use different models for different heads in multi-head attention (Beltagy et al., 2020; Child et al., 2019). For example, one can use one head as a local attention model, and use another head as a global attention model (see Figure 13 (g-h)).

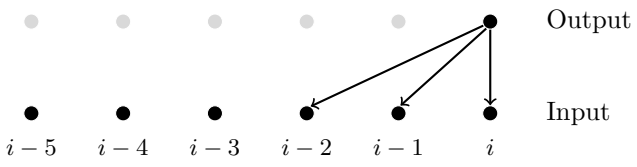
One disadvantage of sparse models compared to dense models is that they are often less efficient on GPUs/CPUs due to hardware constraints. While sparse models can ideally reduce both the memory and computation requirements, the computational throughput of sparse models is much slower than that of dense models. In practice, it is difficult for sparse models to approach the peak FLOPS of a GPU or CPU[12]. Therefore, they are primarily used to improve memory efficiency rather than achieving raw computational speedups on current hardware. On the other hand, sparse models are still of great use to NLP practitioners in the context of memory-efficient Transformer, especially when Transformers are used to deal with extremely long sequences.

5.2 Recurrent and Memory Models

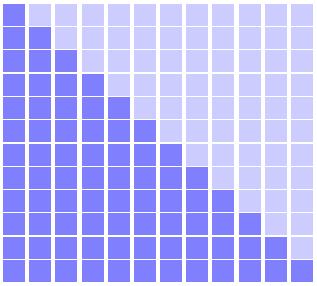
For sequence generation problems, Transformers can also be thought of as memory systems. Consider again the general autoregressive setting, in which we are given the states of the previous $i-1$ positions and wish to predict the next state. In self-attention, this is achieved by using the query at position i (i.e., $\mathbf{q}_i$) to access the key-value pairs of the previous positions (i.e., $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$). Then, we move to position $i+1$ and append $(\mathbf{k}_i, \mathbf{v}_i)$ to the collection of key-value pairs. This procedure can be intuitively interpreted using memory mechanisms. A Transformer model maintains a memory bank that retains past information. As the model processes the sequence, it repeats the same operation: generating outputs by reading from the memory, and then updating the memory to incorporate new information. This is illustrated in Figure 14.

5.2.1 Cache-based Memory

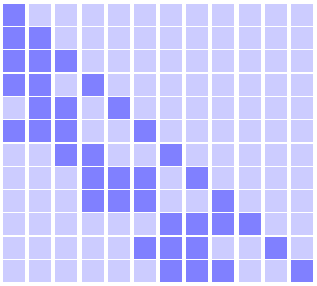
The memory here can be viewed as a datastore of vectors. From a machine learning perspective, this is a non-parametric model, and the cost of accessing the model grows as the sequence becomes longer. Clearly, such a variable-length memory will generally be infeasible if the model deals with extremely long sequences. For modeling sequences of arbitrary length, it is common to use a fixed-length memory instead. As in many NLP problems, one of the simplest ways to do this is to maintain a cache of recent information, that is, we restrict the modeling to a context window. Let n_c be the size of the context window. The model keeps track of the $n_c - 1$ latest states preceding the current position, so that its closest predecessors can be considered at each step. This means that, for each position, a self-attention sub-layer attends to $n_c - 1$ previous positions, as follows



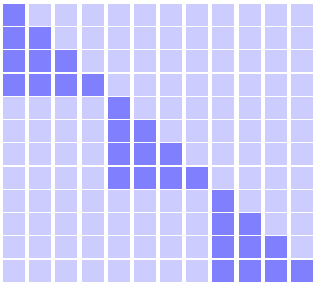
12. FLOPS = floating point operations per second.



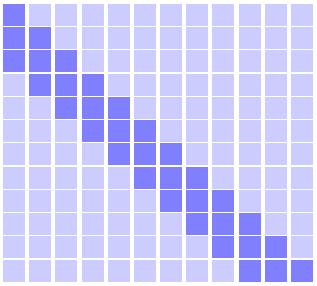
(a) Standard Attention



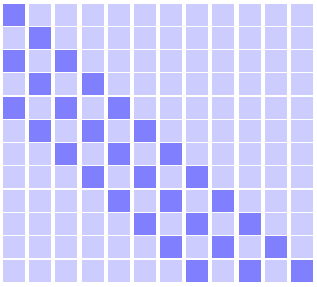
(b) Span-based Attention



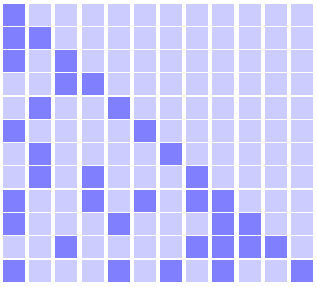
(c) Chunked Attention



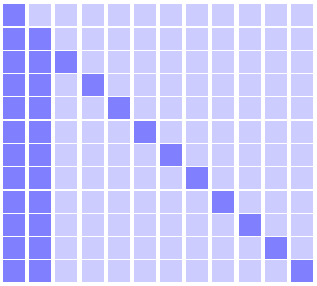
(d) Strided Attention



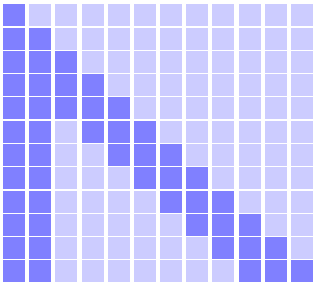
(e) Dilated Strided Attention



(f) Learning Attention Fields



(g) Global Attention



(h) Hybrid Methods

Figure 13: Illustration of the attention maps of different models (self-attention on the decoder side). Dark cells mean $A'(i,j) \neq 0$ (i.e., i attends to j), and light cells mean $A'(i,j) = 0$ (i.e., i does not attend to j). In all these attention maps, we assume that every position attends to itself by default (see diagonals).

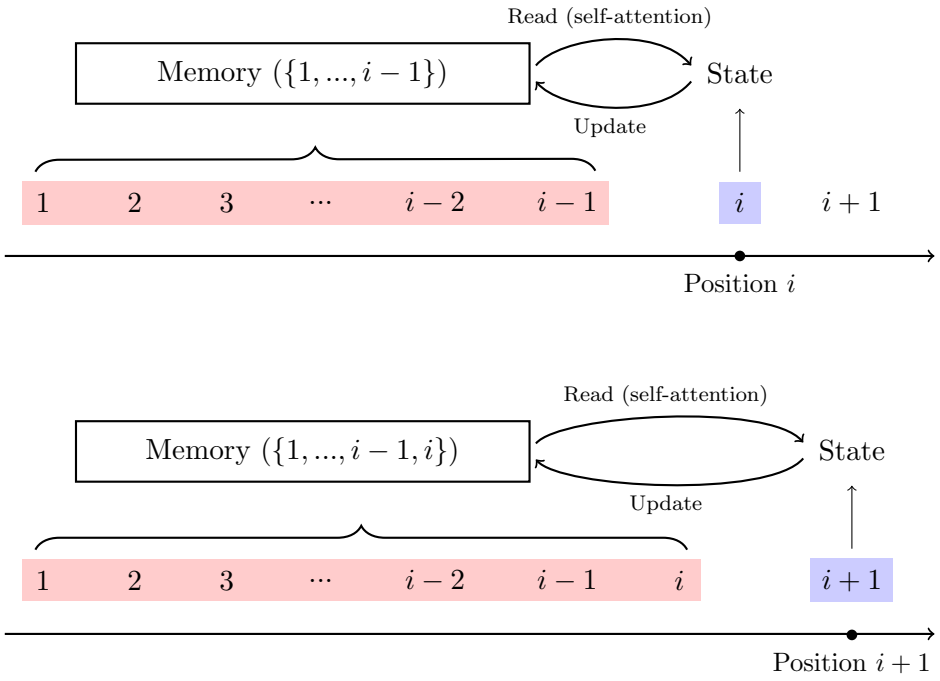
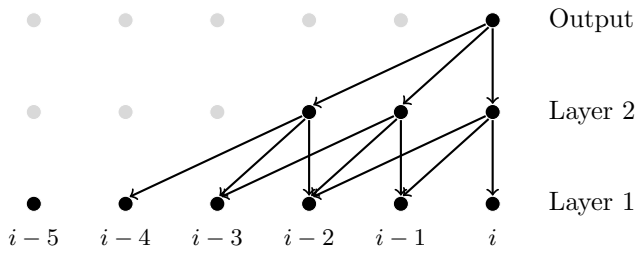


Figure 14: The Transformer as a memory system. At position i, the collection of key-value pairs from positions $\{1, \cdots, i-1\}$ serves as a memory of past context. The model accesses this memory to generate an output, and subsequently adds the key-value pair of position i to the memory. Moving to the next position, this cycle of memory access and update is repeated.

If we stack multiple self-attention sub-layers, a larger context window would be considered. For example, a model involving two self-attention sub-layers has a context window of size $2n_c - 1$, as follows



Therefore, we can capture a sufficiently large context by using a multi-layer Transformer model. Note that the context window model here is essentially the same as the strided attention model presented in the preceding section. Systems of this type are often easy to implement: we slide a window along the sequence, and, in each move, we make predictions at the last position of the window (for inference), or back-propagate errors (for training).

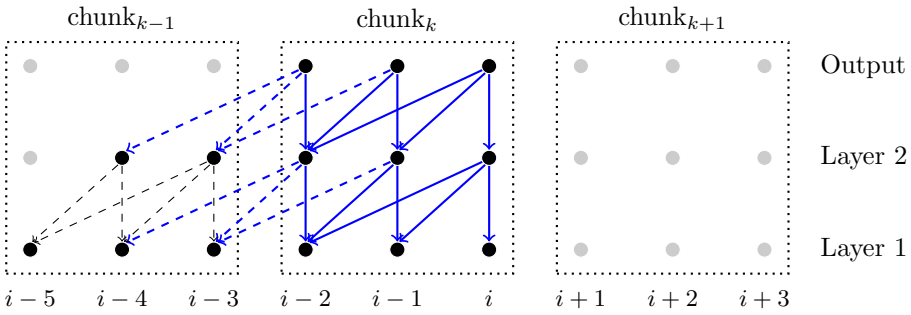
An alternative strategy for training a context window model is chunked attention. The sequence is explicitly partitioned into sub-sequences (or chunks) of a fixed length n_c. These chunks are then treated as independent training samples. This approach, however, completely ignores the relationship between inputs in different chunks. One way to address this issue is to introduce dependence between chunks. For example, the **Transformer-XL** model allows every chunk to access one or more preceding chunks (Dai et al., 2019). In the simplest case, consider an example in which chunk_k can see its predecessor chunk_{k-1}. Each position in chunk_k can attend to all its preceding positions in both chunk_k and chunk_{k-1}.

In Transformer-XL, this approach is implemented in a simplified form. First, each position is constrained to attend to $n_c - 1$ previous positions so that the size of the attention field of a position is the same in the training and inference stages. Such a method turns the problem back to strided attention, making the implementation of the attention model straightforward. However, unlike standard strided attention, Transformer-XL performs training in a chunk-wise manner. Once we finish the training on a chunk, we directly move to the next chunk, rather than sliding the context window a small step forward. Second, while this approach allows for connections between chunks, the parameters of the sub-network corresponding to chunk_{k-1} are fixed, and we only update the parameters of the sub-network on chunk_k in the k-th step. See Figure 15 for an illustration.

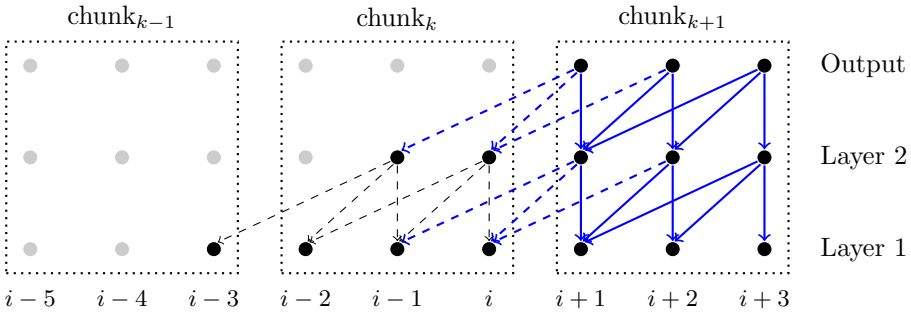
The above model is similar in spirit to recurrent models because both of them require the computation in one step to depend on the states of the preceding steps. However, it is not in the standard form of a recurrent model, in which the output of a recurrent unit in one step is the input in the next step. Instead, the "recurrence" is expressed by involving connections across layers and chunks, that is, the output of one layer in chunk_{k-1} is used as the input of a higher-level layer in chunk_k.

5.2.2 Encoding Long-term Memory

Another idea for representing the states of a sequence is to frame the task as an encoding problem. Instead of storing all key-value vectors during left-to-right generation, we



(a) Step k of chunk-wise training



(b) Step $k+1$ of chunk-wise training

Figure 15: Illustration of chunk-wise training in Transformer-XL (Dai et al., 2019). The input sequence is divided into non-overlapping chunks of length n_c. Training proceeds one chunk at a time. Within chunk$_k$, the attention span for every token is restricted to a left-context window of size n_c. This structure permits cross-chunk attention. For example, position $i-2$ in chunk$_k$ can attend to positions $i-3$ and $i-4$ residing in chunk$_{k-1}$ (see sub-figure (a)). During back-propagation, gradients are computed for the current sub-network (chunk$_k$), treating cached activations from previous chunks as fixed constants. Dashed lines denote forward-pass information flows that do not receive gradients. After completing the training step for chunk$_k$, the process moves to the next chunk and repeats.

compress the memory of the entire "history" into a fixed number of encoded key-value vectors. These encoded vectors can be either a small subset of the original sequence $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$, or a small set of newly generated vectors that encode this entire history.

One way to achieve this encoding is to apply a pooling operation over $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$ (Rae et al., 2019). For example, by using average pooling, the memory is reduced to a single key-value pair $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$:

$$\bar{\mathbf{k}} = \frac{1}{i-1}\sum_{j=1}^{i-1}\mathbf{k}_j, \tag{124}$$

$$\bar{\mathbf{v}} = \frac{1}{i-1}\sum_{j=1}^{i-1}\mathbf{v}_j. \tag{125}$$

This leads to a very efficient model, as we only need to update the aggregated vectors $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$ incrementally (Zhang et al., 2018). Let $(\bar{\mathbf{k}}[i], \bar{\mathbf{v}}[i])$ represent the state of the memory at position i. A more general formulation is given recursively:

$$\bar{\mathbf{k}}[i] = \mathrm{KMem}(\bar{\mathbf{k}}[i-1], \mathbf{k}_{i-1}), \tag{126}$$

$$\bar{\mathbf{v}}[i] = \mathrm{VMem}(\bar{\mathbf{v}}[i-1], \mathbf{v}_{i-1}). \tag{127}$$

where $\mathrm{KMem}(\cdot)$ and $\mathrm{VMem}(\cdot)$ are functions that update the memory by taking both the states of the memory at the previous position (i.e., $\bar{\mathbf{k}}[i-1]$ and $\bar{\mathbf{v}}[i-1]$) and the new states (i.e., $\mathbf{k}_{i-1}$ and $\mathbf{v}_{i-1}$). There are many possible choices for functions $\mathrm{KMem}(\cdot)$ and $\mathrm{VMem}(\cdot)$. For example, if $\mathrm{KMem}(\cdot)$ and $\mathrm{VMem}(\cdot)$ are weighted sum functions, we can derive the same forms as Eqs. (124) and (125). If $\mathrm{KMem}(\cdot)$ and $\mathrm{VMem}(\cdot)$ are recurrent cells (e.g., standard RNN or LSTM cells), we obtain a recurrent model of memory.

Extending the above concept to memories with more than one key-value pair is straightforward. One approach is chunk-level representation. Let $\{(\bar{\mathbf{k}}_1, \bar{\mathbf{v}}_1), \cdots, (\bar{\mathbf{k}}_\kappa, \bar{\mathbf{v}}_\kappa)\}$ be a memory of size κ, where each $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ is a representation of a chunk of length n_c. This memory can thus encode a sequence of maximum length $\kappa \cdot n_c$, computing each $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ via Eqs. (126) and (127) over its corresponding chunk. A second approach is to organize the memory into a priority queue. A learned scoring function, which is often an auxiliary neural network, evaluates and assigns a priority score to each generated key-value pair. High-scoring pairs are inserted into the queue via a push operation. In this way, the memory actively maintains a dynamic collection of only the most valuable key-value pairs across the input sequence.

Although representing memory as a set of vectors is an intuitive design choice for Transformers, its capacity is bounded by the number of vectors. An alternative paradigm is **continuous memory**. This approach leverages function approximation, treating the sets of keys $\{\mathbf{k}_1, \cdots, \mathbf{k}_{i-1}\}$ or values $\{\mathbf{v}_1, \cdots, \mathbf{v}_{i-1}\}$ as a series of data points to be fitted by a continuous function. Instead of explicitly storing these vectors, the memory is parameterized by the functions themselves. A common method is to use a linear combination of simple basis functions to approximate the complex function defined by the discrete key or value vectors (Martins et al., 2022).

It is also straightforward to use a short-term memory and a long-term memory simultaneously so that we can combine the merits of both. For example, we use a cache-based memory to capture local context, and use an efficient long-term memory that encodes the entire history to model long-range dependencies. This idea is also similar to that used in combining different sparse attention models as discussed in the previous subsection.

5.2.3 Retrieval-based Methods

So far in this subsection, we have discussed approaches based on fixed-length models. It is also possible to develop efficient memory models by improving the efficiency of accessing memory, instead of strictly reducing memory capacity. One way to achieve this is to store the past key-value pairs in a database (call it a **vector database**), and to find the most similar key-value pairs when querying the database. To be more precise, given a query $\mathbf{q}$, we use the database to find a set of top-p relevant key-value pairs (denoted by Ω_p) by performing a similarity search based on the dot-product between the query and key vectors. The query $\mathbf{q}$ then attends to Ω_p as in standard self-attention models. The idea behind this method is to consider only a small number of elements that contribute most to the attention result. Therefore, the model is essentially a sparse attention model which is computationally efficient. Another advantage of this method is that it allows for fast similarity search over a very large set of vectors because of the highly optimized implementation of vector databases. Framing memory as a retrieval system falls under the broader paradigm of **retrieval-augmented generation** (**RAG**) or retrieval-augmented approaches. It provides a scalable framework for incorporating external memory into neural architectures like Transformers (Guu et al., 2020; Lewis et al., 2020; Wu et al., 2021).

5.3 Low-dimensional Models

In many practical applications, Transformers are "high-dimensional" models. This is not only because the input and/or output data is in high-dimensional spaces, but also because some of the intermediate representations of the data in the model are high-dimensional. As discussed in Section 5.1, this high dimensionality arises in part from computing the weighted sum of value vectors, as in Eq. (128) (repeated here for convenience)

$$\text{Att}_{\text{self}}(\mathbf{S}) \quad = \quad \mathbf{AV} \tag{128}$$

and computing the attention matrix itself, as in Eq. (129)

$$\mathbf{A} \quad = \quad \text{Softmax}(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}), \tag{129}$$

which involves large matrix multiplications $\mathbf{QK}^\top$ and $\mathbf{AV}$ when the length n and the hidden dimensionality d are large.

The $\mathbf{AV}$ and $\mathbf{QK}^\top$ operations scale with a time complexity of $O(n^2 \cdot d)$ and a space complexity of $O(n^2 + n \cdot d)$. While previous approaches have mitigated this complexity using sparse models, this subsection focuses on methods that approximate these operations via dense computation. One straightforward idea is to project $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into smaller matrices, thereby reducing the computational burden of matrix multiplication. Since $\mathbf{Q}$,

$\mathbf{K}$, and $\mathbf{V}$ all reside in $\mathbb{R}^{n\times d}$, this can be achieved by reducing either the length dimension n, the hidden dimension d, or both.

5.3.1 Reducing n

Note that the output $\text{Att}_{\text{self}}(\mathbf{S})$ is required to be an $n\times d$ matrix, and so we cannot reduce the number of queries. We instead consider reducing the number of keys and values. Suppose n' is an integer with $n' < n$, and $\mathbf{K}$ and $\mathbf{V}$ can be transformed into $n' \times d$ matrices $\mathbf{K}'$ and $\mathbf{V}'$ in some way. We can obtain a "smaller" model simply by replacing $\mathbf{K}$ and $\mathbf{V}$ with $\mathbf{K}'$ and $\mathbf{V}'$, giving

$$\text{Att}_{\text{self}}(\mathbf{S}) = \mathbf{A}\mathbf{V}' \tag{130}$$

$$\mathbf{A} = \text{Softmax}(\frac{\mathbf{Q}[\mathbf{K}']^\top}{\sqrt{d}} + \mathbf{M}). \tag{131}$$

This model is in the standard form of self-attention, but has lower time and space complexities, that is, $O(n' \cdot n \cdot d) < O(n^2 \cdot d)$ and $O(n' \cdot n + n' \cdot d) < O(n^2 + n \cdot d)$. If $n' \ll n$, the resulting model will be linear with n.

The key problem here is how to obtain $\mathbf{K}'$ and $\mathbf{V}'$ in a way that retains much of the information in $\mathbf{K}$ and $\mathbf{V}$. There are several ways to do so. One simple method is to select the keys and values that are thought to be important. The importance of a key (or value) can be computed in terms of some computationally inexpensive measure. For example, we can sample a small number of query-key dot-products and estimate the importance of a key by collecting these dot-product results.

Although straightforward, the above method still relies on sparse operations like sampling and aggregation. Alternatively, dense computation can be used to project $\mathbf{K}$ and $\mathbf{V}$ down to $\mathbf{K}'$ and $\mathbf{V}'$. A common approach employs CNNs (Liu et al., 2018). Let $\text{Conv}(\cdot)$ denote a 1D convolution operation sliding along the n dimension. $\mathbf{K}'$ is then computed as:

$$\mathbf{K}' = \text{Conv}(\mathbf{K}, \mathbf{W}_c, \text{size}_r, \text{stride}), \tag{132}$$

where $\mathbf{W}_c$ is the parameter matrix of the filters, size_r is the size of the receptive field, and stride is the step size of the convolution. In general, we can achieve a high compression rate by choosing large values for size_r and stride. Likewise, we can compute $\mathbf{V}'$ using another convolutional function. It is worth noting that, if the parameter n' is fixed for all samples, compression of $\mathbf{K}$ and $\mathbf{V}$ along the length dimension is essentially the same as the fixed-length memory model as described in the preceding subsection. The methods presented here are more general and could be applied to variable-length memories.

We might also be tempted to model the attention function by considering the attention matrix $\mathbf{A}$ as a high-dimensional representation of data and then applying conventional dimensionality reduction methods. Empirically, in many problems, $\mathbf{A}$ (or more precisely $\mathbf{Q}\mathbf{K}^\top$) is found to be a low-rank matrix. In this case, we can compress $\mathbf{A}$ while retaining as much information as possible. There are many ways to do so. For example, we might use a product of smaller matrices as an approximation to $\mathbf{A}$ via the SVD technique. However, this introduces computational overhead in using SVD compared with the standard attention model. A simpler idea is to directly transform $\mathbf{K}$ and $\mathbf{V}$ into smaller-sized matrices via linear

mappings, given by

$$\mathbf{K}' = \mathbf{U}^k\mathbf{K}, \tag{133}$$
$$\mathbf{V}' = \mathbf{U}^v\mathbf{V}, \tag{134}$$

where $\mathbf{U}^k \in \mathbb{R}^{n'\times n}$ and $\mathbf{U}^v \in \mathbb{R}^{n'\times n}$ are parameter matrices. Clearly, this leads to a model which is equivalent to that described in Eqs. (130) and (131). While highly intuitive, this approach has been proven to yield a sufficiently small approximation error ϵ, provided that n' scales linearly with d/ϵ^2 (Wang et al., 2020b).

5.3.2 Reducing d

Another approach to reducing computational complexity is to reduce the dimensionality d. One of the simplest methods is to project all queries and keys onto a d'-dimensional space ($d' < d$), and to compute the dot-product between query-key pairs in the new space. For modeling, we only need to replace $\mathbf{Q} \in \mathbb{R}^{n\times d}$ and $\mathbf{K} \in \mathbb{R}^{n\times d}$ with new representations $\mathbf{Q}' \in \mathbb{R}^{n\times d'}$ and $\mathbf{K}' \in \mathbb{R}^{n\times d'}$. We can easily modify Eq. (129) to use $\mathbf{Q}'$ and $\mathbf{K}'$ in computing the attention matrix

$$\mathbf{A} = \text{Softmax}(\frac{\mathbf{Q}'[\mathbf{K}']^\top}{\sqrt{d'}} + \mathbf{M}). \tag{135}$$

$\mathbf{Q}'$ and $\mathbf{K}'$ are given by

$$\mathbf{Q}' = \mathbf{Q}\mathbf{U}^q, \tag{136}$$
$$\mathbf{K}' = \mathbf{K}\mathbf{U}^k, \tag{137}$$

where $\mathbf{U}^q \in \mathbb{R}^{d\times d'}$ and $\mathbf{U}^k \in \mathbb{R}^{d\times d'}$ are parameter matrices of linear transformations.

It is also possible to exploit **kernel methods** to obtain an efficient dot-product attention model. The basic idea is to map data points (represented as vectors) from one space to another, transforming a problem that is difficult in the original space into one that is more tractable in the new space. The "kernel trick" allows us to compute such inner products implicitly via a kernel function, without explicitly constructing the mapping $\phi(\cdot)$[13]. Such inner products in feature space are commonly represented by kernel functions, denoted as $K(\cdot,\cdot)$.

It is interesting to approximate $\mathbf{A}$ in a fashion analogous to $K(\cdot,\cdot)$ in kernel methods. To illustrate, note in Eq. (129) $\mathbf{A}$ represents normalized attention weights. The numerator can be written in the form

$$\widetilde{\mathbf{A}} = \text{Mask}(\exp(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}})). \tag{139}$$

13. In mathematical analysis, the inner product is a generalized notion of the dot-product, typically denoted by $\langle\cdot,\cdot\rangle$. A formal definition requires that $\langle\cdot,\cdot\rangle$ satisfies several properties in a vector space. Although the inner product takes different forms in different contexts, in Euclidean space $\mathbb{R}^d$, it is equivalent to the dot-product. That is, given two vectors $\mathbf{a} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}^d$, we have

$$\langle\mathbf{a},\mathbf{b}\rangle = \mathbf{a}\cdot\mathbf{b} = \sum_{i=1}^{d} a_i \cdot b_i \tag{138}$$

Here Mask$(\cdot)$ is a function which has the same effect as using the additive masking variable $\mathbf{M}$. Then, $\mathbf{A}$ can be expressed as

$$\mathbf{A} = \mathbf{D}^{-1}\widetilde{\mathbf{A}}, \tag{140}$$

where $\mathbf{D}$ is an $n \times n$ diagonal matrix. Each entry of the main diagonal is the sum of the entries of the corresponding row in $\widetilde{\mathbf{A}}$, denoting the normalization factor of softmax. Substituting this equation into Eq. (129), we have

$$\mathrm{Att}_{\mathrm{self}}(\mathbf{S}) = \mathbf{D}^{-1}\widetilde{\mathbf{A}}\mathbf{V}. \tag{141}$$

In this model, $\widetilde{A}(i,j)$ can be viewed as a similarity function over all query-key pairs in a d-dimensional space. Here we assume that this function, which is in the form of the dot-product of vectors, can be approximated by a kernel function

$$\begin{aligned}\widetilde{A}(i,j) &= K(\mathbf{q}_i, \mathbf{k}_j) \\ &= \langle \phi(\mathbf{q}_i), \phi(\mathbf{k}_j) \rangle.\end{aligned}$$

$\phi(\cdot)$ is a mapping from $\mathbb{R}^d$ to $\mathbb{R}^{d'}$. We can represent the queries and keys in the following form

$$\begin{aligned}\mathbf{Q}' &= \phi(\mathbf{Q}) \\ &= \begin{bmatrix} \phi(\mathbf{q}_1) \\ \vdots \\ \phi(\mathbf{q}_n) \end{bmatrix}, \qquad (142) \\ \mathbf{K}' &= \phi(\mathbf{K}) \\ &= \begin{bmatrix} \phi(\mathbf{k}_1) \\ \vdots \\ \phi(\mathbf{k}_n) \end{bmatrix}. \qquad (143)\end{aligned}$$

Then, we develop a kernelized attention model by approximating the attention weight $\alpha_{i,j}$ in the form

$$\alpha_{i,j} \approx \frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top}. \tag{144}$$

The key idea behind this kernelized attention model is that we can remove the softmax function if the queries and keys are mapped to a new space. Using this approximation, the i-th output vector of the attention model (i.e., the i-th row vector of $\mathrm{Att}_{\mathrm{self}}(\mathbf{S})$) is given by

$$\begin{aligned}\mathbf{c}_i &= \sum_{j=1}^{n} \alpha_{i,j} \cdot \mathbf{v}_j \\ &\approx \sum_{j=1}^{n} \Big(\frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \cdot \mathbf{v}_j \Big) \\ &= \frac{\sum_{j=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top \mathbf{v}_j}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \\ &= \frac{\phi(\mathbf{q}_i)\big(\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j\big)}{\phi(\mathbf{q}_i)\big(\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top\big)}.\end{aligned} \tag{145}$$

Here $\phi(\mathbf{q}_i) \in \mathbb{R}^{1\times d'}$, while $\phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d'\times d}$. Therefore, the inner term $\sum_{j=1}^n \phi(\mathbf{k}_j)^\top \mathbf{v}_j$ is a $d' \times d$ matrix, and the final product yields a vector in $\mathbb{R}^{1\times d}$.

Although the equation appears a bit complicated, the idea is simple: instead of attending the query to all keys to obtain the attention weight $\alpha_{i,j}$, we can compute the sum of the multiplications $\sum_{j=1}^n \phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d'\times d}$ and then multiply it with the kernelized query $\phi(\mathbf{q}_i)$. Returning to the notation used in Eq. (141), we define the i-th diagonal entry of $\mathbf{D}$ to be $\phi(\mathbf{q}_i)\sum_{j'=1}^n \phi(\mathbf{k}_{j'})^\top$. Then, the attention model can be re-expressed in the form

$$\begin{aligned} \mathrm{Att}_{\mathrm{self}}(\mathbf{S}) &= \mathbf{D}^{-1}\phi(\mathbf{Q})\phi(\mathbf{K})^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\mathbf{Q}'\mathbf{K}'^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\big(\mathbf{Q}'(\mathbf{K}'^\top \mathbf{V})\big). \end{aligned} \tag{146}$$

Here we change the order of computation from left-to-right to right-to-left using parentheses. Given that $\mathbf{Q}' \in \mathbb{R}^{n\times d'}$ and $\mathbf{K}' \in \mathbb{R}^{n\times d'}$, this model has time and space complexities of $O(n\cdot d\cdot d')$ and $O(n\cdot d + n\cdot d' + d\cdot d')$, respectively. Therefore, the model is linear with respect to the sequence length n, and is sometimes called the **linear attention model**. One computational advantage of this model is that we need only compute the multiplication $\mathbf{K}'^\top \mathbf{V}$ (i.e., $\sum_{j=1}^n \phi(\mathbf{k}_j)^\top \mathbf{v}_j$) and the corresponding normalization factor (i.e., $\sum_{j'=1}^n \phi(\mathbf{k}_{j'})^\top$) once. The results can then be used for any query (Katharopoulos et al., 2020). The model needs to maintain $\sum_{j=1}^n \phi(\mathbf{k}_j)^\top \mathbf{v}_j$ and $\sum_{j'=1}^n \phi(\mathbf{k}_{j'})^\top$ and update them when new key and value vectors arrive.

Still, there are several limitations regarding this kernelized model, for example, how to develop the feature map $\phi(\cdot)$ to obtain a good approximation to the standard attention model. Interested readers may refer to Choromanski et al. (2020)'s work for more details.

A second idea for reducing d is to take sub-space models, in which a problem in a d-dimensional space is transformed into sub-problems in lower-dimensional spaces, and the solution to the original problem is approximated by some combination of the solutions to these sub-problems. In a general sub-space model, a d-dimensional key vector $\mathbf{k}$ can be mapped into a set of d'-dimensional vectors $\{\mathbf{K}'_1, \cdots, \mathbf{K}'_\eta\}$. To simplify modeling, we can do this by vector segmentation, that is, we segment $\mathbf{k}$ into η sub-vectors, each having $d' = \frac{d}{\eta}$ dimensions. We can transform all query and value vectors in the same way. Then, the attention model is applied in each of these sub-spaces.

This method, however, does not reduce the total amount of computation. As presented in Lample et al. (2019)'s work, we can instead approximate the dot-product attention over a set of key-value pairs by considering top-p candidates in each sub-space. More precisely, we find p-best key-value pairs in each sub-space, which is computationally cheaper. The Cartesian product of these p-best key sets consists of p^η product keys. Likewise, we obtain p^η product values. The remaining work is simple: the d-dimensional queries attend to these d-dimensional product keys and values. An interesting difference between this sub-space model and the d-dimensional space model is that the generated product keys and values may be different from any of the original key-values $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$. This provides a way for learning new representations of the past information.

So far we have discussed approaches to dimensionality reduction along either the n or d dimension. It is straightforward to combine them to develop a "lower-dimensional" model. As an example, suppose that we have the $n \to n'$ reduction for keys and values, and the

$d \to d'$ reduction for queries and keys. The model takes the form

$$\begin{aligned} \text{Att}_{\text{self}}(\mathbf{S}) &= \mathbf{A}\mathbf{V}', \\ \mathbf{A} &= \text{Softmax}(\frac{\mathbf{Q}'\mathbf{K}'^{\top}}{\sqrt{d'}} + \mathbf{M}), \end{aligned} \tag{147}$$

where $\mathbf{Q}' \in \mathbb{R}^{n \times d'}$, $\mathbf{K}' \in \mathbb{R}^{n' \times d'}$, and $\mathbf{V}' \in \mathbb{R}^{n' \times d'}$ are low-dimensional representations for queries, keys and values. As usual, we can easily obtain these representations through the linear mappings of $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$. The time and space complexities of this model are $O(n' \cdot n \cdot d')$ and $O(n' \cdot n + n' \cdot d')$.

5.4 Parameter and Activation Sharing

Redundancy is common to most large-scale neural networks. As a result, many of these models are over-parameterized, making training and inference less efficient. One common approach to reducing redundancy is to simplify the model by removing unnecessary components. For example, we can either prune a complex model or share sub-modules across different parts of the model to obtain a more compact model. In this subsection, we discuss methods for sharing parameters and intermediate states in Transformer models. We leave the discussion of model transfer and pruning to Section 5.7.

Shared-parameter architectures are widely used in neural network-based systems. Well-known examples include CNNs and RNNs, where the same set of parameters (or layers) is applied across different regions of the input. This produces a large neural network, parts of which have the same architecture and the same shared parameters. For Transformers as well as other sequence models, the sharing mechanism can be applied to different levels of modeling. A simple example is embedding sharing. In machine translation, a typical strategy for dealing with words in two languages is to develop two separate embedding models. Alternatively, one can use a single embedding model for both languages. The parameters of the model are then learned during the training of both the source-side and target-side networks. Such a strategy is also often adopted in multilingual sequence models, such as language models that are able to deal with text in many different languages.

For multi-layer neural networks, a popular method is layer-wise sharing. Suppose there is a stack of layers, all of which have the same form

$$\mathbf{S}^l = \text{Layer}(\mathbf{S}^{l-1}; \theta^l). \tag{148}$$

We can tie the parameters for some or all of these layers. For example, given a set of layers $\{l_1, l_2, \cdots, l_n\}$, we enforce the constraint $\theta^{l_1} = \theta^{l_2} = \cdots = \theta^{l_n}$, so that we can obtain a smaller model and the optimization of the model can be easier. In practice, this shared-layer model is highly advantageous if many layers are involved, because we can repeat the same process many times to construct a very deep neural network (Dehghani et al., 2018). For example, sharing a single FFN sub-layer across all encoder layers is found to be effective in reducing the redundancy in machine translation systems (Pires et al., 2023).

For Transformers, sharing can also be performed in multi-head attention. An example of this is **multi-query attention** (Shazeer, 2019). Recall from Section 2.3 that the output

of a head h in standard multi-head self-attention can be written as

$$\begin{aligned}\mathbf{C}_h^{\text{head}} &= \text{Att}_{\text{qkv}}(\mathbf{S}_h^q, \mathbf{S}_h^k, \mathbf{S}_h^v) \\ &= \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_h^k, \mathbf{S}\mathbf{W}_h^v). \end{aligned} \tag{149}$$

Here $\mathbf{S}_h^q = \mathbf{S}\mathbf{W}_h^q$, $\mathbf{S}_h^k = \mathbf{S}\mathbf{W}_h^k$, and $\mathbf{S}_h^v = \mathbf{S}\mathbf{W}_h^v$ are the queries, keys, and values, which are obtained by linearly transforming the input $\mathbf{S}$ with distinct parameter matrices $\mathbf{W}_h^q$, $\mathbf{W}_h^k$, and $\mathbf{W}_h^v$. In multi-query attention, we share the same keys and values across all the heads, but use different queries for different heads. The form of this model is given by

$$\mathbf{C}_h^{\text{head}} = \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_0^k, \mathbf{S}\mathbf{W}_0^v). \tag{150}$$

Because the keys $\mathbf{S}\mathbf{W}_0^k$ and values $\mathbf{S}\mathbf{W}_0^v$ are independent of h, they are computed and stored only once, rather than redundantly for each head. This yields significant computational and memory bandwidth savings during inference, especially when the number of attention heads is large. Due to its efficiency, multi-query attention (and its variant, grouped-query attention) has been successfully integrated into modern large language models, including Llama 2 (Touvron et al., 2023b) and Falcon[14].

By extending the idea of sharing to more general situations, intermediate states can, in principle, be shared across a neural network. Reusing neuron activations, for instance, allows sub-components to be evaluated multiple times efficiently. In Transformers, this method can be applied directly to the self-attention matrices. Studies have empirically shown that attention maps across adjacent layers are often highly correlated in certain NLP tasks (Xiao et al., 2019). As a result, a computationally efficient strategy is to compute the dense attention map once and reuse it across subsequent layers.

More broadly, the sharing mechanism can be viewed as a process by which we reuse previously computed results rather than recomputing them on the fly. It is thus possible to reuse the information across different runs of a neural network. A related example is **reversible residual networks**, in which activations of one layer can be recovered from the activations of the following layer (Gomez et al., 2017). Hence we only keep the output of the latest layer in the forward pass. Then, in the backward pass of training, we reconstruct the output of each layer from its successor. One advantage of this reversible design is that the information produced in the forward pass is shared implicitly, and the model is memory-efficient (Kitaev et al., 2020).

5.5 Alternatives to Self-Attention

We have seen that the use of self-attention is one of the main sources of high computational and memory costs in Transformers. It is natural to wonder if there are efficient alternatives to self-attention models. Here, we briefly present some Transformer variants in which self-attention sub-layers are replaced with other types of neural architectures.

5.5.1 CNNs as a Replacement for Self-Attention

CNNs are simple and widely used neural networks, and are considered potential alternatives to self-attention models. To apply CNNs to Transformers, all we need is to construct

14. https://falconllm.tii.ae/index.html

a convolutional sub-layer to replace the self-attention sub-layer in a Transformer block. While a CNN filter has a restricted receptive field and thus takes inputs from a "local" context window, large contexts can be easily modeled by stacking multiple convolutional sub-layers. One key advantage of CNNs is that their computational complexity scales linearly with the sequence length n, compared to the quadratic scaling of self-attention models. Furthermore, the availability of highly optimized CNN implementations makes them easy to apply to sequence modeling in practical systems. To further improve memory efficiency, we can utilize lightweight CNN variants, such as depthwise CNNs (Wu et al., 2018a) [15].

5.5.2 Linear Attention

As with many practical approaches to sequence modeling, there is also considerable interest in developing linear models in order to speed up the processing of long sequences. While there are many ways to define a linear model, one general form that is commonly used in sequence models is

$$\mathbf{z}_i \quad = \quad f(a \cdot \mathbf{z}_{i-1} + b \cdot \mathbf{s}_i). \tag{152}$$

Here $\mathbf{s}_i$ represents some intermediate states of the model at step i, and $\mathbf{z}_i$ represents the summary of the past states up to step i. It is easy to see that this is a recurrent model: the output at step i depends only on the input at the current step and the output at the previous step. As with the popular design choices in neural network-based systems, the linear part is followed by a transformation $f(\cdot)$ which can be either an activation function or a feedforward neural network. Note that, Eq. (152) defines a standard linear model only if $f(\cdot)$ is a linear function. The use of $f(\cdot)$ gives greater flexibility in modeling the problem, although the term *linear model* may not be applied if $f(\cdot)$ chooses a non-linear form.

The above formula describes a linearly structured model which can be seen as an instance of a general family of mathematical models. Typically, it can be represented as a chain structure, or an ordered set of nodes. The model repeats the same computation process from the first node to the last, each time taking the information from the current and previous steps and producing an output vector that is used in the following time steps. As a result, the space and time cost of the model scales linearly with the length of the chain.

15. Recall that in CNNs, a filter (or a set of filters) maps the input variables within its receptive field into an output variable (or a set of output variables) via a linear mapping. Suppose the input and output of a problem are represented as sequences of feature vectors. Given a standard filter with a $k \times d$ receptive field, we slide it along the sequence. At each step, the filter takes $k \times d$ input features and produces a single output feature. This procedure is typically expressed as:

$$y \quad = \quad \text{ReduceSum}(\mathbf{x} \odot \mathbf{W}), \tag{151}$$

where $\mathbf{x} \in \mathbb{R}^{k \times d}$ is the matrix representation of the input, $y \in \mathbb{R}$ is the output feature, and $\mathbf{W} \in \mathbb{R}^{k \times d}$ is the weight matrix. The function ReduceSum$(\cdot)$ computes the sum of all element-wise products between $\mathbf{x}$ and $\mathbf{W}$. If we want the input and output to have the same number of features d, we must apply d distinct filters, resulting in a total of $d \cdot (k \cdot d) = d^2 \cdot k$ parameters. In depthwise CNNs, we decouple the spatial convolution from the feature dimensions. Instead of applying full $k \times d$ filters, each of the d input feature channels is convolved independently with its own 1D spatial filter of size k. Thus, the number of unique parameters is drastically reduced to $d \cdot k$ (with each of the d channels corresponding to a dedicated filter containing k unique parameters).

We can extend Eq. (152) to a standard RNN model by simply applying a linear transformation of the current input and the previous state, that is, $\mathbf{z}_i = f(\mathbf{z}_{i-1} \cdot \mathbf{W}_z + \mathbf{s}_i \cdot \mathbf{W}_s)$. It is thus straightforward to apply RNNs and their variants to Transformer to obtain a hybrid model. For example, we can use LSTM and GRUs in building some of the Transformer layers to combine the merits of both recurrent models and self-attentive models (Chen et al., 2018b). As conventional recurrent models are well known, we skip the discussion of them here.

In fact, we may be more interested in developing linear attention models, so that we can obtain an efficient system, while still retaining the benefit of globally attentive sequence modeling. Part of the difficulty in doing this is that the form of self-attention is not linear. Let us take a moment to see how this difficulty arises. Recall that the result of self-attention can be written in the following form

$$\begin{aligned} \text{Att}_{\text{self}} &= \mathbf{A} \cdot \mathbf{V} \\ &= \psi(\mathbf{Q} \cdot \mathbf{K}^\top) \cdot \mathbf{V}. \end{aligned} \tag{153}$$

Here $\psi(\cdot)$ is a function that is composed of scaling, exponentiation, masking, and normalization operations (i.e., $\psi(\mathbf{a}) = \text{Normalize}(\text{Mask}(\exp(\frac{\mathbf{a}}{\sqrt{d}})))$). Because $\psi(\cdot)$ is a complex non-linear function, there is no straightforward way to simplify the computation directly, and we typically need to calculate the two matrix multiplications separately (one inside $\psi(\cdot)$ and one outside $\psi(\cdot)$). As a consequence, we need to store all the key-value pairs explicitly, and visit each of them given a query. Not surprisingly, this leads to a model whose computational cost grows quadratically with the sequence length n.

Although in self-attention keys and values are coupled, they are used in separate steps. An elegant solution decouples the query from the key-value interaction so that the context information can be encoded independently of any specific query. The key insight is to bypass the non-linear softmax by applying a feature map $\phi(\cdot)$ to the queries and keys. As introduced in Section 5.3, we transform $\mathbf{Q}$ and $\mathbf{K}$ into $\mathbf{Q'} = \phi(\mathbf{Q}) \in \mathbb{R}^{n \times d'}$ and $\mathbf{K'} = \phi(\mathbf{K}) \in \mathbb{R}^{n \times d'}$. The attention mechanism can then be reformulated as:

$$\begin{aligned} \text{Att}_{\text{self}} &\equiv \psi'(\mathbf{Q'} \cdot \mathbf{K'}^\top) \cdot \mathbf{V} \\ &= \mathbf{D}^{-1}(\mathbf{Q'} \cdot \mathbf{K'}^\top) \cdot \mathbf{V} \\ &= \mathbf{D}^{-1}\Big(\mathbf{Q'} \cdot (\mathbf{K'}^\top \cdot \mathbf{V})\Big), \end{aligned} \tag{154}$$

where $\mathbf{D}$ is the diagonal normalization matrix, and $\psi'(\mathbf{Z}) = \mathbf{D}^{-1}\mathbf{Z}$[16]. In this transformed space, the query-key dot product does not require exponential softmax normalization. It corresponds to a normalization by multiplying with $\mathbf{D}^{-1}$. Because the core operation is now pure matrix multiplication, we can exploit matrix associativity to change the order of computation.

This yields a powerful autoregressive formulation: keys and values are first aggregated via $\mathbf{K'}^\top \cdot \mathbf{V}$, and then queries attend to this global representation. Given that $\mathbf{K'}^\top \cdot \mathbf{V} = \sum_{j=1}^{n} \mathbf{k'}_j^\top \cdot \mathbf{v}_j$, we can write $\mathbf{K'}^\top \cdot \mathbf{V}$ in the form of Eq. (152), as follows

$$\boldsymbol{\mu}_j = \boldsymbol{\mu}_{j-1} + \mathbf{k'}_j^\top \cdot \mathbf{v}_j. \tag{155}$$

16. If $\mathbf{D}$ is a diagonal matrix, we occasionally denote $\mathbf{D}^{-1}\mathbf{A}$ as $\frac{\mathbf{A}}{\mathbf{D}}$.

Here $\boldsymbol{\mu}_j \in \mathbb{R}^{d' \times d}$ is a variable that adds $\mathbf{k'}_j^\top \cdot \mathbf{v}_j$ at a time. Likewise, we can define another variable $\boldsymbol{\nu}_j \in \mathbb{R}^{d'}$

$$\boldsymbol{\nu}_j \quad = \quad \boldsymbol{\nu}_{j-1} + \mathbf{k'}_j^\top. \tag{156}$$

Then, the output of self-attention for the j-th query can be written as (see also Eq. (145))

$$\text{Att}_{\text{self},j} \quad = \quad \frac{\mathbf{q}'_j \cdot \boldsymbol{\mu}_n}{\mathbf{q}'_j \cdot \boldsymbol{\nu}_n}. \tag{157}$$

Clearly, this yields a linear-time model, because $\boldsymbol{\mu}_n$ and $\boldsymbol{\nu}_n$ are linear with respect to n. In simple implementations of this model, only $\boldsymbol{\mu}_j$ and $\boldsymbol{\nu}_j$ are kept. Each time a new query is encountered, we update $\boldsymbol{\mu}_j$ and $\boldsymbol{\nu}_j$ using Eqs. (155) and (156), and then compute $\text{Att}_{\text{self},j} = \frac{\mathbf{q}'_j \cdot \boldsymbol{\mu}_j}{\mathbf{q}'_j \cdot \boldsymbol{\nu}_j}$[17].

One straightforward extension to the linear attention model is to allow Eqs. (155) and (156) to combine different terms with different weights. For example, we can redefine $\boldsymbol{\mu}_j$ and $\boldsymbol{\nu}_j$ as

$$\boldsymbol{\mu}_j \quad = \quad a \cdot \boldsymbol{\mu}_{j-1} + (1-a) \cdot \mathbf{k'}_j^\top \cdot \mathbf{v}_j, \tag{158}$$

$$\boldsymbol{\nu}_j \quad = \quad a \cdot \boldsymbol{\nu}_{j-1} + (1-a) \cdot \mathbf{k'}_j^\top \tag{159}$$

and train the parameter a as usual. Also, we can treat a as a gate and use another neural network to compute a (Peng et al., 2021). Another model design is to add more terms to Eqs. (155) and (156) in order to provide a more expressive formulation of the linear attention approach (Bello, 2020; Schlag et al., 2021).

We have seen a general idea of designing linear models for the attention mechanism. The key design choice of such models is to remove the softmax-based normalization, thereby yielding linear forms of representation based on various intermediate states of the models. This motivates several recently developed alternatives to self-attention in which efficient inference systems are developed on the basis of recurrent models of sequence modeling (Peng et al., 2023; Sun et al., 2023). While these systems have different architectures, the underlying models have a similar form, as described in Eq. (152). Note that, by using the general formulation of recurrent models, we are not restricted to standard QKV attention. Instead we may give new meanings and forms to the queries, keys, and values.

The discussion here is also related to the memory models discussed in Section 5.2. From a memory perspective, the keys and values can be treated as encodings of the context. Therefore, in the linear attention model above we have a memory system in which two simple variables $\boldsymbol{\mu}_j$ and $\boldsymbol{\nu}_j$ are used to represent all the context information up to position j. This results in a fixed-length memory which is very useful in practice. There are also other linear approaches to encoding long sequences. For example, we can view the **moving average** model as an instance of Eq. (152), and average a series of state vectors of a Transformer model, either weighted or unweighted.

17. In autoregressive generation, we generate a sequence from left to right. In this case, we need not consider the keys and values for positions $> j$.

5.5.3 State-Space Models

In control systems, **state-space models** (**SSMs**) are representations of a system whose input and output are related by some **state variables** (or states for short), and whose dynamics are described by first-order differential equations of these states. As a simple example, we consider a continuous-time linear time-invariant system which is given in the form of the state-space representation. We adopt a row-vector convention throughout this section, where all variables are represented as row vectors and linear transformations are applied on the right:

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}, \tag{160}$$

$$\mathbf{o}(t) = \mathbf{z}(t) \cdot \mathbf{C} + \mathbf{s}(t) \cdot \mathbf{D}. \tag{161}$$

Here, $\mathbf{s}(t)$, $\mathbf{o}(t)$, and $\mathbf{z}(t)$ represent the values of the input variable, output variable, and state variable at time t, respectively[18]. In a general setting, these variables may possess different dimensionalities. For simplicity, we assume $\mathbf{s}(t), \mathbf{o}(t) \in \mathbb{R}^d$ and $\mathbf{z}(t) \in \mathbb{R}^{d_z}$[19]. Eq. (160) is called the **state equation**, where $\mathbf{A} \in \mathbb{R}^{d_z \times d_z}$ is the state matrix and $\mathbf{B} \in \mathbb{R}^{d \times d_z}$ is the input matrix. Eq. (161) is called the **output equation**, where $\mathbf{C} \in \mathbb{R}^{d_z \times d}$ is the output matrix and $\mathbf{D} \in \mathbb{R}^{d \times d}$ is the feedforward matrix.

These equations describe a continuous mapping from the input $\mathbf{s}(t)$ to the output $\mathbf{o}(t)$ over time. They are, therefore, often used to deal with continuous time series data. To apply this model to sequence modeling, we need to modify the above equations to obtain a discrete form of the state-space representation. Suppose that $\{\mathbf{s}_0, \mathbf{s}_1, \cdots, \mathbf{s}_n\}$ is a sequence of input data points sampled from $\mathbf{s}(t)$ with time step Δt. Similarly, we define $\{\mathbf{z}_0, \mathbf{z}_1, \cdots, \mathbf{z}_n\}$ and $\{\mathbf{o}_0, \mathbf{o}_1, \cdots, \mathbf{o}_n\}$ as sequences of the state and output vectors. Given this notation, we now have a discretized version of the SSM, written as

$$\mathbf{z}_t = \mathbf{z}_{t-1} \cdot \overline{\mathbf{A}} + \mathbf{s}_t \cdot \overline{\mathbf{B}}, \tag{162}$$

$$\mathbf{o}_t = \mathbf{z}_t \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{163}$$

This formulation of the SSM defines an RNN with a residual connection. To be more precise, Eq. (162) describes a recurrent unit that reads the input at step t and the state at step $t-1$, without using any activation function. Eq. (163) describes an output layer that sums the linear transformations of the state $\mathbf{z}_t$ and the residual mapping $\mathbf{s}_t$.

The parameters $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, $\overline{\mathbf{C}}$, and $\overline{\mathbf{D}}$ can be induced from $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ and $\mathbf{D}$ in several different ways, depending on how Eq. (160) is approximated by Eq. (162)[20]. One approach to time

18. We use boldface letters to emphasize that these variables are vectors.
19. In general state-space theory, these variables and matrices are defined over the field of complex numbers. Because models defined over the complex field generalize to real-valued state spaces, we restrict our discussion to the multi-dimensional real number field for simplicity.
20. The discretization process can be interpreted as numerically solving the underlying differential equation. Note that Eq. (160) is an ODE:

$$\frac{d\mathbf{z}(t)}{dt} = g(\mathbf{z}(t), t), \tag{164}$$

where

$$g(\mathbf{z}(t), t) = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}. \tag{165}$$

discretization is the **bilinear transform** (also known as Tustin's method). An alternative approach is to use the **Zero-Order-Hold** (**ZOH**) discretization. A detailed discussion of these approaches lies beyond the scope of this book, and we refer the interested reader to standard textbooks on control theory for further details (Åström and Wittenmark, 2013).

The recurrent form of Eq. (162) makes it easy to compute the states and outputs over a sequence of discrete time steps. We can unroll $\mathbf{z}_t$ and $\mathbf{o}_t$ in a feedforward fashion

$$\begin{array}{l|l} \mathbf{z}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} & \mathbf{o}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_0 \cdot \overline{\mathbf{D}} \\ \mathbf{z}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} & \mathbf{o}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{D}} \\ \mathbf{z}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_2 \cdot \overline{\mathbf{B}} & \mathbf{o}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \\ & \quad \mathbf{s}_2 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_2 \cdot \overline{\mathbf{D}} \\ \ldots\ldots & \ldots\ldots \end{array}$$

It is easy to write

$$\mathbf{z}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i}, \tag{170}$$

$$\mathbf{o}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{171}$$

Clearly, the right-hand side of Eq. (171) can be interpreted as the sum of a convolutional term and a linear term. Given that

$$\sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} = \begin{bmatrix} \mathbf{s}_0 & \mathbf{s}_1 & \cdots & \mathbf{s}_t \end{bmatrix} \cdot \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}^\top, \tag{172}$$

we define a convolutional filter with parameters

$$\mathbf{W}_{\text{ssm}} = \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}} \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}, \tag{173}$$

There are many numerical methods to approximate the solution to an ODE. For example, the Euler method for solving ODEs can be expressed as (see Section 4.3)

$$\mathbf{z}_t = \mathbf{z}_{t-1} + \Delta t \cdot g(\mathbf{z}_{t-1}, t). \tag{166}$$

Substituting Eq. (165) into Eq. (166) yields

$$\begin{aligned} \mathbf{z}_t &= \mathbf{z}_{t-1} + \Delta t(\mathbf{z}_{t-1} \cdot \mathbf{A} + \mathbf{s}_t \cdot \mathbf{B}) \\ &= \mathbf{z}_{t-1} \cdot (\mathbf{I} + \Delta t \cdot \mathbf{A}) + \mathbf{s}_t \cdot (\Delta t \cdot \mathbf{B}). \end{aligned} \tag{167}$$

This gives one of the simplest forms of the discretized state equations (Gu et al., 2022b), that is,

$$\overline{\mathbf{A}} = \mathbf{I} + \Delta t \cdot \mathbf{A}, \tag{168}$$

$$\overline{\mathbf{B}} = \Delta t \cdot \mathbf{B} \tag{169}$$

where $n_{\max}$ is the maximum length of the sequence[21]. Then, the output of the state-space model for a sequence $\mathbf{S} = \begin{bmatrix} \mathbf{s}_0 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$ can be expressed as

$$\mathbf{O} = \text{Conv}(\mathbf{S}, \mathbf{W}_{\text{ssm}}) + \text{Linear}(\mathbf{S}, \overline{\mathbf{D}}), \tag{174}$$

where $\text{Conv}(\cdot)$ is the convolution operation, and $\text{Linear}(\cdot)$ is the linear transformation operation. Such a treatment of the state-space model enables the system to be efficiently implemented using fast parallel convolution algorithms.

However, naive implementations of this model often perform poorly. As with many deep neural architectures, careful parameter initialization is crucial. Specifically, restricting the state matrix $\mathbf{A}$ to particular structures (e.g., HiPPO matrices) has proven essential for memorizing and generalizing over long sequences (Gu et al., 2022a).

Another computational bottleneck is the repeated matrix multiplication required to compute $\overline{\mathbf{A}}^n$. For large n, this operation is computationally expensive and numerically unstable. A common approach in modern SSMs is **diagonalization**. The idea is to apply a basis transformation to the state space such that $\mathbf{A}$ (or $\overline{\mathbf{A}}$) becomes diagonal. Given an SSM parameterized by $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$, we can define an equivalent model $(\mathbf{UAU}^{-1}, \mathbf{BU}^{-1}, \mathbf{UC}, \mathbf{D})$ via an invertible transformation matrix $\mathbf{U}$. It can be shown that both representations are mathematically equivalent under a change of basis[22].

By applying this transformation, and assuming $\overline{\mathbf{A}}$ can be diagonalized into the canonical form $\mathbf{P}^{-1}\Lambda\mathbf{P}$ (where Λ is diagonal), we enforce the constraint that the state matrix is diagonal, giving rise to **diagonal state-space models**. To illustrate, consider the filter used in the convolutional representation (Eq. (172)). Substituting $\overline{\mathbf{A}} = \mathbf{P}^{-1}\Lambda\mathbf{P}$, we can rewrite the term $\overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}}$ as:

$$\begin{aligned} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}} &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P})^t \cdot \overline{\mathbf{C}} \\ &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdots (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot \overline{\mathbf{C}} \\ &= (\overline{\mathbf{B}} \cdot \mathbf{P}^{-1}) \cdot \Lambda^t \cdot (\mathbf{P} \cdot \overline{\mathbf{C}}). \end{aligned} \tag{175}$$

Since Λ is a diagonal matrix, we can efficiently compute Λ^t by simply raising all the entries of Λ to the t-th power. We then have a computationally cheaper model, in which

$$\overline{\mathbf{A}}' = \Lambda, \tag{176}$$
$$\overline{\mathbf{B}}' = \overline{\mathbf{B}} \cdot \mathbf{P}^{-1}, \tag{177}$$
$$\overline{\mathbf{C}}' = \mathbf{P} \cdot \overline{\mathbf{C}}, \tag{178}$$
$$\overline{\mathbf{D}}' = \overline{\mathbf{D}}. \tag{179}$$

More detailed discussions of diagonal state-space models in sequence modeling can be found in Gu et al. (2021)'s work.

The application of state-space models to Transformers is simple. Each self-attention sub-layer is replaced in this case by an SSM sub-layer as described in Eqs. (162) and (163).

21. Here $\mathbf{W}_{\text{ssm}}$ can be represented as an $n_{\max} \times d \times d$ tensor.
22. A state-space transformation maps the state variables into a new coordinate system: $\mathbf{z}'(t) = \mathbf{z}(t) \cdot \mathbf{U}^{-1}$.

As we have seen there is a close relationship between state-space models and both CNNs and RNNs. For sequence modeling, we can deal with a sequence of tokens either sequentially as in RNNs, or in parallel as in CNNs. This leads to a new paradigm that takes both the sequential view and the parallel view of the sequence modeling problem — for training, the system operates like CNNs to make use of fast parallel training algorithms; for prediction, the problem is re-cast as a sequential update problem which can be efficiently solved by using RNN-like models.

While the formalism of state-space models is different from that we discussed in this chapter, it provides a unifying framework for sequence modeling. By viewing the problem through complementary perspectives (parallel and sequential), we can optimize for different hardware and latency constraints. Several recent architectures are motivated by this duality, yielding systems that effectively combine the parallelizability of Transformers with the inference efficiency of RNNs (Orvieto et al., 2023; Sun et al., 2023).

5.6 Conditional Computation

So far in our discussion of efficient Transformer models, we have assumed that the model architecture is determined before training and remains fixed throughout. We now turn to the case of learning efficient model architectures. We can write a model in the form

$$\mathbf{y} \quad = \quad \text{Model}(\mathbf{x}, g(\mathbf{x})), \tag{180}$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input and output of the model. $g(\mathbf{x})$ is a function that returns the model architecture and corresponding parameters for the given input $\mathbf{x}$. In general, we adopt the convention prevalent in learning problems of using a fixed model architecture and learning only the parameters, say, $g(\mathbf{x}) = \theta$. In this case, the goal is to learn optimal parameter values given the architecture and training data. On test data, we make predictions using the same model architecture along with the optimized parameters.

A natural extension of this approach is to consider the learning of both the model architecture and parameters. In architecture learning, we would like to find a function $\hat{g}(\mathbf{x})$ that produces the optimal model architecture and parameter values given the input $\mathbf{x}$. However, searching the entire hypothesis space of all possible combinations of architectures and parameters is intractable, and so practical methods are required. Two classes of methods can be applied:

- **Neural Architecture Search** (**NAS**). In **automated machine learning** (**AutoML**), neural architecture search is the process of exploring a space of neural networks to find one that best fits some criteria (Elsken et al., 2019; Zoph and Le, 2016). Once the optimal neural network is determined, its parameters are trained as usual, and then applied to new data. In order to make search tractable, several additional techniques, such as search space pruning and fast search algorithms, are typically used. Applying neural architecture search to the development of efficient neural networks is straightforward (Howard et al., 2019; Tan and Le, 2019). We need only incorporate efficiency measures into the performance estimation of neural networks, for example, the search can be guided by a criterion that penalizes neural networks with high latency or excessive memory requirements.

- **Dynamic Neural Networks**. The key idea of dynamic neural networks is to adapt a neural network to various inputs dynamically (Gupta et al., 2004; Han et al., 2021). Ideally, we would like to learn $\hat{g}(\cdot)$, and then, for any input $\mathbf{x}_{\text{new}}$, we apply the model $\text{Model}(\mathbf{x}_{\text{new}}, \hat{g}(\mathbf{x}_{\text{new}}))$. As a result, at test time we may have different model structures and/or different parameters for different inputs. However, it is infeasible to develop a function $\hat{g}(\cdot)$ that can model arbitrary neural networks. In practice, $\hat{g}(\cdot)$ is often considered to represent a family of sub-networks of a super-network. The problem is therefore reframed as a simpler problem to learn to choose which sub-network is used for a given input.

From a machine learning perspective, the approaches to neural architecture search are general and can be applied to any neural network. On the other hand, from a practical perspective, it is still difficult to find an efficient neural network that is sufficiently powerful and generalizes well. While neural architecture search provides interesting ideas for developing efficient Transformer models, we do not discuss it in depth here. Instead, the reader can refer to the above papers to have a general idea of it, and refer to So et al. (2019), Wang et al. (2020a), and Hu et al. (2021)'s work for its application to Transformers.

In this subsection, we focus on a particular family of approaches to dynamic neural networks, called **conditional computation**. This concept was originally motivated by the dynamic selection of neurons of a neural network (Bengio et al., 2015, 2013). More recently, it has often been used to refer to the process of dynamically selecting parts of a neural network. A narrow view of conditional computation is to see $g(\cdot)$ as an adaptive neural network which dynamically reduces or grows the number of computation units (such as neurons and layers). As a result, computation can adapt to changing conditions, and we can seek a good accuracy-latency trade-off by this adaptation mechanism.

A common way to achieve this is to learn how to skip some computation steps so that we can work with a subset of the network (Xu and Mcauley, 2023). One of the simplest methods, sometimes called **early exiting**, is to stop the computation at some point during encoding or decoding a sequence. This technique is often used in practical sequence generation applications where a low latency is required. Suppose $y_1 \cdots y_{n_{\max}}$ is the longest sequence that the system can generate, and $\mathbf{s}_1 \cdots \mathbf{s}_{n_{\max}}$ is the corresponding sequence of the hidden states of the top Transformer layer. Then we develop a model $f_{\text{stop}}(\cdot)$ that takes one hidden state $\mathbf{s}_i$ at a time and produces a distribution of a binary variable $c \in \{\text{stop}, \text{nonstop}\}$:

$$\Pr(c|\mathbf{s}_i) \quad = \quad f_{\text{stop}}(\mathbf{s}_i). \tag{181}$$

The generation process terminates if $\Pr(\text{stop}|\mathbf{s}_i)$ is sufficiently large, for example

$$\Pr(\text{stop}|\mathbf{s}_i) \quad \geq \quad \Pr(\text{nonstop}|\mathbf{s}_i) + \theta_{\text{stop}}, \tag{182}$$

where θ_{stop} denotes the minimal margin for distinguishing the two actions[23]. This formulation is also related to the stopping criterion problem that is frequently discussed in search algorithms for sequence generation. $f_{\text{stop}}(\cdot)$ can be designed in several different ways. For

23. An equivalent form of Eq. (182) is $\Pr(\text{stop}|\mathbf{s}_i) \geq \frac{1+\theta_{\text{stop}}}{2}$.

example, in many practical applications, the stopping criterion is based on simple heuristics. Alternatively, we can define the function $f_{\text{stop}}(\cdot)$ as a neural network and train it using labeled data.

The above approach can be easily extended to handle situations in which some of the tokens are skipped. This learning-to-skip approach is typically used in the encoding stage in which all input tokens are given in advance. Let $\mathbf{h}_1 \cdots \mathbf{h}_m$ be low-level representations of a sequence $x_1 \cdots x_m$. Like Eq. (181), we can develop a model $\Pr(c|\mathbf{s}_i)$ ($c \in \{\text{skip}, \text{nonskip}\}$) to determine whether the token x_i can be skipped. Figure 16 (a) and (b) show illustrations of early exiting and skipping. Note that the learning-to-skip method overlaps with other lines of research on training neural networks. For example, erasing some of the input tokens in training is found to be useful for achieving higher generalization of Transformer models (Kim and Cho, 2021; Shen et al., 2020). This method is also related to the downsampling methods which will be discussed in Section 5.8.

A second approach to conditional computation is to resort to **sparse expert models**, or its popular instance — MoE (Yuksel et al., 2012). In deep learning, a model of this kind is typically built from a number of experts which are neural networks having the same structure but with different parameters. In this way, we can construct a large model by simply increasing the number of experts. When running this model, during either training or prediction, we activate only a small number of the experts by some routing algorithms (see Figure 16 (c)). An MoE model is an adaptive network since each time we have a new input, the model routes it to different experts. In Section 4.4, we presented the basic form of MoE, and showed how Transformer models can be scaled up by this sparse method. For a comprehensive review of the recent advances in MoE, we refer the interested reader to Fedus et al. (2022a)'s work.

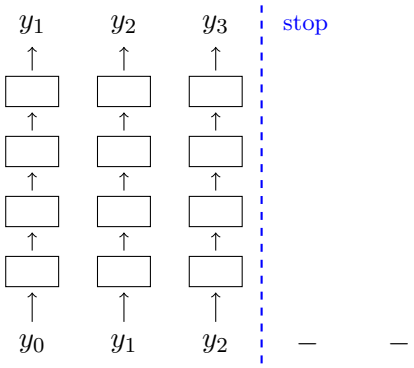
A third approach that can be used to adapt a Transformer model to changing input is to dynamically shrink the number of layers. Several methods have been proposed to do this in an attempt to improve inference efficiency. The simplest of these is to exit at some hidden layers by which we can still make accurate predictions for the input (see Figure 16 (d) and (e)). To do this, we can either determine the appropriate depth for the entire sequence (call it a **sentence-level depth-adaptive model**), or use an adaptive depth for each token (call it a **token-level depth-adaptive model**). Here we consider token-level depth-adaptive models but the methods can be easily extended to sequence-level depth-adaptive models.

Suppose there are L stacked layers at position i[24]. We would ideally like to find a layer in the stack, which can be used as the last hidden layer for making predictions, and whose depth is as low as possible. However, we cannot simply use the L-th layer of the stack as the oracle for this problem, because we never know in advance what the last layer generates during inference. Instead, we need to determine whether the network should stop growing at depth i, considering the layers generated so far.

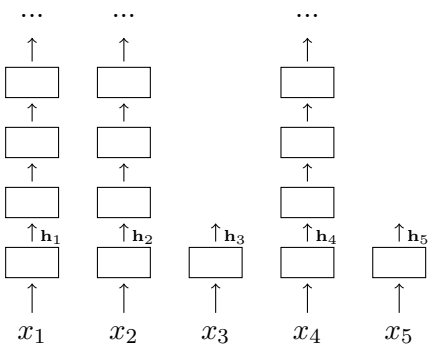
Now suppose we have a Transformer decoder which produces a distribution over a vocabulary V at each step. As usual, we denote the output of the l-th layer at step i by $\mathbf{s}_i^l$. For each $\mathbf{s}_i^l$, we create an output layer that produces a distribution $\mathbf{p}_i^l$ over the vocabulary (call it an **early exit classifier**), given by

$$\mathbf{p}_i^l \quad = \quad \text{Softmax}(\mathbf{s}_i^l \cdot \mathbf{W}_{\text{o}}^l), \tag{183}$$

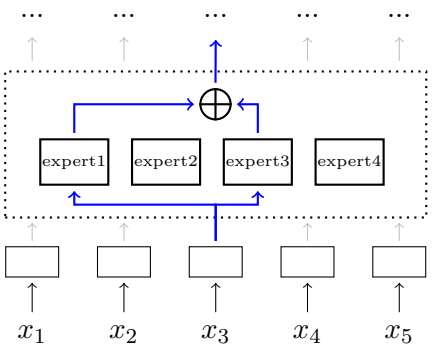
24. A layer is a standard Transformer block consisting of a few sub-layers.



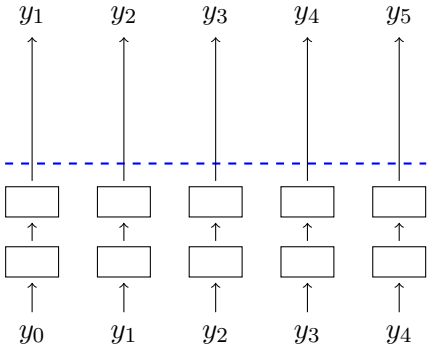
(a) Early Stopping (Decoder)



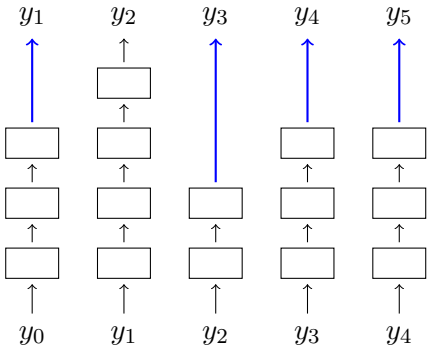
(b) Token Skipping (Encoder)



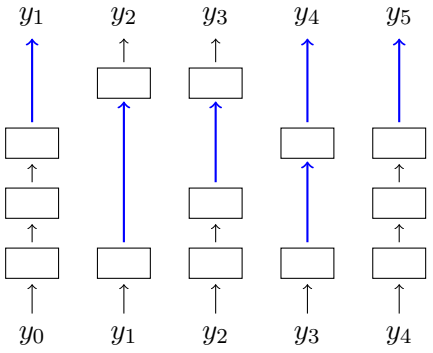
(c) MoE (Encoder)



(d) Sentence-level Depth Adaptation (Decoder)



(e) Token-level Depth Adaptation (Decoder)



(f) Layer Skipping (Decoder)

Figure 16: Methods of conditional computation, including early exiting, token skipping, MoE, sentence-level depth adaptation, token-level depth adaptation, and layer skipping. While these methods are illustrated using either the encoding or decoding process, most of them can be applied to both Transformer encoders and decoders.

where $\mathbf{W}_{\mathrm{o}}^{l} \in \mathbb{R}^{d \times |V|}$ is the parameter matrix. Hence we have $L-1$ additional output layers, each corresponding to a hidden layer from depth 1 to $L-1$. At training time, we consider the cross-entropy losses of $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^{L-1}\}$, and train these layers together with the Transformer model. At test time, the depth of the network grows as usual, and we use $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^l\}$ and/or $\{\mathbf{s}_i^1, \cdots, \mathbf{s}_i^l\}$ to determine whether we should exit at the l-th layer. There are several exit criteria, for example,

- Common criteria are based on measures of the confidence of predictions. A simple method is to compute the entropy of $\mathbf{p}_i^l$, and exit if this entropy is above a pre-defined value.
- Alternatively, one can view the maximum probability of the entries of $\mathbf{p}_i^l$ as the confidence of the prediction.
- Instead of considering the output of a single layer, we can also examine the change in the outputs or hidden states over a number of layers. For example, we can measure the similarity between $\mathbf{p}_i^{l-1}$ and $\mathbf{p}_i^l$ or between $\mathbf{s}_i^{l-1}$ and $\mathbf{s}_i^l$. If the similarity is above a given threshold, then we say that the output of the neural network tends to converge and the number of layers can stop growing.
- The above methods can be extended to examine the change in the predictions made by the classifiers associated with the layers. For example, the model can choose to exit if the predictions made by the classifiers remain unchanged for a number of layers.

Discussions of these criteria can be found in the related papers (Schuster et al., 2022; Xin et al., 2020; Zhou et al., 2020). There are a variety of ways to improve these early exit methods. One is to explore other forms of the prediction for each layer. For example, we can develop a model that directly predicts how many layers we need to model the input (Elbayad et al., 2020). Another line of research on early exit focuses on better training for these models, for example, we can consider various loss functions for training the classifiers (Schuster et al., 2022; Schwartz et al., 2020). In addition, there is also interest in learning the combination of the outputs of multiple layers so that we can make predictions by using multiple levels of representation (Liao et al., 2021; Zhou et al., 2020).

A problem with token-level adaptive-depth models is that the representations at certain depths may be absent in the previous steps. In this case, standard self-attention is not directly applicable, because we may not attend to the previous tokens in the same level of representation. For training, this can be addressed by using all the L layers of the full model. For inference, we can either duplicate the layer from which we exit to fill up the layer stack, or modify the self-attention model to enable it to attend to the representations of the previous tokens at different depths.

It is also possible to select any subset of the layers for constructing a shallow network. The adaptive models therefore can be generalized to skipping models (see Figure 16 (f)). As with the early exit problem, the skipping problem can be framed as a learning task, in which a classifier is trained to decide whether a layer should be dropped. The learning-to-skip problem has been studied in the field of computer vision (Wang et al., 2018b; Wu et al., 2018b). However, learning a skipping model for large-scale, deep neural networks is difficult. For practical systems, it still seems reasonable to use heuristics or cheap models to obtain a

neural network having skipped layers, which has been discussed in recent pre-trained NLP models (Del Corro et al., 2023; Wang et al., 2022c).

5.7 Model Transfer and Pruning

Many large Transformer models have been successfully developed to address NLP problems. A common question is: can we transform a large, well-trained model into a smaller one that allows for more efficient inference? At a high level, this can be thought of as a **transfer learning** problem in which the knowledge is transferred from one model to another. However, we will not discuss this general topic, which spans a broad range of issues and models, many of which fall outside the scope of this chapter. Instead, we narrow our discussion to two approaches that are widely used for training smaller neural networks from larger ones.

5.7.1 Knowledge Distillation

Knowledge distillation is a process of compressing the knowledge in a large neural network (or an ensemble of neural networks) into a smaller neural network (Hinton et al., 2015). In the supervised learning of neural networks, objective functions are generally designed to quantify the error between the predicted and true answers. Hence, by minimizing this loss, models are trained to output the correct answers. While models are typically optimized on the training data in this manner, what we really want is to generalize to unseen data. This is, however, difficult because training only with ground-truth labels provides no direct supervision signal regarding generalization. In knowledge distillation, instead of forcing a model to match the ground-truth output, we train it to mimic the behavior (or output distributions) of a larger model. To achieve this, we directly transfer the knowledge of a pre-trained model to the model we wish to train.

A frequently used approach to knowledge distillation is **teacher-student training**. A teacher model is typically a relatively large neural network that has already been trained and can generalize well. A student model is a smaller neural network, such as one with fewer layers, to which we transfer the knowledge. A simple way to distill the knowledge from the teacher model into the student model is to use the output of the teacher model as the "correct" answer for training the student model. Suppose we have a teacher Transformer model that can generate a sequence of distributions $\{\Pr(\cdot|y_0, \mathbf{x}), \cdots, \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x})\}$ for the input $\mathbf{x}$. To keep the notation simple, we denote the distribution $\Pr(\cdot|y_0 \cdots y_{i-1}, \mathbf{x})$ as $\widetilde{\mathbf{p}}_i$. Similarly, we denote the output of the student Transformer model for the same input as $\mathbf{p}_i$. As usual, we consider a loss function $\text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i)$ (such as the cross-entropy function) for computing the distance between $\widetilde{\mathbf{p}}_i$ and $\mathbf{p}_i$. Then, we can define the loss over the entire sequence as

$$\mathcal{L}(\mathbf{x}, \theta) \quad = \quad \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i), \tag{184}$$

where θ denotes the parameters of the student model[25]. Using this loss, we can optimize θ for any given set of source sequences $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$ by minimizing the objective $\sum_{k=1}^{K} \mathcal{L}(\mathbf{x}_k, \theta)$.

25. We omit the parameters of the teacher model because they are fixed throughout the training process.

Several extensions to this basic method have been developed to model the problem of knowledge transfer between two models. A simple way is to use the hidden states instead of the output probabilities as the training targets (Romero et al., 2014). In this case, the objective is to minimize the difference between some hidden states of the teacher model and the corresponding states of the student model. Rather than using the outputs of various layers as the targets for training the student model, another technique is to model the relations between samples and train the student model by minimizing the difference between the relation encodings of the teacher and student models (Park et al., 2019; Peng et al., 2019). For example, we can develop a relation encoding model based on the Transformer architecture. The goal is then to optimize the student model so that its corresponding relation encoding of a group of samples is as close as possible to that of the teacher model.

For sequence generation problems, a special case of knowledge distillation, which can be viewed as a means of **data augmentation**, is often used for developing lightweight models (Kim and Rush, 2016). For example, consider the problem of transferring the translation ability of a well-developed machine translation model (i.e., the teacher model) to a new model (i.e., the student model). Given a set of source-side sentences $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$, we can use the teacher model to translate each $\mathbf{x}_k$ to a target-side sentence $\widetilde{\mathbf{y}}_k$. Then, by treating $\mathbf{x}_k$ and $\widetilde{\mathbf{y}}_k$ as paired sentences, we obtain a bilingual dataset consisting of $\{(\mathbf{x}_1, \widetilde{\mathbf{y}}_1), \cdots, (\mathbf{x}_K, \widetilde{\mathbf{y}}_K)\}$. We can use this bilingual dataset as the labeled dataset to train the student model as usual. One advantage of this data augmentation method is that it is architecture-agnostic. It does not require access to the internal architectures of either the teacher or the student models. Hence, it can be applied even if the teacher model is used as a black-box model. More detailed discussions on knowledge distillation can be found in the surveys by Gou et al. (2021) and Wang and Yoon (2021).

5.7.2 Structured Pruning

Pruning is among the most popular model compression methods and has been applied to a broad range of systems. One common approach is **unstructured pruning**, in which we retain only a sparse subset of connections between neurons. However, as with most sparse models, networks pruned in this way typically require specialized implementations and hardware support, which in turn limits their efficiency gains in certain applications. A simpler yet more aggressive alternative is **structured pruning**. In deep learning, structured pruning is a technique that removes entire groups of neurons or connections simultaneously. For example, we can remove an entire layer of neurons from a neural network to obtain a shallower model. Given their multi-layer and multi-head architecture, Transformers are naturally suited to structured pruning, and we can prune a Transformer network in several different ways. For instance, we can prune specific heads in the multi-head attention mechanism (Michel et al., 2019; Voita et al., 2019), or entire layers within the stack (Hou et al., 2020; Kim and Awadalla, 2020).

Formally, we can represent a neural network as a set of parameter groups $\{\theta_1, \cdots, \theta_R\}$, each corresponding to a specific component or sub-module of the network. Our goal is to find a subset of $\{\theta_1, \cdots, \theta_R\}$ that allows us to build a model yielding strong performance while maintaining significantly lower computational complexity. However, an exhaustive

search for such an optimal subset is infeasible because there is a combinatorial number of possible candidates, and evaluating all of them is computationally prohibitive.

One approach to structured pruning is to randomly discard components of a model. We can repeat this random pruning process multiple times to generate a pool of model candidates, and select the best-performing one. Another approach relies on heuristics to determine which components are redundant and can be safely removed. Common measures for evaluating the importance of a parameter group θ_r include various metrics based on the norms of its weights or gradients (Santacroce et al., 2023). We can then prune θ_r if these metrics fall below or exceed predefined thresholds. A third approach frames the pruning problem as a continuous optimization task by introducing trainable gates that indicate the presence or absence of different components (Lagunas et al., 2021; McCarley et al., 2019; Wang et al., 2020c). The final pruned model can be directly derived using these trained gates. It is worth noting that, in many cases, pruning is no longer merely a post-processing step for a fully trained model, but rather an integral part of the training process itself.

5.8 Sequence Compression

In sequence modeling and generation problems, time and space complexities are strongly influenced by the length of the input or output sequences. Therefore, shorter sequences are generally preferred. This is particularly important for Transformer models, as their time and space complexities scale quadratically with sequence length, making memory footprint and latency significant computational bottlenecks for very long sequences. While previous subsections discussed modifications to the Transformer architecture to handle long sequences, in this section, we instead explore methods for compressing sequences to more manageable lengths.

One simple approach is to map the input sequence to a fixed-size representation. For example, using the recurrent models discussed in Section 5.2, we can encode a sequence of vectors into a single vector. This method can be easily extended to generate a larger representation so that this representation can retain more information from the original input. For example, we can select a fixed number of the hidden states over the sequence to form a new sequence of fixed length. Another way to represent a variable-length sequence as a fixed-length sequence is to perform cross-attention between the input vectors and a set of learnable hidden representations. In the work of Jaegle et al. (2021), this is done by introducing r hidden representations $\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$, and then applying cross-attention between the input vectors $\mathbf{x}_1, \cdots, \mathbf{x}_m$ and these hidden representations. The attention model can be a standard QKV attention model in which we view $\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$ as queries and $\{\mathbf{x}_1, \cdots, \mathbf{x}_m\}$ as keys and values. The output of this model is a sequence of r vectors, which can be used as fixed-length input to downstream systems.

A second approach is to use downsampling to compress the sequence into a shorter one. A typical method of downsampling is strided convolution, which has been widely used in computer vision and speech processing. For example, suppose there is a sequence of m vectors in $\mathbb{R}^d$. We can define a filter with a width of 2 and a stride of 2. By taking the sequence as input, the filter produces a sequence of $\frac{m}{2}$ new vectors in $\mathbb{R}^d$, and so we have a reduction factor of 2. Also, we can stack multiple convolutional layers or pooling layers to achieve a desired level of length reduction, called **progressive downsampling**. However,

it seems inevitable that downsampling will lead to information loss (Burchi and Vielzeuf, 2021; Han et al., 2020). We need to consider a trade-off between the compressed sequence length and the performance of downstream systems (Xu et al., 2023b).

In NLP, the problem of sequence compression is also closely related to the problem of tokenizing input strings. Therefore, tokenization is a practical approach that can be taken to address the length issue. Segmenting a string into small tokens (such as characters) generally reduces the sparsity of the data, which makes it easier to learn the embeddings of these tokens, but such approaches often lead to a long sequence. By contrast, we will have a shorter sequence if we segment the input string into larger units, but this may suffer from data sparsity. In deterministic tokenization methods, which produce tokenization results using statistics collected from the entire dataset, the sequence length can be controlled to some extent by adjusting specific hyperparameters, for example, in byte pair encoding (Sennrich et al., 2016), increasing the size of the vocabulary generally reduces the number of resulting tokens. Another way to obtain an appropriate sequence of tokens is to use a model for choosing among tokenization candidates (Kudo, 2018; Provilkov et al., 2020). As with many probabilistic models for text generation, in this case, we can incorporate priors to the criterion for tokenization selection so that we can express a preference for shorter sequences over longer sequences.

A fourth approach to sequence compression is to drop some of the tokens in the sequence. For instance, in many practical applications, sequences are simply truncated when their lengths exceed a predefined threshold. We can relate this to the early exiting and skipping approaches in conditional computation. Thus the methods discussed in Section 5.6 can be directly applied. The token dropping methods can also be viewed as pruning methods, called **token pruning**. By discarding tokens that are less important for representing the entire sequence, token pruning can significantly reduce the sequence length while maintaining the performance of NLP systems on downstream tasks (Kim et al., 2023).

5.9 High Performance Computing Methods

So far in this section, we have discussed efficient Transformer models from the perspective of deep learning. However, we have not considered their hardware efficiency. As modern hardware provides a variety of execution paradigms, the practical time and memory footprint savings generally depend on the specifications of the hardware systems. One line of research on the efficient use of computing resources explores parallel computing methods. There have been many attempts to develop large-scale Transformer models by using a cluster of machines. Typically, scaling Transformers to models with billions or even tens of billions of parameters requires a careful design of parallelism strategies for sharding the big networks. More efficient implementations of such systems also require consideration of communication overhead in the cluster, as well as the utilization of sparse models that activate only a small subset of the parameters for each sample, thereby enabling the use of very large models. Most of these methods have been extensively studied in the literature (Barham et al., 2022; Fedus et al., 2022b; Lepikhin et al., 2021). The results of these studies are foundational to many follow-on works investigating the **scaling laws** for large language models (Brown et al., 2020; Chowdhery et al., 2022). Since large-scale distributed

models are generic and not specialized to the case of Transformers, we skip their discussion here. Interested readers can refer to the above papers for more detailed discussions.

In this subsection, we consider hardware-aware methods to seek greater computational efficiency for Transformer models. We first consider a simple but widely used method that aims to represent and execute neural networks using lower- or mixed-precision formats (Gholami et al., 2022). Conventional neural networks are typically based on single-precision and/or double-precision floating-point representations of data. While single-precision floating-point data types provide a sufficiently precise way to represent parameters and intermediate states in most cases, in some applications, they are not essential. As an alternative, one can use half-precision (or even lower-precision) formats for representing floating-point numbers in neural networks. The size of the resulting model is thus half the size of the original model. One advantage of using half-precision floating-point representations is that, although processing such data types requires new APIs for linear algebra operations and hardware support, it does not change the model architecture, requiring only slight modifications to the system. For example, half-precision floating-point representations can be applied to either the training or inference of Transformers, or both.

Recently, the deployment of large Transformer models has been further improved by quantizing these models. In signal processing, quantization is a process of mapping continuous values (i.e., floating-point representations) to a set of discrete values (i.e., fixed-point representations). This process is generally implemented using a system called a quantizer. In the context of neural networks, a quantizer involves two functions — the quantization function and the de-quantization function. The quantization function maps a floating-point number to a (lower-bit) integer. A simple quantization function is given by

$$Q(x) \quad = \quad \lfloor \frac{x}{s} \rceil, \tag{185}$$

where $\lfloor \cdot \rceil$ is a rounding function[26], x is the real-valued input, and s is the quantization step size that controls the level of quantization. The quantization function is coupled with a de-quantization function

$$D(r) \quad = \quad s \cdot r. \tag{186}$$

With this notation, the quantizer can be expressed as $D(Q(x)) = s \cdot \lfloor \frac{x}{s} \rceil$. The difference between $D(Q(x))$ and x is called the quantization error. A smaller value of s typically reduces the quantization error. In practice, however, we wish to choose an appropriate value of s in order to spread the possible values of $Q(x)$ evenly across the integer representation range. For example, $s = \frac{\max\{D(r)\}}{2^p - 1}$, where p is the number of bits used to represent an integer and $\max\{D(r)\}$ is the maximum representable value after de-quantization. The equations above show one of the simplest cases of quantization. More general discussions of quantization can be found in books on digital signal processing and related surveys (Gray, 1998; Oppenheim and Schafer, 1975; Rabiner and Gold, 1975).

Applying quantization to Transformers is relatively straightforward. The idea is that we quantize the inputs and model parameters using $Q(x)$ and feed them to a quantized Transformer model in which all layers operate on integer-valued tensors. In other words,

26. $\lfloor a \rceil$ returns the integer closest to a.

we implement the model using integer-only arithmetic. However, the price to be paid for this compressed model, as with many approximation approaches to deep learning, is that its predictions are less accurate than those of the standard Transformer. Using integer operations to approximate continuous-valued operations generally leads to approximation errors. These errors will accumulate if the quantized neural network is deep. Furthermore, Transformer models involve components (such as self-attention sub-layers) that require relatively complex linear algebra operations. Simply applying quantization to these modules will lead to a significant loss in accuracy. One solution is to simplify the model architecture and develop new sub-models that are more suited for quantization. Alternatively, a more common paradigm in quantized neural networks is to add de-quantization functions to the neural networks so that the outputs of a layer are floating-point tensors and can be used as usual in the following steps. Consider a simple example where we multiply a real-valued input matrix $\mathbf{a}$ with a real-valued parameter matrix $\mathbf{A}$. We first quantize $\mathbf{a}$ and $\mathbf{A}$, and multiply them using integer-based matrix multiplication. The result is then de-quantized to a real-valued matrix. In this way, we obtain an approximation $D(Q(\mathbf{a}) \cdot Q(\mathbf{A}))$ to $\mathbf{a} \cdot \mathbf{A}$ at a very low computational cost.

However, sandwiching each layer between $Q(\cdot)$ and $D(\cdot)$ will lead to the additional cost of running $Q(\cdot)$ and $D(\cdot)$. In some practical applications, the computational overhead introduced by $Q(\cdot)$ and $D(\cdot)$ is even larger than the time savings of performing integer-based operations. In general, the benefits of quantizing neural networks are greater than the costs only if the neural networks are sufficiently large. Therefore, in practice, it is common to perform quantized computation only for operations whose computational costs are high. For example, in recent large language models, quantization is primarily applied to the multiplication of large matrices, yielding significant time and memory savings.

While quantization approaches can be used in both training and inference, a widely used approach is to quantize Transformer models after training (called **post-training quantization**). In this approach, quantization is performed on well-trained floating-point-based neural networks, resulting in fewer quantization-related errors. However, these errors cannot be easily compensated for because they are introduced after training is complete. A more promising approach is to incorporate quantization during training so that the model can learn to compensate for quantization-related errors (Jacob et al., 2018; Nagel et al., 2021). There have been several attempts to apply quantization-aware training to Transformers (Bondarenko et al., 2021; Stock et al., 2021; Yang et al., 2023b). In addition to computational efficiency, another important consideration for high-performance systems is the constraints of the memory hierarchy. In general, better system design requires considering the speeds and capacities of different memory levels. The problem is even more complicated when training large Transformer models on modern hardware where both GPUs and CPUs are utilized. A general principle of system design is that memory transfers between different memory levels should be minimized. While we would ideally like to have a large, high-level memory on which we can store all the data we need to process, in many practical situations, the size of fast, on-chip memory is orders of magnitude smaller than the total size of the data. In this case, we can re-order the memory access in the algorithms so that data used in adjacent computation steps can be loaded into high-speed memory all at once. This idea motivates the development of many fast linear algebra libraries. For

example, there are matrix multiplication algorithms that are highly optimized for different shapes of input matrices.

It is relatively straightforward to use these optimized linear algebra algorithms to build a Transformer-based system, but the modules of this system are not jointly optimized for efficiency. For example, a self-attention sub-layer involves a series of scaling, normalization, and matrix multiplication operations. Although each of these operations is implemented in supported and efficient linear algebra libraries, successive calls to them still require multiple memory transfers when switching from one operation to another. In practice, a better approach would be to keep some of the intermediate states in the on-chip memory and reuse them in subsequent computation steps instead of fetching them again from slower memory. For example, on modern GPUs, a simple way to achieve this is to merge multiple operations into a single operation, a technique known as **kernel fusion**. For Transformer models, a general idea is to design data partitioning and layout strategies that maximize the computation on each data block loaded into high-performance memory while simultaneously minimizing memory transfers. There have been several attempts to use these strategies to improve the attention models in Transformers (Ivanov et al., 2021; Pope et al., 2023). Some of these methods, such as FlashAttention and PagedAttention, have been successfully incorporated into recent large language models (Dao et al., 2022; Kwon et al., 2023).

6. Applications

Transformers have a wide range of applications across NLP tasks. While the Transformer model introduced by Vaswani et al. (2017) is based on a standard encoder-decoder architecture, it is primarily used in three different configurations:

- **Decoder-only Models**. If the cross-attention sub-layers are removed, the decoder becomes a standard language model. Hence, this decoder-only model can be applied to text generation problems. For example, given a sequence of left-context tokens, we use the model to predict subsequent tokens.
- **Encoder-only Models**. Transformer encoders can be treated as sequence models that take a sequence of tokens as input and produce a sequence of representations, each corresponding to an input token. These representations can be viewed as a form of encoding of the input sequence, and are often taken as input to a prediction model. This encoder-predictor architecture forms the basis of many NLP systems, such as systems for sentence classification and sequence labeling. Pre-trained Transformer encoders can also be used to map texts into the same vector space so that we can compute the distance or similarity between any two texts.
- **Encoder-Decoder Models**. Encoder-decoder models are typically used to model sequence-to-sequence problems. Applications include many tasks in NLP and related fields, such as machine translation and image captioning.

Note that while most Transformer-based systems can be categorized into the above three types, the same NLP problem can generally be addressed using different types of models. For example, recent decoder-only models have demonstrated strong performance on a broad range of problems by framing them as text generation tasks, even though many of these problems were historically addressed using encoder-decoder or encoder-only models. To

illustrate how the above models are applied, this section considers a few applications where Transformers are chosen as backbone models.

6.1 Language Modeling

Language modeling is an NLP task in which we predict the next token given its preceding tokens. This is generally formulated as a problem of estimating the distribution of tokens at position $i+1$ given tokens at positions $0 \sim i$. Here we denote this distribution by $\Pr(\cdot \mid x_0, \ldots, x_i)$, where $x_0, \ldots, x_i$ are the observed tokens. The predicted token is obtained by maximizing the probability:

$$\hat{x}_{i+1} = \arg\max_{x_{i+1} \in V} \Pr(x_{i+1} | x_0, \cdots, x_i), \tag{187}$$

where V is the vocabulary. The prediction can be extended to the tokens following $\hat{x}_{i+1}$

$$\hat{x}_{k+1} = \arg\max_{x_{k+1} \in V} \Pr(x_{k+1} | x_0, \cdots, x_i, \hat{x}_{i+1}, \cdots, \hat{x}_k). \tag{188}$$

This model forms the basis of many systems for text generation: given the context tokens $x_0 \cdots x_i$, we generate the remaining tokens $\hat{x}_{i+1} \cdots \hat{x}_{k+1}$ to make the sequence complete and coherent.

As discussed in Section 2.1, Transformer decoders are essentially language models. A key difference between the problem of decoding in an encoder-decoder Transformer and the problem of language modeling is that the Transformer decoder makes predictions conditioned on the context tokens on both the encoder and decoder sides, rather than being conditioned only on preceding tokens. To modify the Transformer decoder to implement a standard language model, the cross-attention sub-layers are simply removed and a Transformer decoding block can be expressed as

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{self}}), \tag{189}$$

$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \tag{190}$$

Here, $\mathbf{S}^l$ denotes the output of the block at depth l. $\text{Layer}_{\text{self}}(\cdot)$ denotes the self-attention sub-layer, and $\text{Layer}_{\text{ffn}}(\cdot)$ denotes the FFN sub-layer. We see that this decoding block has the same form as an encoding block. The difference between the decoding and encoding blocks arises from the masking strategies adopted during training: the former masks the attention from a position i to any right-context position $k > i$, whereas the latter has no such restriction. A softmax layer is stacked on top of the last block and is used to produce the distribution over the vocabulary at each position. For inference, the Transformer decoder works in an autoregressive manner, as described in Eq. (188).

Training these models follows a standard paradigm. We learn the model by repeatedly updating the parameters, based on the gradients of the loss on the training samples. This paradigm can be extended to the training of large Transformer-based language models, which have been widely applied in generative AI. However, training Transformer models at scale, including decoder-only, encoder-only, and encoder-decoder models, may lead to new difficulties, such as training instabilities.

6.2 Text Encoding

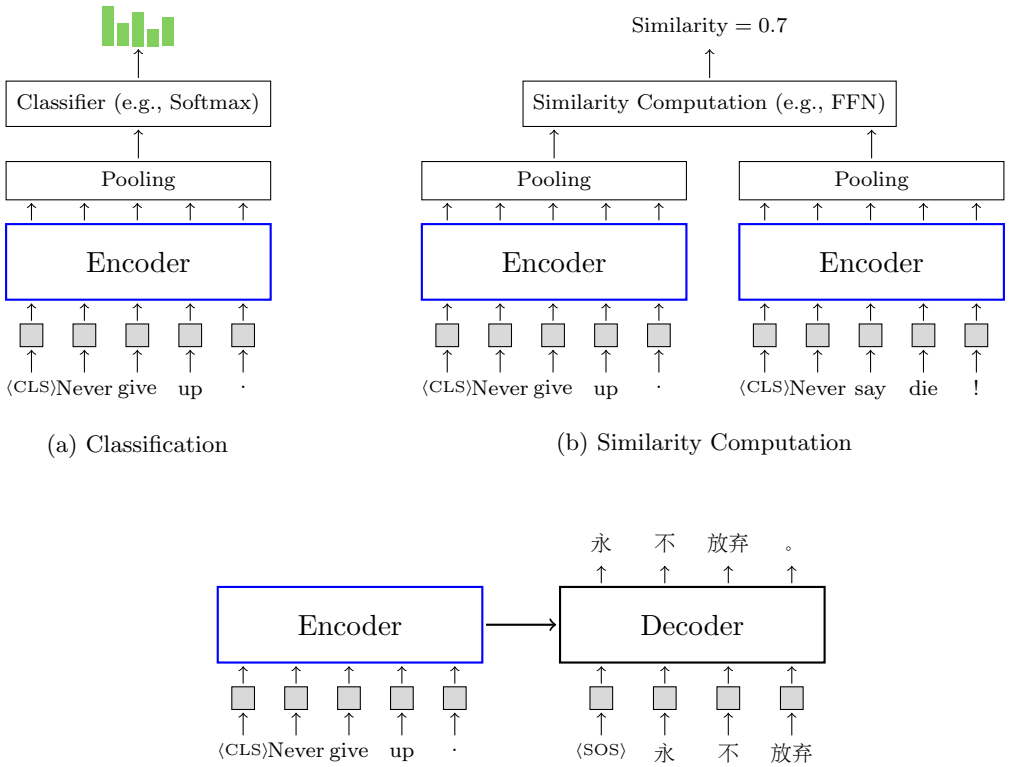
For many NLP problems, a widely used paradigm is to first learn a representation of an input sequence, and then make predictions for downstream tasks based on this representation. As a result, we separate sequence representation from specific NLP tasks. One of the advantages of this paradigm is that we can train a sequence model that is not specialized to particular problems, thereby enabling it to generalize well.

Clearly, Transformer encoders are a type of sequence model and can be used as text encoders. Consider a Transformer encoder with L encoding layers. The output of the last encoding layer can be viewed as the sequence representation. Here, we prepend a special token x_0 to each sequence, indicating the beginning of the sequence (often written as $\langle$SOS$\rangle$ or [CLS]). Given a sequence of $m+1$ input tokens $x_0, x_1, \cdots, x_m$, the output of the encoder will be a sequence of $m+1$ vectors $\mathbf{h}_0^L, \mathbf{h}_1^L, \cdots, \mathbf{h}_m^L$. Since x_0 does not correspond to an actual word and has a fixed positional embedding, it serves as a global representation token for collecting information from other positions via the self-attention mechanism. Hence, $\mathbf{h}_0^L$ acts as a representation of the entire sequence, without being biased toward any specific tokens or positions. In many cases, we require a single representation of a sequence to serve as input for downstream components of the system. For example, we can construct a sentence classification system based on a single vector generated from $\{\mathbf{h}_0^L, \ldots, \mathbf{h}_m^L\}$. In this scenario, we can simply use $\mathbf{h}_0^L$ as the sequence representation. A more general approach is to add a pooling layer to the encoder, which allows us to explore various pooling methods to generate the sequence embedding from $\{\mathbf{h}_0^L, \cdots, \mathbf{h}_m^L\}$.

In text encoding, token sequences are represented by real-valued vectors, often referred to as sentence representations or sentence embeddings, which can be interpreted as points in a multi-dimensional space (Hill et al., 2016). Consequently, another use of text encoding is to compute the semantic or syntactic similarity of token sequences based on their relative proximity in this space. A straightforward method for this is to compute the Euclidean distance or cosine similarity between sequence embeddings. The shorter the distance between two sequences, the more similar they are considered to be. There are many distance metrics available, and it is possible to combine them to obtain a more robust measure of sequence similarity. Such similarity computations are applied in areas such as textual entailment, information retrieval, and translation evaluation (Cer et al., 2018; Reimers and Gurevych, 2019). Additionally, they are frequently used to assess the inherent quality of text encoding models.

Text encoding is also a crucial component of sequence-to-sequence models. In sequence-to-sequence tasks, we can develop a separate Transformer encoder for source-side sequence modeling within an encoder-decoder system (see Figure 17). For instance, we can pre-train a Transformer encoder on a large-scale corpus of source-side texts and subsequently use it as the encoder in a downstream encoder-decoder model. It is worth noting that while the encoder is based on the Transformer architecture, the decoder is not restricted to this architecture. This flexibility enables us to incorporate pre-trained Transformer encoders into hybrid sequence-to-sequence architectures, such as systems that combine a Transformer encoder with an LSTM decoder.

In supervised learning scenarios, training a Transformer encoder is straightforward. We can treat it as a regular component of the target model and train this model on task-



(a) Classification

(b) Similarity Computation

(c) Sequence-to-Sequence Modeling

Figure 17: Integrating Transformer encoders as components of different systems. A common approach is to feed the output of the encoder (with pooling) into a classifier to obtain a sequence classification system. Another way to utilize Transformer encoders is to compute the similarity between two sequences. We use the same encoder to represent the two sequences, and then build a neural network over the two representations to produce a similarity score between them. Furthermore, Transformer encoders can also be used in encoder-decoder systems to model sequence-to-sequence problems.

specific labeled data. However, such a method requires the encoder to be optimized on each task, and the resulting encoder might not always generalize well to other tasks, especially given that labeled data is scarce in most cases. A more prevalent approach is to frame the training of text encoders as an independent task in which supervision signals are derived solely from raw text. This led researchers to develop self-supervised Transformer encoders, such as BERT, which make use of large-scale unlabeled text, and these encoders were found to generalize well across many downstream tasks. Further discussions of pre-trained Transformer encoders can be found in the literature on pre-trained models.

6.3 Speech Translation

As illustrated in Section 2, the standard encoder-decoder Transformer model was proposed to model sequence-to-sequence problems. Here we consider the problem of translating speech in one language to text in another language — a problem that is conventionally addressed using both automatic speech recognition (ASR) and machine translation techniques. Instead of cascading an automatic speech recognition system and a machine translation system, we can use Transformer models to build an end-to-end speech-to-text (S2T) translation system to directly translate the input speech to the output text.

To simplify the discussion, we assume that the input to an S2T translation system is a sequence of source-side acoustic feature vectors, denoted by $\mathbf{a}_1 \cdots \mathbf{a}_m$, and the output of the system is a sequence of target-side tokens, denoted by $y_1 \cdots y_n$.[27] Mapping $\mathbf{a}_1 \cdots \mathbf{a}_m$ to $y_1 \cdots y_n$ is a sequence-to-sequence problem. Thus it is straightforward to model the problem using an encoder-decoder Transformer model, and the training and inference of this model are standard, as in neural machine translation.

In S2T translation, however, we have to deal with sequence mappings across both modalities and languages. This poses new challenges compared to conventional machine translation problems and influences the design of S2T translation models. There have been several improvements to Transformers to adapt them better for S2T translation tasks. Some improvements focus on the design of Transformer layers (Di Gangi et al., 2019). For example, in Gulati et al. (2020)'s system, a CNN sub-layer and relative positional embeddings are integrated into each Transformer layer, enabling the model to efficiently capture both local and global features.

Another line of research on S2T translation focuses on improving the encoder-decoder architecture. This involves modifying the encoder, the decoder, or both. To illustrate, Figure 18 shows the architectures of three S2T translation models. All of them are based on Transformers but have different encoder architectures. As shown in the figure, the standard encoder-decoder architecture has one Transformer encoder for reading the source-side input $\mathbf{a}_1, \cdots, \mathbf{a}_m$ and one Transformer decoder for producing the target-side output $y_1, \cdots, y_n$. By contrast, the decoupled encoder model decomposes the encoder into two stacked encoders — one for acoustic modeling (referred to as the **speech encoder**) and one for textual modeling (referred to as the **text encoder**) (Liu et al., 2020c; Xu et al.,

27. In order to obtain the input sequence to the system, we need to discretize continuous speech into signals represented by feature vectors. This process is typically nontrivial, requiring either a feature extractor based on a variety of signal processing operations or a neural network that learns feature mappings in an end-to-end manner. But we will not dive into the details of these methods and simply treat the input feature extractor as an upstream system.

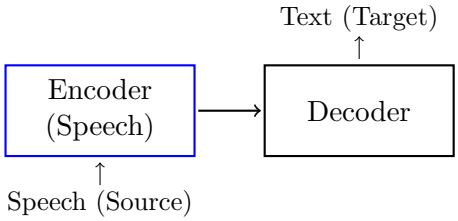
2021a). This design reflects a modeling hierarchy in which representations at different levels of the network are concerned with different aspects of the problem. For example, the speech encoder models low-level features when mapping acoustic embeddings into larger language units, and the text encoder models the semantic or syntactic features when representing the entire input sequence. An advantage of separating out the text encoder is that the encoding process follows our prior knowledge: we need to first transcribe the speech input and then translate the transcript into the target language. Therefore, we can train the speech encoder in a manner similar to how we train an ASR system. This enables us to pre-train the speech encoder and the text encoder on unlabeled data and subsequently incorporate the pre-trained encoders into S2T translation systems.

An alternative encoding architecture is the two-stream architecture, as shown in Figure 18(c). Like the decoupled encoder architecture, this architecture has a speech encoder and a text encoder, but the two encoders work in parallel rather than in sequence (Ye et al., 2021). The speech encoder takes acoustic features as input, and the text encoder takes tokens (or their embeddings) as input. A third encoder, called the **shared encoder**, integrates the outputs from both the speech and text encoders, merging the representations from the two modalities. This two-stream architecture is flexible because it provides multiple ways to train S2T translation models. A common approach is to train each branch individually. For example, if we mask the speech encoder, the model will transform into a machine translation model which can be trained using bilingual texts. Conversely, if we mask the text encoder, we can train the model as a standard S2T translation model. For inference, the text encoder can be dropped, and the speech input is modeled using the speech encoder and the shared encoder.

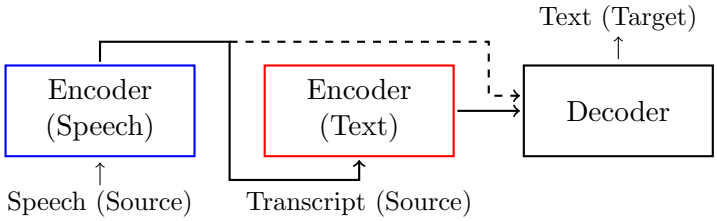
In deep learning, training is often intertwined with architecture design. Because we are dealing with data in two modalities and two languages, we can develop multiple supervision signals for the multi-task learning of S2T translation models. A widely used method is to introduce an ASR-related loss into the training of speech encoders. For example, in the decoupled encoder model, a classifier can be constructed based on the output from the speech encoder. By minimizing the connectionist temporal classification (CTC) loss for this classifier, the speech encoder can be optimized in a manner similar to ASR. In general, training S2T translation models is challenging because speech-to-text aligned data is scarce. Typical responses to this challenge include data augmentation, pre-training, knowledge distillation with machine translation, and so on. However, an in-depth discussion of these methods goes beyond the scope of this section on Transformers. Interested readers can refer to a recent survey on speech translation for more detailed information (Xu et al., 2023a).

6.4 Vision Models

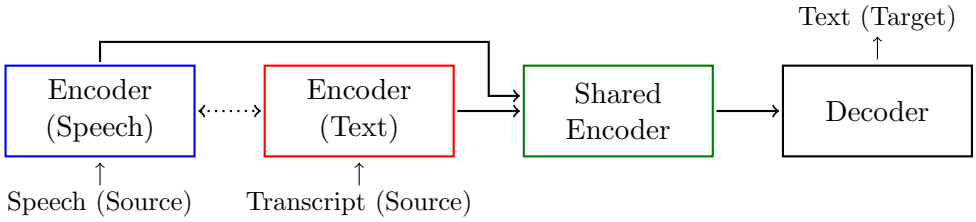
While Transformers were first used in NLP, their application to other domains has been a prominent research topic. In computer vision, for instance, there is a notable trend of shifting from CNNs to Transformers as backbone models. In this subsection, we consider the **Vision Transformer** (**ViT**) — an interesting application of Transformers to image classification (Dosovitskiy et al., 2021). The Vision Transformer is a milestone model that opened the door to purely Transformer-based vision models. Here, we consider the basic structure of the Vision Transformer to keep this section focused and coherent, although extensive



(a) Single Encoder + Single Decoder



(b) Decoupled Encoder + Single Decoder



(c) Two-stream Encoder + Single Decoder

Figure 18: Architectures of speech-to-text translation models based on Transformers. In addition to the standard encoder-decoder architecture, we can explicitly model the acoustic and textual (semantic) information using two separate encoders, called the speech encoder and the text encoder. In the decoupled encoder architecture, the two encoders are stacked, that is, text encoding is a subsequent process after speech encoding. In the two-stream encoder architecture, the two encoders work in parallel, and their outputs are merged using an additional encoder, called the shared encoder. The dotted line indicates the potential for interaction between the two encoders. For example, we could define a loss function to minimize the difference between their outputs, thereby guiding the model towards more aligned representations.

literature exists on the Vision Transformer and its variants. More detailed discussions of vision models can be found in recent surveys (Han et al., 2022; Liu et al., 2023b).

The core idea behind the Vision Transformer is to transform an image into a sequence of visual tokens, and input this sequence into a Transformer encoder to generate a representation of the image. The Transformer encoder is standard, and so we will not discuss it here, given the introduction to Transformers we have presented so far in this chapter. Mapping a 2D image into a sequence of tokens requires additional processing. Suppose we have an image represented as an $H \times W \times C$ feature map, where H is the height, W is the width, and C is the number of channels. The first step is to partition this image into a set of **patches**. Suppose all patches are squares of side length P. Then, the resulting patches can be represented by feature maps of shape $P \times P \times C$. By ordering these patches in a specific manner, we obtain a sequence of $\frac{HW}{P^2}$ patches, with each patch being treated as a "token."

Given this patch sequence, the subsequent steps are straightforward. For the patch at each position, we obtain a d-dimensional embedding via a linear transformation of the flattened patch. The input to the Transformer encoder is a sequence of d-dimensional vectors, each of which is the sum of the corresponding patch embedding and positional embedding. Figure 19 illustrates the patching and embedding steps in the Vision Transformer.

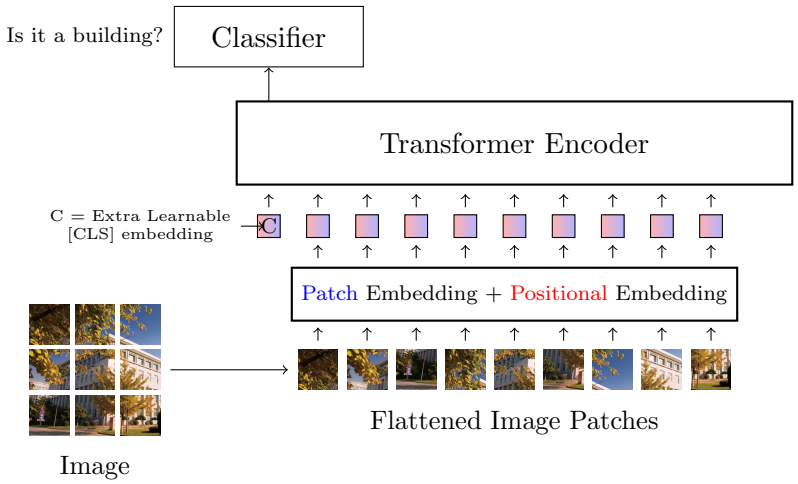


Figure 19: Illustration of the Vision Transformer for image classification (Dosovitskiy et al., 2021). There are three steps. In the first step, the input image is partitioned into patches, which are then flattened and mapped into embeddings. In the second step, a Transformer encoder is employed to process the sequence of embeddings, representing the image as a real-valued vector (e.g., the output of the encoder at the first position). In the last step, a classifier is built on top of this image representation.

Once we have a sequence of vectors for representing the image, we can employ the Transformer encoder to encode the sequence. The encoding process is exactly the same as that in text encoding, as discussed in Section 6.2. For classification problems, we need only

a single representation of the input. It is convenient to take the output of the encoder at position 0 (denoted by $\mathbf{h}_0^L$) and feed it into a classifier. Given that the first token ([CLS]) serves as a special global token that is attended to by all other tokens, $\mathbf{h}_0^L$ can be viewed as an aggregated representation of the entire sequence.

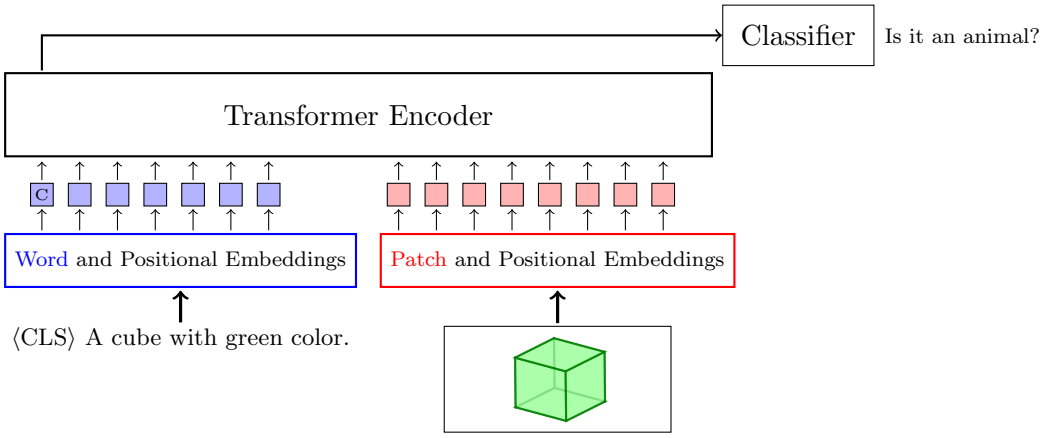
A standard way to train the Vision Transformer is to minimize a task-specific loss on labeled data, such as ImageNet. More recently, inspired by self-supervised learning in BERT-like models, there have been successful attempts to train Transformer-based image encoders on large-scale unlabeled data (Bao et al., 2021; Caron et al., 2021; He et al., 2022). Note that one of the most significant contributions of the Vision Transformer is that it unifies the representation models for different modalities. This suggests that if an object, whether an image or text, is represented as a sequence of embeddings, it can be easily modeled using the Transformer architecture.

6.5 Multimodal Models

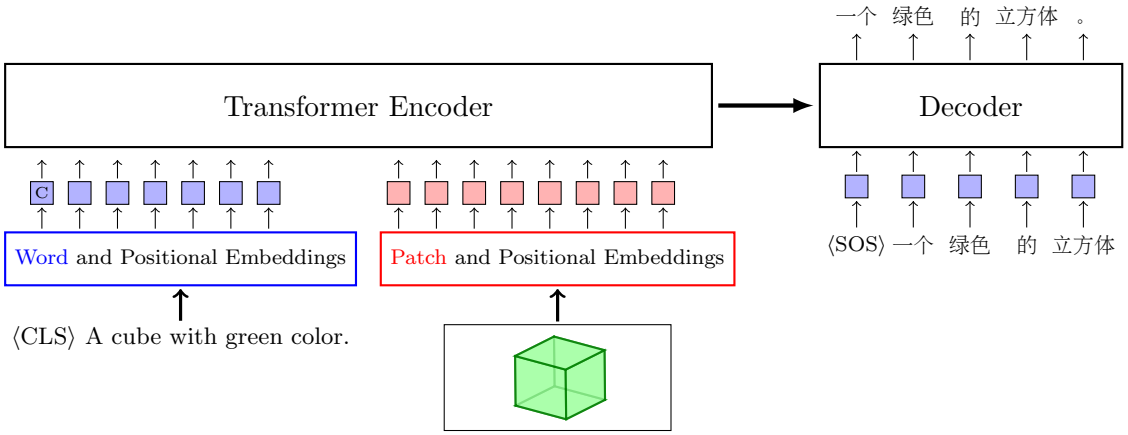
The above discussion of the Vision Transformer offers the possibility of unifying the representations from multiple modalities using the same Transformer architecture. In fact, many recent multimodal systems draw heavy inspiration from Transformers (Xu et al., 2023c). Such systems convert objects from different modalities into vector sequences and feed these vectors into a single Transformer model. The output is a fused representation of all inputs, which can then be used in downstream systems.

As a simple example, consider the task of encoding a pair consisting of text and its corresponding image. First, we represent both the text and the image as sequences of embeddings that have the same dimensionality. This is a common step in sequence modeling, which we have encountered many times so far. We can do this by using either a simple embedding model (e.g., a word or patch embedding model) or a pre-trained model (e.g., a vision model). Then, these two sequences are concatenated into a long sequence comprising both textual and visual embeddings. The subsequent step is standard: a Transformer encoder takes the concatenated sequence of embeddings as input and produces representations of the text and image as output. Note that concatenating textual and visual sequences is one of the simplest methods for vision-text modeling. There are several alternative ways to merge information from different modalities. For example, we can feed visual representations into the attention layers of a text encoder or decoder (Alayrac et al., 2022; Li et al., 2022d).

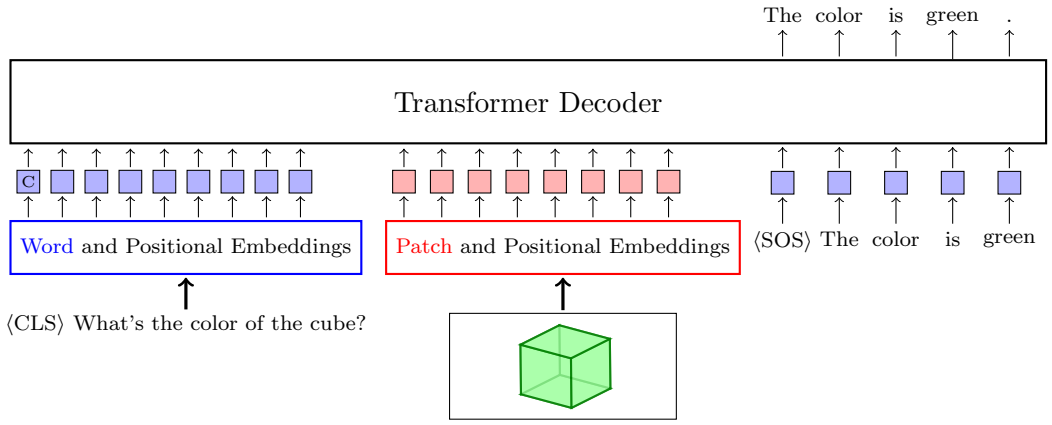
The above multimodal encoder can be used in both encoder-only and encoder-decoder systems. For encoder-only systems, consider an example where, given an image and its description, we predict the class of the image using a classifier built on top of the encoder (Kim et al., 2021). For encoder-decoder systems, we pair the encoder with a decoder, as in sequence-to-sequence modeling (Cho et al., 2021). For example, we might employ a Transformer decoder to generate text based on the output of the encoder. A common application of this architecture is **visual question answering** (**VQA**), where an image and a question about the image are provided, and the system is tasked with generating an answer (Antol et al., 2015). The architectures of these models are illustrated in Figure 20 (a-b).



(a) Multi-modal Encoder + Classifier



(b) Multi-modal Encoder + Text Decoder (Translation)



(c) Multi-modal Decoder (Language Modeling)

Figure 20: Vision-text models. Blue boxes represent word+position embeddings, and red boxes represent image patch+position embeddings.

Large language models have also been extended to handle both textual and other modalities, such as images, video, and audio, leading to new advances in multimodal processing (Liu et al., 2023a; Yin et al., 2023). By representing all inputs as a sequence of token embeddings, the problem becomes straightforward: we predict the next token given its context. This can be done using decoder-only systems, as shown in Figure 20 (c).

7. Summary

Transformer models have been widely adopted since the architecture was first proposed by Vaswani et al. (2017). This adoption has accelerated the development of a wide range of algorithms, systems, and concepts. A thorough discussion of Transformers requires broad coverage, and so it is impossible to address all topics or provide an exhaustive list of references. Figure 21 provides a high-level overview of the Transformer landscape. Note that these models and related techniques can be categorized in various ways, and we present one such perspective. To summarize, we highlight the following points:

- **Foundations of Transformers**. Although the impact of Transformers has been revolutionary, they are not completely "new" models. From a deep learning perspective, Transformers are composed of common building blocks, including word and positional embeddings (Bengio et al., 2003; Gehring et al., 2017; Mikolov et al., 2013), attention mechanisms (Bahdanau et al., 2014; Luong et al., 2015), residual connections (He et al., 2016), layer normalization (Ba et al., 2016), and so on. Many of these components were presented in earlier systems, for example, similar ideas with QKV attention can be found in memory networks (Sukhbaatar et al., 2015) and hierarchical attention networks (Yang et al., 2016). Transformers offer a novel approach to integrating these components, resulting in a unique architecture. For example, in Transformers, the combination of multi-head attention and dot-product QKV attention, along with the incorporation of layer normalization and residual connections, gives rise to a distinctive neural network block, specifically a self-attention sub-layer. This design has since become a de facto standard in many follow-on sequence modeling systems.
- **Attention Models**. The success of Transformers on NLP tasks has largely been attributed to the use of multi-head self-attention for sequence modeling. This has led to a surge of interest in enhancing the attention mechanisms within Transformers. While it is impossible to detail every attention model, there are several notable research directions. One prominent direction involves modifying the forms of QKV attention and multi-head attention for improved performance. The scope of this direction is vast, as there are numerous aspects to consider when enhancing Transformers (Lin et al., 2022a). For example, one may add new components to self-attention sub-layers to adapt them to specific tasks, resulting in various Transformer variants. A second direction is to incorporate prior knowledge into the design of attention models. This makes sense, because much of the emphasis in traditional NLP has been on using linguistic insights to guide system design, and we generally want NLP systems to be linguistically explainable. For example, many Transformer-based systems take syntactic parses as input in various forms and make use of syntax in sequence modeling. A third direction is to develop efficient attention models (Tay et al., 2020b). Self-

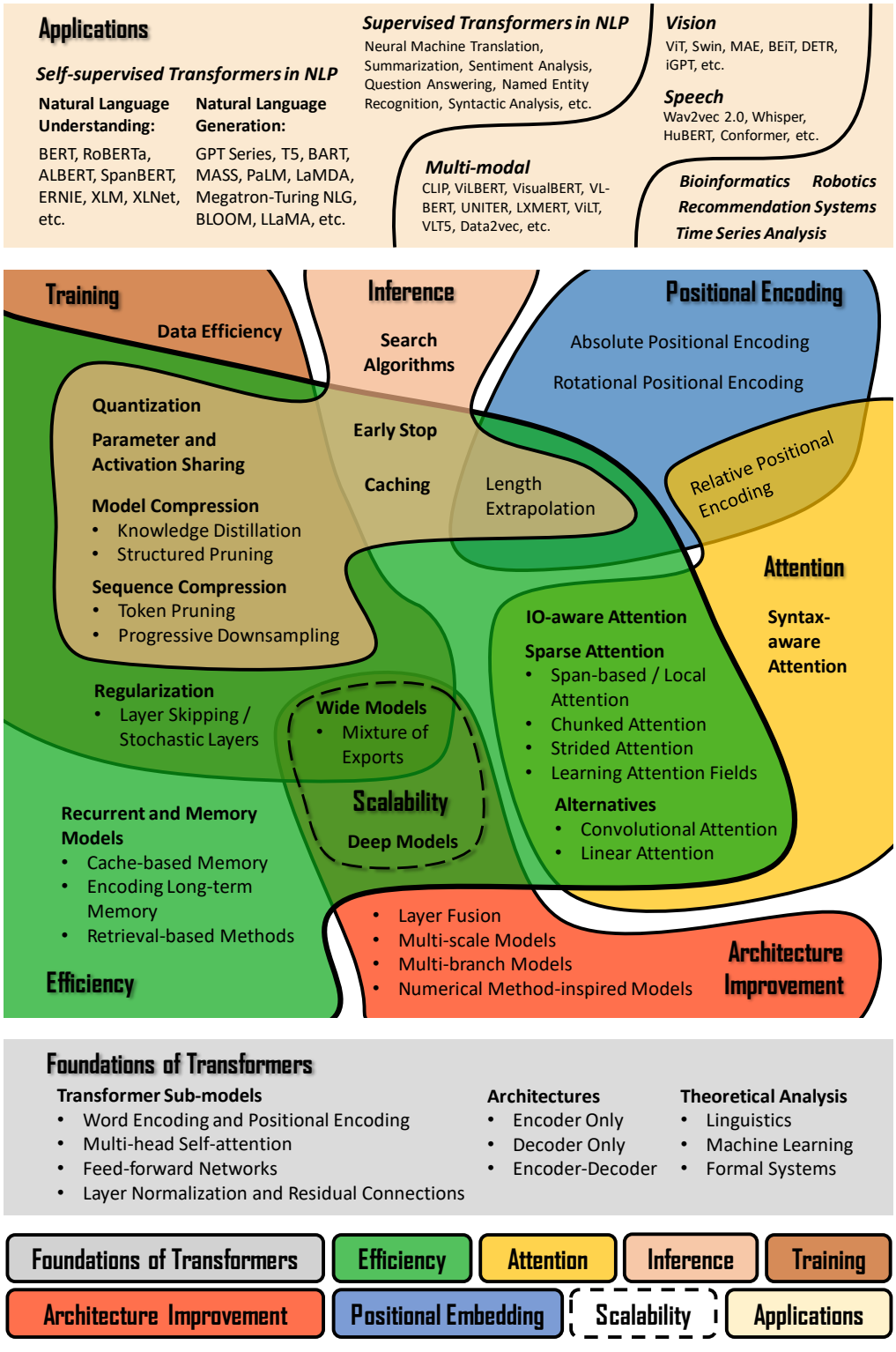


Figure 21: An overview of Transformers.

attention has long been criticized for its quadratic time complexity and dependency on all previous tokens for each new token. In response, many researchers have focused on simplifying the structure of self-attention, or on approximating it using sparse or recurrent models. This concern for efficiency also motivates the development of alternatives to self-attention, such as attention models with linear time complexity. In addition to exploring stronger and more efficient attention models, it is natural to examine what knowledge is learned by such models. Interestingly, researchers have found that the underlying structure of languages can be learned by multi-head self-attention models, although these models are not trained to represent such knowledge (Manning et al., 2020).

- **Word and Positional Embeddings**. Transformers represent each input word (or token) through a combination of word and positional embeddings. Learning these word embeddings is not unique to Transformers; one can either utilize pre-trained embeddings, such as Word2Vec or GloVe, or treat them as learnable parameters within the model. A closely related issue is the tokenization of input sequences. Generally, tokenization determines the number of resulting tokens and affects the difficulty of learning their corresponding embeddings. Therefore, selecting an appropriate tokenization method is crucial for many applications. Furthermore, positional embeddings play a vital role in Transformers, as the underlying attention mechanisms are order-insensitive by design (Dufter et al., 2022). While positional encoding is a general research topic, much effort has been dedicated to tailoring it for Transformers, leading to various architectural modifications (Huang et al., 2018; Shaw et al., 2018). Additionally, studies show that for sequences significantly longer than those encountered during training, better extrapolation can be achieved by replacing sinusoidal embeddings with rotary positional embeddings (RoPE) or by scaling attention weights with positional scalars (Press et al., 2021; Raffel et al., 2020; Su et al., 2021).
- **Training and Model Scaling**. In the era of deep learning, high-performance systems are typically built using large-scale neural networks. A straightforward approach to increasing Transformer capacity is to stack more layers or enlarge the hidden representation size. Indeed, deep and wide Transformer models consistently outperform their smaller counterparts. However, training extremely large models poses significant challenges, particularly when applying gradient descent over vast datasets, which demands substantial computational resources. Engineering solutions involve distributing training across computer clusters (Chowdhery et al., 2022; Lepikhin et al., 2021). Although distributed training is a general paradigm not restricted to Transformers, it heavily influences architectural design; for instance, sparse mixture-of-experts (MoE) models can facilitate training with massive parameter sets and serve as the foundation for many expansive systems. Scaling also enables the study of scaling laws — quantifying how performance scales with model size, data volume, and compute budget (Hestness et al., 2017; Kaplan et al., 2020). This scaling is occasionally accompanied by "emergence" (Wei et al., 2022), where models manifest unpredictable new abilities. In recent research, the acquisition of such emergent abilities has been viewed as a hallmark of powerful large language models.

- **Efficient Models**. There are different goals for efficiency. A system may prioritize memory efficiency for memory-bound problems or low latency when inference speed is critical. Balancing these requirements leads to diverse optimization strategies. In Transformers, many optimizations involve modifying attention mechanisms; for instance, several variants reduce the memory footprint when processing long sequences (Tay et al., 2020b), while others minimize computation to decrease latency. Furthermore, Transformers can benefit from architecture-independent optimizations typical of neural networks, such as conditional computation, knowledge distillation, structured pruning, and sequence compression. Efficiency can also be addressed from a computer architecture perspective (Kim et al., 2023). In sequence-to-sequence tasks, for example, the encoding process is typically compute-intensive, whereas the autoregressive decoding process is often IO-intensive. Therefore, we can employ different optimization methods for different components of Transformers.
- **Inference**. The inference problem is commonly discussed in sequence generation. In NLP, we often need to find the "best" hypothesis in a space involving sequences of hundreds or even thousands of tokens over a vocabulary. As a classic search problem in artificial intelligence, various algorithms are applicable, including breadth-first, depth-first, and A* search. In many practical applications of NLP, the efficiency of the search systems is an important consideration. As a result, optimized search algorithms are required. Most of these algorithms have been explored in machine translation and ASR, and are directly applicable to neural text generation models like Transformers. Beyond traditional search, recent optimizations such as speculative decoding are specifically tailored to the Transformer architecture (Leviathan et al., 2023). Moreover, the efficient attention models mentioned previously are now widely deployed to accelerate large language models and neural machine translation systems (Dao et al., 2023; Heafield et al., 2021).
- **Applications**. Applications of Transformers cover a wide variety of NLP problems. During the development of Transformers, they were at first used to build supervised models that perform particular tasks. Later, a greater success was achieved by using them as backbone networks for large scale self-supervised learning of foundation models (Bommasani et al., 2021). This shift markedly changed the NLP paradigm: a model is first pre-trained on vast amounts of text to acquire general linguistic knowledge, and then adapted to downstream tasks through efficient methods such as fine-tuning or prompting. Beyond NLP, we have witnessed an explosion of Transformer applications in fields like computer vision, speech processing, and bioinformatics. The core idea is to represent diverse data types as sequences of tokens, extending Transformers into general-purpose representation models capable of handling multi-modal data.
- **Large Language Models as Foundation Models**. Transformers form the basis of recent large language models, such as the GPT series, which show surprising breakthroughs in NLP (Bubeck et al., 2023; Yang et al., 2023a). Much of the research in large language models is more or less related to Transformers. As discussed in Section 6.1, training these language models is often synonymous with training large-scale Transformer decoders, and architectural refinements to the latter directly benefit the former. On the other hand, the rapid development of large language models has

spurred further innovation in Transformer-related techniques, particularly in developing efficient and low-cost adaptation methods for massive models.

- **Theoretical Analysis**. While Transformers dominate empirically, their theoretical foundations have historically received less focus compared to engineering improvements — an empirical-first trend common across the broader machine learning community. In response, researchers have begun to analyze Transformers more deeply. One approach is to view Transformers as deep neural networks and interpret them via mathematical tools. For instance, the residual connections within Transformers can be viewed as Euler discretizations of ODEs. This equivalence suggests that we can leverage insights from numerical ODE methods to inform model design. Another promising avenue aims to develop a theoretical understanding of the self-attention mechanism, which distinguishes Transformers from earlier deep learning models. Studies have interpreted self-attention through various theoretical lenses, including data compression (Yu et al., 2023), optimization (Li et al., 2022c), and function approximation (Yun et al., 2019). Moreover, Transformers can be mapped to formal systems, such as Turing machines (Pérez et al., 2018), counter machines (Bhattamishra et al., 2020), regular and context-free languages (Hahn, 2020), Boolean circuits (Hao et al., 2022; Merrill et al., 2022), programming languages (Weiss et al., 2021), and first-order logic (Chiang et al., 2023). These connections provide theoretical tools to rigorously study the expressivity of Transformers. It is worth noting, however, that while we can analyze Transformers through these diverse perspectives, no unified theory yet explains the fundamental nature of these models. Addressing this theoretical gap remains a central challenge in the field, yet it is essential for developing complex neural systems with explainable and predictable behaviors.

Acknowledgements

We would like to thank Yongyu Mu, Chenglong Wang, Bei Li, Weiqiao Shan, Yuchun Fan, Kaiyan Chang, Tong Zheng, and Huiwen Bao for their suggestions on improving the early version of this work.

A. Sinusoidal Positional Encoding

In Transformers, positions are represented as vectors. Although vectorizing the representations of positions sounds complicated, a simple idea is to use a carrying system which describes how a natural number is expressed by a polynomial with respect to a base (Kernes, 2021). For example, i can be written as

$$i = \sum_{k=0}^{k_{\max}} a(i,k) b^k \tag{191}$$

where $a(i,k)$ is the k-th digit, $k_{\max}+1$ is the maximum number of digits, and b is the base of the system. The carrying occurs when $a(i,k)$ reaches b: we increase $a(i,k+1)$ by 1 and roll back $a(i,k)$ to 0. In this way we can change $a(i,k)$ with a period of b^k, that is, $a(i,0)$ changes with a period of b^0, $a(i,1)$ changes with a period of b^1, $a(i,2)$ changes with a period of b^2, and so on.

Using this system, i can be represented as a vector

$$\text{PE}(i) = \begin{bmatrix} a(i,0) & a(i,1) & ... & a(i,k_{\max}) \end{bmatrix} \tag{192}$$

For example, when $b=2$, $\text{PE}(11) = \begin{bmatrix} 1 & 1 & 0 & 1 \end{bmatrix}$. However, in Eq. (192), $\text{PE}(i)$ is still a discrete function. We may want a continuous vector representation that can describe intermediate states between discrete events. Considering $a(i,k)$ as a periodic function, a common choice is the sine function. Thus $a(i,k)$ can be re-defined, as follows

$$a(i,k) = \sin(i \cdot \omega_k) \tag{193}$$

This function has an amplitude of 1 and a period of $\frac{2\pi}{\omega_k}$. Using an analogous form of periods to that used in Eq. (191), we define ω_k as

$$\omega_k = \frac{1}{(b_{\text{model}})^{k/d_{\text{model}}}} \tag{194}$$

where $b_{\text{model}} > 0$ and $d_{\text{model}} > 0$ are hyper-parameters of the model. Obviously, we have $\frac{2\pi}{\omega_0} < \frac{2\pi}{\omega_1} < ... < \frac{2\pi}{\omega_{k_{\max}}}$.

Similarly, we can define $a(i,k)$ via the cosine function

$$a(i,k) = \cos(i \cdot \omega_k) \tag{195}$$

Taking both Eqs. (193) and (195), we create a new representation of i, as follows

$$\text{PE}(i) = \begin{bmatrix} \sin(i \cdot \omega_0) & \cos(i \cdot \omega_0) & ... & \sin(i \cdot \omega_{k_{\max}}) & \cos(i \cdot \omega_{k_{\max}}) \end{bmatrix} \tag{196}$$

Vaswani et al. (2017) instantiated the above form by setting $b_{\text{model}} = 10,000$. Let $\text{PE}(i,k)$ be the k-th dimension of $\text{PE}(i)$. Vaswani et al. (2017)'s version of positional encoding is written as

$$\text{PE}(i,2k) = \sin(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}) \tag{197}$$

$$\text{PE}(i,2k+1) = \cos(i \cdot \frac{1}{10000^{2k/d_{\text{model}}}}) \tag{198}$$

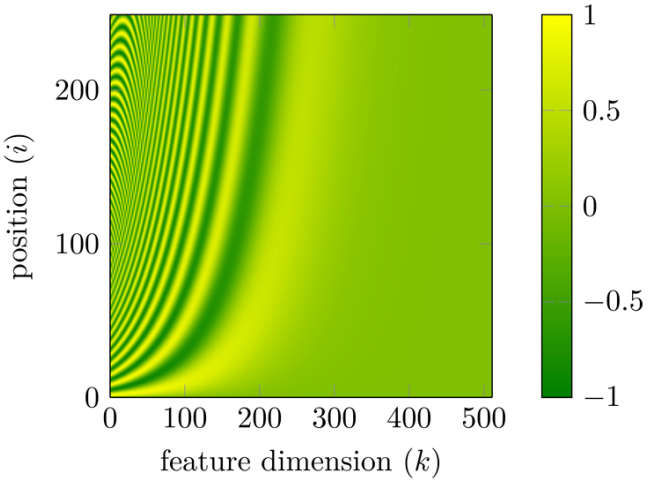


Figure 22: A heat map of the positional embedding model of Eqs. (197) and (198) ($b_{\text{model}} = 10,000$ and $d_{\text{model}} = 512$). Consider a position i (i.e., the i-th row), then move another position j from i upwards or downwards. Intuitively, when i and j are closer, the corresponding row vectors are more similar. By contrast, when j moves away from i, the similarity is not that obvious. This property helps explain the idea behind the positional embedding model: the "distance" between two positions is implicitly modeled by comparing their multi-dimensional representations.

Choosing $b_{\text{model}} = 10,000$ is empirical. One can adjust it for specific tasks. Figure 22 plots the positional encoding for different positions. We see that, when k becomes larger, the change of the color follows a larger period.

Note that Eqs. (197) and (198) have a useful property that $\text{PE}(i + \mu)$ can be easily expressed by a linear function of $\text{PE}(i)$ for a given offset μ[28]

$$\begin{aligned} \text{PE}(i+\mu, 2k) &= \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k+1) + \\ &\quad \text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k) \qquad (201) \\ \text{PE}(i+\mu, 2k+1) &= \text{PE}(i, 2k+1) \cdot \text{PE}(\mu, 2k+1) - \\ &\quad \text{PE}(i, 2k) \cdot \text{PE}(\mu, 2k) \qquad (202) \end{aligned}$$

The resulting benefit is that the encoding can somewhat model relative positions. That is, the state at position $i + \mu$ can be described by starting with i and then appending it with the offset μ.

28. One can derive these by taking

$$\begin{aligned} \sin(\alpha+\beta) &= \sin(\alpha) \cdot \cos(\beta) + \cos(\alpha) \cdot \sin(\beta) \qquad (199) \\ \cos(\alpha+\beta) &= \cos(\alpha) \cdot \cos(\beta) - \sin(\alpha) \cdot \sin(\beta) \qquad (200) \end{aligned}$$

When applying the sinusoidal positional encoding, one way is to concatenate $\mathbf{x}_i$ and $\text{PE}(i)$. In Vaswani et al. (2017)'s work, they instead assume $\text{PE}(i)$ to be a vector of the same size as $\mathbf{x}_i$ (i.e., $|\text{PE}(i)| = |\mathbf{x}_i| = d_e$), and add $\text{PE}(i)$ to $\mathbf{x}_i$, like this

$$\mathbf{xp}_i \quad = \quad \mathbf{x}_i + \text{PE}(i) \tag{203}$$

This sinusoidal addictive model has been the basis of many positional encoding approaches (Dehghani et al., 2018; Likhomanenko et al., 2021; Su et al., 2021).

References

Yossi Adi, Einat Kermany, Yonatan Belinkov, Ofer Lavi, and Yoav Goldberg. Fine-grained analysis of sentence embeddings using auxiliary prediction tasks. In *Proceedings of International Conference on Learning Representations*, 2016.

Joshua Ainslie, Santiago Ontanon, Chris Alberti, Vaclav Cvicek, Zachary Fisher, Philip Pham, Anirudh Ravula, Sumit Sanghai, Qifan Wang, and Li Yang. Etc: Encoding long and structured inputs in transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 268–284, 2020.

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35:23716–23736, 2022.

Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015.

Karl J Åström and Björn Wittenmark. *Computer-controlled systems: theory and design*. Courier Corporation, 2013.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Gary Cottrell, and Julian McAuley. Rezero is all you need: Fast convergence at large depth. In *Proceedings of Uncertainty in Artificial Intelligence*, pages 1352–1361. PMLR, 2021.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Jiangang Bai, Yujing Wang, Yiren Chen, Yaming Yang, Jing Bai, Jing Yu, and Yunhai Tong. Syntax-bert: Improving pre-trained transformers with syntax trees. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3011–3020, 2021.

Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. Beit: Bert pre-training of image transformers. In *Proceedings of International Conference on Learning Representations*, 2021.

Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Daniel Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, et al. Pathways: Asynchronous distributed dataflow for ml. In *Proceedings of Machine Learning and Systems*, volume 4, pages 430–449, 2022.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

Irwan Bello. Lambdanetworks: Modeling long-range interactions without attention. In *Proceedings of International Conference on Learning Representations*, 2020.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

Yoshua Bengio, R é jean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:11371155, 2003.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal. On the ability and limitations of transformers to recognize formal languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7096–7116, 2020.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren E. Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir P. Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher R'e, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *ArXiv*, 2021.

Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Understanding and overcoming the challenges of efficient transformer quantization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7947–7969, 2021.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Maxime Burchi and Valentin Vielzeuf. Efficient conformer: Progressive downsampling and grouped attention for automatic speech recognition. In *Proceedings of 2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 8–15. IEEE, 2021.

Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.

Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, et al. Universal sentence encoder. *arXiv preprint arXiv:1803.11175*, 2018.

Kehai Chen, Rui Wang, Masao Utiyama, Eiichiro Sumita, and Tiejun Zhao. Syntax-directed attention for neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence*, 2018a.

Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Mike Schuster, Noam Shazeer, Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. The best of both worlds: Combining recent advances in neural machine translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 76–86, 2018b.

Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018c.

Tianqi Chen, Ian Goodfellow, and Jonathon Shlens. Net2net: Accelerating learning via knowledge transfer. *arXiv preprint arXiv:1511.05641*, 2015.

David Chiang, Peter Cholak, and Anand Pillay. Tighter bounds on the expressivity of transformer encoders. *arXiv preprint arXiv:2301.10743*, 2023.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Jaemin Cho, Jie Lei, Hao Tan, and Mohit Bansal. Unifying vision-and-language tasks via text generation. In *International Conference on Machine Learning*, pages 1931–1942. PMLR, 2021.

Do Kook Choe and Eugene Charniak. Parsing as language modeling. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2331–2336, 2016.

Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *Proceedings of International Conference on Learning Representations*, 2020.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D Manning. What does bert look at? an analysis of bert' s attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286, 2019.

Alexis Conneau, Germán Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. What you can cram into a single vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136, 2018.

Anna Currey and Kenneth Heafield. Multi-source syntactic neural machine translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2961–2966, 2018.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. `https://pytorch.org/blog/flash-decoding/`, 2023. Retrieved 2023-10-23.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.

Luciano Del Corro, Allie Del Giorno, Sahaj Agarwal, Bin Yu, Ahmed Awadallah, and Subhabrata Mukherjee. Skipdecode: Autoregressive skip decoding with batching and caching for efficient llm inference. *arXiv preprint arXiv:2307.02628*, 2023.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

Mattia Antonino Di Gangi, Matteo Negri, Roldano Cattoni, Roberto Dessi, and Marco Turchi. Enhancing transformer for end-to-end speech-to-text translation. In *Proceedings of Machine Translation Summit XVII: Research Track*, pages 21–31, 2019.

Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, et al. Cogview: Mastering text-to-image generation via transformers. *Advances in Neural Information Processing Systems*, 34:19822–19835, 2021.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of ICLR 2021*, 2021.

Philipp Dufter, Martin Schmitt, and Hinrich Schütze. Position information in transformers: An overview. *Computational Linguistics*, 48(3):733–763, 2022.

Weinan Ee. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5:1–11, 02 2017.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *Proceedings of International Conference on Learning Representations*, 2020.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *The Journal of Machine Learning Research*, 20(1):1997–2017, 2019.

Haoqi Fan, Bo Xiong, Karttikeya Mangalam, Yanghao Li, Zhicheng Yan, Jitendra Malik, and Christoph Feichtenhofer. Multiscale vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6824–6835, 2021.

Yang Fan, Shufang Xie, Yingce Xia, Lijun Wu, Tao Qin, Xiang-Yang Li, and Tie-Yan Liu. Multi-branch attentive transformer. *arXiv preprint arXiv:2006.10270*, 2020.

William Fedus, Jeff Dean, and Barret Zoph. A review of sparse expert models in deep learning. *arXiv preprint arXiv:2209.01667*, 2022a.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 23(1):5232–5270, 2022b.

Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International conference on machine learning*, pages 1243–1252. PMLR, 2017.

Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, pages 291–326. Chapman and Hall/CRC, 2022.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. *Advances in neural information processing systems*, 30, 2017.

Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129:1789–1819, 2021.

Robert M. Gray. Quantization. *IEEE transactions on information theory*, 44(6):2325–2383, 1998.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *Proceedings of International Conference on Learning Representations*, 2021.

Albert Gu, Karan Goel, Ankit Gupta, and Christopher Ré. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 35:35971–35983, 2022a.

Albert Gu, Karan Goel, Khaled Saab, and Chris Ré. Structured state spaces: Combining continuous-time, recurrent, and convolutional models. `https://hazyresearch.stanford.edu/blog/2022-01-14-s4-3`, 2022b. Retrieved 2022-01-14.

Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. *Proceedings of Interspeech 2020*, pages 5036–5040, 2020.

Qipeng Guo, Xipeng Qiu, Pengfei Liu, Xiangyang Xue, and Zheng Zhang. Multi-scale self-attention for text classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7847–7854, 2020.

Madan Gupta, Liang Jin, and Noriyasu Homma. *Static and dynamic neural networks: from fundamentals to advanced theory*. John Wiley & Sons, 2004.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *Proceedings of International conference on machine learning*, pages 3929–3938. PMLR, 2020.

Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse problems*, 34(1):014004, 2017.

Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.

Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence*, 45(1):87–110, 2022.

Wei Han, Zhengdong Zhang, Yu Zhang, Jiahui Yu, Chung-Cheng Chiu, James Qin, Anmol Gulati, Ruoming Pang, and Yonghui Wu. Contextnet: Improving convolutional neural networks for automatic speech recognition with global context. In *Proceedings of Interspeech 2020*, pages 3610–3614, 2020.

Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. Dynamic neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7436–7456, 2021.

Jie Hao, Xing Wang, Shuming Shi, Jinfeng Zhang, and Zhaopeng Tu. Multi-granularity self-attention for neural machine translation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 887–897, 2019.

Yiding Hao, Dana Angluin, and Robert Frank. Formal language recognition by hard attention transformers: Perspectives from circuit complexity. *Transactions of the Association for Computational Linguistics*, 10:800–810, 2022.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *Proceedings of ECCV 2016*, pages 630–645, 2016.

Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. In *Proceedings of International Conference on Learning Representations*, 2021.

Kenneth Heafield, Qianqian Zhu, and Roman Grundkiewicz. Findings of the WMT 2021 shared task on efficient translation. In *Proceedings of the Sixth Conference on Machine Translation*, pages 639–651, 2021.

Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, 2019.

Felix Hill, Kyunghyun Cho, and Anna Korhonen. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1367–1377, 2016.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Lu Hou, Zhiqi Huang, Lifeng Shang, Xin Jiang, Xiao Chen, and Qun Liu. Dynabert: Dynamic bert with adaptive width and depth. *Advances in Neural Information Processing Systems*, 33:9782–9793, 2020.

Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019.

Chi Hu, Chenglong Wang, Xiangnan Ma, Xia Meng, Yinqiao Li, Tong Xiao, Jingbo Zhu, and Changliang Li. Ranknas: Efficient neural architecture search by pairwise ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 2469–2480, 2021.

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M Dai, Matthew D Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *Proceedings of International Conference on Learning Representations*, 2018.

Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *Proceedings of the 14th European Conference*, pages 646–661. Springer, 2016.

Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

Zhiheng Huang, Davis Liang, Peng Xu, and Bing Xiang. Improve transformer models with better relative position embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 3327–3335, 2020.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems*, volume 3, pages 711–732, 2021.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. Perceiver io: A general architecture for structured inputs & outputs. In *Proceedings of International Conference on Learning Representations*, 2021.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

Jonathan Kernes. Master positional encoding: Part i, 05 2021. URL `https://towardsdatascience.com/master-positional-encoding-part-i-63c05d90a0c3`.

Patrick Kidger. On neural differential equations. *arXiv preprint arXiv:2202.02435*, 2022.

Gyuwan Kim and Kyunghyun Cho. Length-adaptive transformer: Train once with length drop, use anytime with search. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6501–6511, 2021.

Najoung Kim, Roma Patel, Adam Poliak, Patrick Xia, Alex Wang, Tom McCoy, Ian Tenney, Alexis Ross, Tal Linzen, Benjamin Van Durme, et al. Probing what different nlp tasks teach machines about function word comprehension. In *Proceedings of the Eighth Joint Conference on Lexical and Computational Semantics (* SEM 2019)*, pages 235–249, 2019.

Sehoon Kim, Coleman Hooper, Thanakul Wattanawong, Minwoo Kang, Ruohan Yan, Hasan Genc, Grace Dinh, Qijing Huang, Kurt Keutzer, Michael W Mahoney, et al. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*, 2023.

Wonjae Kim, Bokyung Son, and Ildoo Kim. Vilt: Vision-and-language transformer without convolution or region supervision. In *Proceedings of International Conference on Machine Learning*, pages 5583–5594. PMLR, 2021.

Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, 2016.

Young Jin Kim and Hany Hassan Awadalla. Fastformers: Highly efficient transformer models for natural language understanding. In *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*, pages 149–158, 2020.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *Proceedings of International Conference on Learning Representations*, 2020.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, 2018.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

François Lagunas, Ella Charlaix, Victor Sanh, and Alexander M Rush. Block pruning for faster transformers. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 10619–10629, 2021.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys. *Advances in Neural Information Processing Systems*, 32, 2019.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of International Conference on Learning Representations*, 2021.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Bei Li, Hui Liu, Ziyang Wang, Yufan Jiang, Tong Xiao, Jingbo Zhu, Tongran Liu, and Changliang Li. Does multi-encoder help? a case study on context-aware neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3512–3518, 2020a.

Bei Li, Ziyang Wang, Hui Liu, Yufan Jiang, Quan Du, Tong Xiao, Huizhen Wang, and Jingbo Zhu. Shallow-to-deep training for neural machine translation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 995–1005, 2020b.

Bei Li, Ziyang Wang, Hui Liu, Quan Du, Tong Xiao, Chunliang Zhang, and Jingbo Zhu. Learning light-weight translation models from deep transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13217–13225, 2021.

Bei Li, Quan Du, Tao Zhou, Yi Jing, Shuhan Zhou, Xin Zeng, Tong Xiao, Jingbo Zhu, Xuebo Liu, and Min Zhang. Ode transformer: An ordinary differential equation-inspired model for sequence generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8335–8351, 2022a.

Bei Li, Tong Zheng, Yi Jing, Chengbo Jiao, Tong Xiao, and Jingbo Zhu. Learning multiscale transformer models for sequence generation. In *International Conference on Machine Learning*, pages 13225–13241. PMLR, 2022b.

Hongkang Li, Meng Wang, Sijia Liu, and Pin-Yu Chen. A theoretical understanding of shallow vision transformers: Learning, generalization, and sample complexity. In *The Eleventh International Conference on Learning Representations*, 2022c.

Junhui Li, Deyi Xiong, Zhaopeng Tu, Muhua Zhu, Min Zhang, and Guodong Zhou. Modeling source syntax for neural machine translation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 688–697, 2017.

Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, pages 12888–12900. PMLR, 2022d.

Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. Mvitv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022e.

Kaiyuan Liao, Yi Zhang, Xuancheng Ren, Qi Su, Xu Sun, and Bin He. A global past-future early exit method for accelerating inference of pre-trained language models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2013–2023, 2021.

Tatiana Likhomanenko, Qiantong Xu, Gabriel Synnaeve, Ronan Collobert, and Alex Rogozhnikov. Cape: Encoding relative positions with continuous augmented positional embeddings. *Advances in Neural Information Processing Systems*, 34:16079–16092, 2021.

Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI Open*, 2022a.

Ye Lin, Shuhan Zhou, Yanyang Li, Anxiang Ma, Tong Xiao, and Jingbo Zhu. Multi-path transformer is better: A case study on neural machine translation. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 5646–5656, 2022b.

Fenglin Liu, Xuancheng Ren, Zhiyuan Zhang, Xu Sun, and Yuexian Zou. Rethinking skip connection with layer normalization. In *Proceedings of the 28th international conference on computational linguistics*, pages 3586–3598, 2020a.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023a.

Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. Understanding the difficulty of training transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5747–5763, 2020b.

Peter J Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. In *Proceedings of International Conference on Learning Representations*, 2018.

Yang Liu, Yao Zhang, Yixin Wang, Feng Hou, Jin Yuan, Jiang Tian, Yang Zhang, Zhongchao Shi, Jianping Fan, and Zhiqiang He. A survey of visual transformers. *IEEE Transactions on Neural Networks and Learning Systems*, 2023b.

Yuchen Liu, Junnan Zhu, Jiajun Zhang, and Chengqing Zong. Bridging the modality gap for speech-to-text translation. *arXiv preprint arXiv:2010.14920*, 2020c.

Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1412–1421, 2015.

Christopher D Manning, Kevin Clark, John Hewitt, Urvashi Khandelwal, and Omer Levy. Emergent linguistic structure in artificial neural networks trained by self-supervision. *Proceedings of the National Academy of Sciences*, 117(48):30046–30054, 2020.

Pedro Henrique Martins, Zita Marinho, and André FT Martins. ∞-former: Infinite memory transformer-former: Infinite memory transformer. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5468–5485, 2022.

Saeed Masoudnia and Reza Ebrahimpour. Mixture of experts: a literature survey. *The Artificial Intelligence Review*, 42(2):275, 2014.

JS McCarley, Rishav Chakravarti, and Avirup Sil. Structured pruning of a bert-based question answering model. *arXiv preprint arXiv:1910.06360*, 2019.

William Merrill, Ashish Sabharwal, and Noah A Smith. Saturated transformers are constant-depth threshold circuits. *Transactions of the Association for Computational Linguistics*, 10:843–856, 2022.

Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? *Advances in neural information processing systems*, 32, 2019.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*, page 31113119, 2013.

Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.

Alan V Oppenheim and Ronald W Schafer. Digital signal processing(book). *Prentice-Hall*, 1975.

Antonio Orvieto, Samuel L Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Wonpyo Park, Dongju Kim, Yan Lu, and Minsu Cho. Relational knowledge distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3967–3976, 2019.

Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In *International conference on machine learning*, pages 4055–4064. PMLR, 2018.

Baoyun Peng, Xiao Jin, Jiaheng Liu, Dongsheng Li, Yichao Wu, Yu Liu, Shunfeng Zhou, and Zhaoning Zhang. Correlation congruence for knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5007–5016, 2019.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

H Peng, N Pappas, D Yogatama, R Schwartz, N Smith, and L Kong. Random feature attention. In *Proceedings of International Conference on Learning Representations (ICLR 2021)*, 2021.

Jorge Pérez, Javier Marinković, and Pablo Barceló. On the turing completeness of modern neural network architectures. In *Proceedings of International Conference on Learning Representations*, 2018.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, 2019.

Ngoc-Quan Pham, Thai-Son Nguyen, Jan Niehues, Markus Müller, Sebastian Stüker, and Alexander Waibel. Very deep self-attention networks for end-to-end speech recognition. *arXiv preprint arXiv:1904.13377*, 2019.

Telmo Pessoa Pires, António V Lopes, Yannick Assogba, and Hendra Setiawan. One wide feedforward is all you need. *arXiv preprint arXiv:2309.01826*, 2023.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, 2023.

Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of International Conference on Learning Representations*, 2021.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. Bpe-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892, 2020.

Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2555–2565, 2020.

Lawrence R Rabiner and Bernard Gold. Theory and application of digital signal processing. *Prentice-Hall*, 1975.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *Proceedings of International Conference on Learning Representations*, 2019.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J Liu, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140): 1–67, 2020.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2014.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

Michael Santacroce, Zixin Wen, Yelong Shen, and Yuanzhi Li. What matters in the structured pruning of generative language models? *arXiv preprint arXiv:2302.03773*, 2023.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.

Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A Smith. The right tool for the job: Matching model and instance complexities. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6640–6651, 2020.

Abigail See. Deep learning, structure and innate priors: A discussion between yann lecun and christopher manning, 02 2018. URL `http://www.abigailsee.com/2018/02/21/deep-learning-structure-and-innate-priors.html`.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, 2016.

Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, 2018.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of International Conference on Learning Representations*, 2017.

Dinghan Shen, Mingzhi Zheng, Yelong Shen, Yanru Qu, and Weizhu Chen. A simple but tough-to-beat data augmentation approach for natural language understanding and generation. *arXiv preprint arXiv:2009.13818*, 2020.

Xing Shi, Inkit Padhi, and Kevin Knight. Does string-based neural mt learn source syntax? In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1526–1534, 2016.

Maciej Skorski, Alessandro Temperoni, and Martin Theobald. Revisiting weight initialization of deep neural networks. In *Asian Conference on Machine Learning*, pages 1192–1207. PMLR, 2021.

David So, Quoc Le, and Chen Liang. The evolved transformer. In *Proceedings of International conference on machine learning*, pages 5877–5886. PMLR, 2019.

Matthias Sperber, Jan Niehues, Graham Neubig, Sebastian Stüker, and Alex Waibel. Self-attentional acoustic models. In *Proceedings of Interspeech 2018*, pages 3723–3727, 2018.

Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.

Pierre Stock, Angela Fan, Benjamin Graham, Edouard Grave, Rémi Gribonval, Herve Jegou, and Armand Joulin. Training with quantization noise for extreme model compression. In *Proceedings of International Conference on Learning Representations*, 2021.

Emma Strubell, Patrick Verga, Daniel Andor, David Weiss, and Andrew McCallum. Linguistically-informed self-attention for semantic role labeling. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 5027–5038, 2018.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.

Sainbayar Sukhbaatar, Édouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 331–335, 2019.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *Proceedings of 2nd International Conference on Learning Representations (ICLR 2014)*, 2014.

Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

Yi Tay, Dara Bahri, Liu Yang, Donald Metzler, and Da-Cheng Juan. Sparse sinkhorn attention. In *Proceedings of International Conference on Machine Learning*, pages 9438–9447. PMLR, 2020a.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *CoRR*, abs/2009.06732, 2020b.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, 2019a.

Ian Tenney, Patrick Xia, Berlin Chen, Alex Wang, Adam Poliak, R Thomas McCoy, Najoung Kim, Benjamin Van Durme, Samuel R Bowman, Dipanjan Das, et al. What do you learn from context? probing for sentence structure in contextualized word representations. In *Proceedings of International Conference on Learning Representations*, 2019b.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems*, volume 30, 2017.

Oriol Vinyals, Łukasz Kaiser, Terry Koo, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. *Advances in neural information processing systems*, 28, 2015.

Elena Voita, Pavel Serdyukov, Rico Sennrich, and Ivan Titov. Context-aware neural machine translation learns anaphora resolution. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1264–1274, 2018.

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808, 2019.

Eric Wallace, Yizhong Wang, Sujian Li, Sameer Singh, and Matt Gardner. Do nlp models know numbers? probing numeracy in embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5307–5315, 2019.

Hanrui Wang, Zhanghao Wu, Zhijian Liu, Han Cai, Ligeng Zhu, Chuang Gan, and Song Han. Hat: Hardware-aware transformers for efficient natural language processing. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7675–7688, 2020a.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022a.

Hongyu Wang, Shuming Ma, Shaohan Huang, Li Dong, Wenhui Wang, Zhiliang Peng, Yu Wu, Payal Bajaj, Saksham Singhal, Alon Benhaim, et al. Foundation transformers. *arXiv preprint arXiv:2210.06423*, 2022b.

Jue Wang, Ke Chen, Gang Chen, Lidan Shou, and Julian McAuley. Skipbert: Efficient inference with shallow layer skipping. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7287–7301, 2022c.

Lin Wang and Kuk-Jin Yoon. Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks. *IEEE transactions on pattern analysis and machine intelligence*, 44(6):3048–3068, 2021.

Peihao Wang, Rameswar Panda, Lucas Torroba Hennigen, Philip Greengard, Leonid Karlinsky, Rogerio Feris, David Daniel Cox, Zhangyang Wang, and Yoon Kim. Learning to grow pretrained models for efficient transformer training. In *Proceedings of The Eleventh International Conference on Learning Representations*, 2023.

Qiang Wang, Fuxue Li, Tong Xiao, Yanyang Li, Yinqiao Li, and Jingbo Zhu. Multi-layer representation fusion for neural machine translation. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 3015–3026, 2018a.

Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F Wong, and Lidia S Chao. Learning deep transformer models for machine translation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1810–1822, 2019.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020b.

Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E Gonzalez. Skipnet: Learning dynamic routing in convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 409–424, 2018b.

Ziheng Wang, Jeremy Wohlwend, and Tao Lei. Structured pruning of large language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6151–6162, 2020c.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *Proceedings of International Conference on Machine Learning*, pages 11080–11090. PMLR, 2021.

Felix Wu, Angela Fan, Alexei Baevski, Yann Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *Proceedings of International Conference on Learning Representations*, 2018a.

Felix Wu, Angela Fan, Alexei Baevski, Yann Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *Proceedings of International Conference on Learning Representations*, 2019.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *Proceedings of International Conference on Learning Representations*, 2021.

Zhanghao Wu, Zhijian Liu, Ji Lin, Yujun Lin, and Song Han. Lite transformer with long-short range attention. In *Proceedings of International Conference on Learning Representations (ICLR)*, 2020.

Zuxuan Wu, Tushar Nagarajan, Abhishek Kumar, Steven Rennie, Larry S Davis, Kristen Grauman, and Rogerio Feris. Blockdrop: Dynamic inference paths in residual networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8817–8826, 2018b.

Tong Xiao, Yinqiao Li, Jingbo Zhu, Zhengtao Yu, and Tongran Liu. Sharing attention weights for fast transformer. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*, pages 5292–5298, 2019.

Tong Xiao, Junhao Ruan, Bei Li, Zhengtao Yu, Min Zhang, and Jingbo Zhu. Ordinary differential equations in vision and language. *Authorea Preprints*, 2026.

Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1492–1500, 2017.

Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2246–2251, 2020.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, pages 10524–10533, 2020.

Canwen Xu and Julian Mcauley. A survey on dynamic neural networks for natural language processing. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 2325–2336, 2023.

Chen Xu, Bojie Hu, Yanyang Li, Yuhao Zhang, Shen Huang, Qi Ju, Tong Xiao, and Jingbo Zhu. Stacked acoustic-and-textual encoding: Integrating the pre-trained models into speech translation encoders. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2619–2630, 2021a.

Chen Xu, Rong Ye, Qianqian Dong, Chengqi Zhao, Tom Ko, Mingxuan Wang, Tong Xiao, and Jingbo Zhu. Recent advances in direct speech-to-text translation. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23): Survey Track*, pages 6796–6804, 2023a.

Chen Xu, Yuhao Zhang, Chengbo Jiao, Xiaoqian Liu, Chi Hu, Xin Zeng, Tong Xiao, Anxiang Ma, Huizhen Wang, and Jingbo Zhu. Bridging the granularity gap for acoustic modeling. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10816–10833, 2023b.

Hongfei Xu, Qiuhui Liu, Josef van Genabith, Deyi Xiong, and Jingyi Zhang. Lipschitz constrained parameter initialization for deep transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 397–402, July 2020.

Peng Xu, Xiatian Zhu, and David A Clifton. Multimodal learning with transformers: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023c.

Zenan Xu, Daya Guo, Duyu Tang, Qinliang Su, Linjun Shou, Ming Gong, Wanjun Zhong, Xiaojun Quan, Daxin Jiang, and Nan Duan. Syntax-enhanced pre-trained model. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5412–5422, 2021b.

Baosong Yang, Zhaopeng Tu, Derek F Wong, Fandong Meng, Lidia S Chao, and Tong Zhang. Modeling localness for self-attention networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4449–4458, 2018.

Zhengyuan Yang, Linjie Li, Kevin Lin, Jianfeng Wang, Chung-Ching Lin, Zicheng Liu, and Lijuan Wang. The dawn of lmms: Preliminary explorations with gpt-4v (ision). *arXiv preprint arXiv:2309.17421*, 2023a.

Zi Yang, Samridhi Choudhary, Siegfried Kunzmann, and Zheng Zhang. Quantization-aware and tensor-compressed training of transformers for natural language understanding. *arXiv preprint arXiv:2306.01076*, 2023b.

Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 1480–1489, 2016.

Rong Ye, Mingxuan Wang, and Lei Li. End-to-end speech translation via cross-modal progressive training. *arXiv preprint arXiv:2104.10380*, 2021.

Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549*, 2023.

Yaodong Yu, Sam Buchanan, Druv Pai, Tianzhe Chu, Ziyang Wu, Shengbang Tong, Benjamin D Haeffele, and Yi Ma. White-box transformers via sparse rate reduction. *arXiv preprint arXiv:2306.01129*, 2023.

Seniha Esen Yuksel, Joseph N Wilson, and Paul D Gader. Twenty years of mixture of experts. *IEEE transactions on neural networks and learning systems*, 23(8):1177–1193, 2012.

Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions? In *Proceedings of International Conference on Learning Representations*, 2019.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.

Biao Zhang, Deyi Xiong, and Jinsong Su. Accelerating neural transformer via an average attention network. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1789–1798, 2018.

Biao Zhang, Ivan Titov, and Rico Sennrich. Improving deep transformer with depth-scaled initialization and merged attention. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 898–909, 2019.

Zhuosheng Zhang, Yuwei Wu, Junru Zhou, Sufeng Duan, Hai Zhao, and Rui Wang. Sg-net: Syntax-guided machine reading comprehension. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 9636–9643, 2020.

Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.

Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. Bert loses patience: Fast and robust inference with early exit. *Advances in Neural Information Processing Systems*, 33:18330–18341, 2020.

Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman and Hall/CRC, 2012.

Barret Zoph and Quoc Le. Neural architecture search with reinforcement learning. In *Proceedings of International Conference on Learning Representations*, 2016.